An Adaptive Hybrid Network Security Framework Using Machine Learning and Quantum Assisted Key Distribution

Course Code: ICE496C
Course Title: Capstone Project

Atya Tahsin Orny
ID: 2023-1-50-007

Fihima Tajrian
ID: 2023-1-50-006

Mahamudul Hasan Pramanik
ID: 2023-1-50-013

Kaniz Fatima
ID: 2023-1-50-014

A Capstone project report submitted in partial fulfillment of the requirements for the degree of Bachelor of Information and Communications Engineering



Department of Computer Science and Engineering
East West University, Dhaka-1212, Bangladesh

2026

Declaration

We, **Atya Tahsin Orny**, **Fihima Tajrian**, **Mahamudul Hasan Pramanik**, and **Kaniz Fatima**, hereby declare that the work presented in this capstone project report is the outcome of the investigation performed by us under the supervision of **Dr. Maheen Islam**, Chairperson, Associate Professor, Department of Computer Science and Engineering, East West University. We also declare that no part of this project has been or is being submitted elsewhere for the award of any degree or diploma, except for publication.

Countersigned

. . . . . .
Dr. Maheen Islam
Supervisor

Signature

. . . . . .
Atya Tahsin Orny
2023-1-50-007

Signature

. . . . . .
Fihima Tajrian
2023-1-50-006

Signature

. . . . . .
Mahamudul Hasan Pramanik
2023-1-50-013

Signature

. . . . . .
Kaniz Fatima

2023-1-50-014

Letter of Acceptance

The capstone project report entitled "An Adaptive Hybrid Network Security Framework Using Machine Learning and Quantum Assisted Key Distribution" is submitted by Atya Tahsin Orny, Fihima Tajrian, Mahamudul Hasan Pramanik, and Kaniz Fatima to the Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh. It is accepted for the partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering.

Board of Examiners

1.
Dr. Maheen Islam (Capstone Project Supervisor)
Chairperson, Associate Professor
Department of Computer Science and Engineering
East West University

2.
Dr. Maheen Islam (Chairperson)
Associate Professor
Department of Computer Science and Engineering
East West University

Abstract

The rapid growth of IoT has made communication networks larger and, in many cases less secure. At the same time, future quantum computers may be powerful enough to break traditional security method such as RSA and elliptic-curve cryptography. To deal with these challenges, this project proposes an Adaptive Hybrid Network Security Feamework. The proposed framework combines Machine Learning-based instruction detection with selective Quantum Key distribution (ML-QKD). To test the framework, used three different datasets: X-IIoTD (Australia, 2021), HQC-2026 (Bangladesh, 2026), and CIC-UNSW-NB15 (Canada, 2024). A total of twelve supervised ML models were trained and evaluated on each dataset. Among them, LightGBM gave the best F1-scores of 0.9971, 0.8082, and 0.9580 respectively. The framework also includes a three-level Adaptive Risk Classification Engine that checks each network flow. Based on the predicted probability of an attack, each flow is placed into one of three risk: LOW, MEDIUM, and HIGH. Only the flows identified as HIGH-risk are allowed to activate the simulated secure session. This selective method helps to reduce the extra overhead of QKD compared to using QKD for all network traffic. The overhead was reduced by around 51.6% to 81.5% across the three datasets. The BB84 protocol was tested under clean, low-eavesdrop, and high-eavesdrop conditions. In the clean channel condition, the Quantum Bit Error Rate (QBER) remained almost zero. When eavesdropping was present, the QBER stayed above the 11% security threshold, which indicates that the communication was not secure. Another multiclass ML model was used to identify the specific attack type. It checks each HIGH-risk flow and determines which attack category it belongs to. After identifying the attack, the response engine recommends a suitable security action to handle that particular attack. The SHAP analysis showed that Fwd Seg Size Min was the most important feature. It ranked first in both the HQC-2026 and CIC-UNSW-NB15 datasets. This shows that important traffic features can remain consistent across different datasets. Overall, the framework helps to reduces the cost and overhead of post-quantum security. Because of this, it can be useful for providing quantum-resilient protection to organization with limited resource, such as small healthcare providers and municipal utility services.

Acknowledgments

This project was successfully completed in the Department of Computer Science and Engineering at East West University, Bangladesh. First of all, we would like to thank Almighty God for endless blessings, strength, and guidance throughout this journey. We are truly grateful for the opportunity and perseverance to complete this research successfully. We would like to express our heartfelt gratitude to our supervisor, Dr. Maheen Islam. We sincerely appreciate her valuable guidance, encouragement, and continuous support throughout this project. Her helpful feedback, patience, and careful attention to detail helped us improved our research. She reviewed our work at every stage and guided us whenever we faced difficulties. She also encouraged us to identify and resolve the data leakage issues that we found during the early stage of the project. Her mentorship played a vital role in the successful completion of this project. We are grateful and feel fortunate to have worked under her supervision.

We would also like to express our gratitude to the Department of Computer Science and Engineering at East West University. They provided us supportive academic environment and the necessary guidance during this project. Their review and feedback helped us improved our work from proposal stage to completion. Finally, we would like to thank our families and friends for their constant support and encouragement. Their support helped us stay motivated throughout the project, especially during late nights of model training, testing and report writing.

Atya Tahsin Orny
Fihima Tajrian
Mahamudul Hasan Pramanik
Kaniz Fatima

Table of Contents

Declaration 2
Letter of Acceptance 3
Abstract 4
Acknowledgments 5
Table of Contents 6
List of Figures 8
List of Tables 10
List of Algorithms 11
List of Acronyms 12
Chapter 1 Introduction 13
1.1 Background 13
1.2 Problem Statement and Analysis 14
1.3 Research Objectives 15
1.4 Project Contributions 16
1.5 Challenges 17
Chapter 2 Related Works 18
2.1 Summary 25
Chapter 3 Methodology 30
3.1 Datasets 34
3.1.1 Dataset Collection 34
3.1.2 Dataset Exploration 35
3.1.3 Dataset Preprocessing 37
3.1.4 Research Environment and Devices 38
3.2 Method 39
3.2.1 Proposed Models' Architecture and Mathematical Formulation 40
3.2.2 Performance Evaluation Metrics 57
3.2.3 Classical Encryption versus Quantum Key Distribution 58
3.2.4 Adaptive Risk Classification Engine 63
3.2.5 BB84 QKD Simulation 64
3.2.6 Attack-Type-Aware Response Recommendation Engine 66
3.2.7 Experiment Setup 68
3.3 Overall Work Procedure and Rationale 69
3.4 Summary 70

Chapter 4 Results and Discussion 71
4.1 X-IIoTID (Australia) — Results 71
4.2 HQC-2026 (Bangladesh) — Results 82
4.3 CIC-UNSW-NB15 (Canada) — Results 93
4.4 Cross-Dataset Comparison 103
4.5 Impact of Societal, Health, Safety and Ethical Issues 112
4.6 Discussion 114
4.7 Summary 114
Chapter 5 Conclusion 115
5.1 Overall Contributions 115
5.2 Future Works 116
5.3 Limitations 117
Bibliography 118

List of Figures

3.1 Framework Overview - Adaptive ML-Guided BB84 QKD Pipeline 30
3.2 Architecture of Random Forest 41
3.3 Architecture of Gradient Boosting 42
3.4 Architecture of XGBoost 43
3.5 Architecture of Support Vector Machine 44
3.6 Architecture of LightGBM 46
3.7 Architecture of MLP Neural Network 50
3.8 Architecture of CatBoost 51
3.9 Architecture of Extra Trees, AdaBoost, and Naive Bayes 53
3.10 Architecture of Stacking Classifier (Hybrid) 55
3.11 Architecture of Voting Ensemble (Hybrid) 56
3.12 BB84 QKD Protocol — Step-by-Step Simulation Flow 64
4.1 X-IIoTID - Exploratory Data Analysis 71
4.2 X-IIoTID - Model Performance Comparison 74
4.3 X-IIoTID - Confusion Matrices 75
4.4 X-IIoTID - ROC Curves 75
4.5 X-IIoTID - SHAP Feature Importance (LightGBM) 76
4.6 X-IIoTID - Adaptive Risk Engine and BB84 QKD Results 78
4.7 X-IIoTID - Attack-Type-Aware Response Recommendation 80
4.8 X-IIoTID - Detection and Response Routing Pipeline 81
4.9 HQC-2026 - Exploratory Data Analysis 82
4.10 HQC-2026 - Model Performance Comparison 85
4.11 HQC-2026 - Confusion Matrices 86
4.12 HQC-2026 - ROC Curves 86
4.13 HQC-2026 - SHAP Feature Importance (LightGBM) 87
4.14 HQC-2026 - Adaptive Risk Engine and BB84 QKD Results 89
4.15 HQC-2026 - Attack-Type-Aware Response Recommendation 91
4.16 HQC-2026 - Detection and Response Routing Pipeline 92
4.17 CIC-UNSW-NB15 - Exploratory Data Analysis 93
4.18 CIC-UNSW-NB15 - Model Performance Comparison 96
4.19 CIC-UNSW-NB15 - Confusion Matrices 97
4.20 CIC-UNSW-NB15 - ROC Curves 97
4.21 CIC-UNSW-NB15 - SHAP Feature Importance (LightGBM) 98
4.22 CIC-UNSW-NB15 - Adaptive Risk Engine and BB84 QKD Results (3-Panel) 100

4.23 CIC-UNSW-NB15 - Attack-Type-Aware Response Recommendation 101
4.24 CIC-UNSW-NB15 - Detection and Response Routing Pipeline 102
4.25 CIC-UNSW-NB15 - BB84 QKD Protocol Step-by-Step Pipeline 102
4.26 Cross-Dataset - Model Performance Comparison..... 104
4.27 Cross-Dataset - QKD Overhead and Risk-Engine Comparison..... 109
4.28 Cross-Dataset - SHAP Feature Importance Comparison 109
4.29 Cross-Dataset - Radar Chart of Best Models 110
4.30 Cross-Dataset - Precision-Recall Curves 110
4.31 Cross-Dataset - Confusion Matrices 111
4.32 Cross-Dataset - Threshold Sensitivity Analysis..... 111

List of Tables

2.1 Summary of Related Works 25
3.1 Dataset Overview - X-IIoTID, HQC-2026, CIC-UNSW-NB15 35
3.2 X-IIoTID - Attack-Type Distribution 35
3.3 HQC-2026 - Label Distribution 36
3.4 CIC-UNSW-NB15 - Attack-Category Distribution 36
3.5 Twelve Model Configurations and Hyperparameters 40
3.6 Adaptive Risk Tier Definitions 63
3.7 Attack-Type-to-Response Mapping and Rationale 66
4.1 X-IIoTID - Comparison of All 12 ML Models 72
4.2 X-IIoTID - Adaptive Risk Tier Distribution 77
4.3 X-IIoTID - BB84 QKD Simulation Results 77
4.4 X-IIoTID - Multiclass Attack-Type Classification Summary 79
4.5 HQC-2026 - Comparison of All 12 ML Models 83
4.6 HQC-2026 - Adaptive Risk Tier Distribution 88
4.7 HQC-2026 - BB84 QKD Simulation Results 88
4.8 HQC-2026 - Multiclass Attack-Type Classification Summary 90
4.9 CIC-UNSW-NB15 - Comparison of All 12 ML Models 94
4.10 CIC-UNSW-NB15 - Adaptive Risk Tier Distribution 99
4.11 CIC-UNSW-NB15 - BB84 QKD Simulation Results 99
4.12 CIC-UNSW-NB15 - Multiclass Attack-Category Classification Summary 100
4.13 Cross-Dataset Final Summary - Best Model, F1, AUC, Overhead Saved 103
4.14 Why LightGBM Won Dataset Characteristics Matched to Mechanisms 106
4.15 Cross-Dataset - BB84 QKD Simulation Results 108
4.16 Cross-Dataset - Adaptive Risk Engine Summary 108

List of Algorithms

1. Held-Out Test Split with Original Class-Ratio Preservation .....
2. SMOTE-Based Train-Pool Balancing .....
3. Non-Overlapping Stratified 3 x 50k Sampling and Stability Check.....
4. Checkpoint-Safe Per-Model Training (12 Models) .....
5. Merged-Pool Final Training with SVM/Stacking Subset Capping .....
6. Adaptive Risk Tier Classification (LOW / MEDIUM / HIGH) .....
7. BB84 Quantum Key Distribution Simulation.....
8. Hybrid Undersample-SMOTE Multiclass Rebalancing .....
9. Attack-Type-Aware Response Recommendation Mapping .....
10. SHAP-Based Feature Importance Extraction .....

List of Acronyms

IoT	Internet of Things
IIoT	Industrial Internet of Things
ML	Machine Learning
DL	Deep Learning
AI	Artificial Intelligence
QKD	Quantum Key Distribution
BB84	Bennett-Brassard 1984 Protocol
QBER	Quantum Bit Error Rate
SHAP	Shapley Additive explanations
XAI	Explainable Artificial Intelligence
SMOTE	Synthetic Minority Over-sampling Technique
RF	Random Forest
GB	Gradient Boosting
XGB	XGBoost (Extreme Gradient Boosting)
SVM	Support Vector Machine
MLP	Multi-Layer Perceptron
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
F1	F1-Score (harmonic mean of precision and recall)
AES	Advanced Encryption Standard
RSA	Rivest-Shamir-Adleman Cryptosystem
NIST	National Institute of Standards and Technology
PQC	Post-Quantum Cryptography
SCADA	Supervisory Control and Data Acquisition
CSV	Comma-Separated Values

Chapter 1

Introduction

IoT devices are growing rapidly in different industries, which is creating a large and vulnerable communication network. At the same time, the development of quantum computing may create a serious threat to traditional encryption methods. These two challenges, including current cyberattacks and future quantum threats, motivated us to develop an adaptive security framework that combines Machine Learning with Quantum Key Distribution to provide better and more flexible security.

1.1 Background

The Internet of Things (IoT) has changed digital infrastructure in many different sectors. The number of connected IoT devices around the world has grown rapidly and was estimated to reach 18.8 billion devices by the end of 2024 [1]. Industrial systems, smart grids, wearable health devices, and autonomous vehicles continuously generate large amount of network traffic. This traffic needs strong and real-time security. However many IoT devices have limited processing power, memory, and battery capacity, which makes it difficult to use complex cryptographic methods on these devices [2]. IoT networks also face different types of attacks, such as denial-of-service, reconnaissance, and data theft [3]. The 2016 Mirai botnet showed how serious the security risks can be when IoT devices are attacked on a large-scale [4]. Another major concern is the future threat from quantum computing. Shor's algorithm [5] can potentially break widely used encryption methods such as RSA and elliptic-curve cryptography when powerful quantum computers became available. The harvest-now-decrypt-later approach also makes this threat important even today, because attackers can collect encrypted data now and try to decrypt it in the future [6]. Quantum Key Distribution (QKD) provides a security solution based on the principle of quantum physics. The BB84 protocol, introduced by Bennett and Brassard in 1984 [7], can detect eavesdropping by observing changes in the Quantum Bit Error Rate (QBER) [8]. However, using BB84 for every IoT network flow can cause high computational and operational overhead. To solve this problem, this project uses Machine Learning (ML) to identify high-risk network flows. The BB84 protocol is then activated only for these high-risk flows.

1.2 Problem Statement and Analysis

IoT networks mainly face two major types of security threats. The first one is classical cyberattacks, which are already happening today. The second one is the quantum threat to current key-exchange methods. This project focuses on this research gap by testing the same ML-QKD framework across three independent datasets. The study uses twelve ML models, a three-tier risk classification engine, and BB84 simulation. The three datasets differ in their country, collection year, traffic characteristics**,** and class distribution. The datasets used in this study are X-IIoTID, HQC-2026, and CIC-UNSW-NB15. The main goal is to determine whether one adaptive ML-QKD framework can provide consistent performance when it is tested on different and previously unseen types of network traffic.

1. **Does the ML-guided risk engine save QKD overhead consistently across datasets?**

   Yes, savings were high on all three datasets (51.6%, 81.5%, and 78.5%). But the exact amount depended on each dataset's attack rate. So the method works everywhere, but the savings size changes by dataset.

2. **Do the same risk thresholds (0.35 and 0.70) work well across all datasets?**

   The same thresholds were used for all datasets, with no changes. They worked okay overall, but the MEDIUM-risk group size varied a lot (0.1% vs 7.5%). This shows the thresholds may need fine-tuning in future work.

3. **Does the BB84 simulation correctly detect clean vs. eavesdropped channels on every dataset?**

   Yes. On clean channels, the error rate (QBER) was 0.0000 for all datasets. Under heavy eavesdropping, QBER rose to 0.19, well above the 11% safety limit. This means the system correctly caught the attack every time.

1.3 Research Objectives

This project develops and tests an adaptive ML-QKD security framework using three independent network-traffic datasets. The main purpose is to check whether the framework can maintain consistent performance in risk detection, ML model accuracy, and QKD overhead reduction when it is tested on different datasets instead of only one dataset.

- **Get the data ready:** Collect three network-traffic datasets (X-IIoTID, HQC-2026, CIC-UNSW-NB15) from different countries and years. Clean the datasets properly and check them carefully. Remove any features and information that could cause data leakage and affect the final results.
- **Build a safe sampling process:** Take three separate samples of 50,000 rows from each dataset and train all 12 models on each sample. Compare the result to check whether the model performance stay stable. Then combine the three samples into single 150,000-row dataset and train the model again.
- **Find the best model:** For each dataset, test 12 different classifiers such as Random Forest, XGBoost, SVM, and others. Compare their results and select the model that achieves the best F1-score for each dataset.
- **Build a risk-level system:** Create a system that classifies network threats into three risk levels: LOW, MEDIUM, and HIGH, based on the model's prediction. The system should identify which network flows are riskier and apply stronger security only when needed. This helps reduce the effort and overhead compared to using full QKD security for every network flow.
- **Simulate quantum key distribution (BB84):** Build a working model of the BB84 protocol that include photon encoding, sifting, and error-rate checks under clean, low-attack, and high-attack conditions. Also use SHA-256 to provide an additional layer of privacy protection. Then test the BB84 simulation with all three datasets.
- **Detect attack types:** Build a second ML model that identify the specific attack types. Use SMOTE and undersampling techniques to balance the data. Then connect the model to a system that suggests the right security action for each attack type.

- **Explain the model's choices:** Use SHAP to identify which features matter most for each dataset's best model. Then compare these features across all three datasets. This will help us understand whether the models depend on similar network traffic features when making their predictions.
- **Compare everything together:** Compare all three datasets based on model accuracy, QKD savings, risk levels, and SHAP results. Then check whether the the ML-QKD framework work effectively beyond a single dataset.

1.4 Project Contributions

In this project, the framework is tested using three datasets from three different countries. It also includes a comparison of 12 ML models, a sampling process to check the stability of the results, and a two-level BB84 simulation. In addition, an attack-type-based response system is include to suggest suitable security actions for different types of attacks.

- **Testing the Framework on Multiple Datasets:** This project tests on three different datasets from three countries, and checking whether the results generalize.
- **Bigger Model Comparison**: This project compares 12 ML models on all three datasets, giving 36 full result sets.
- **Safer, More Reliable Sampling:** A new sampling method was built using three non-overlapping samples and checks their stability before final training. This helps reduce hidden bias and ignored the overlapping data between the training and testing sets. As a result, the model can be evaluated using separate and unseen data, which helps produce more reliable and accurate results.
- **Detailed BB84 Simulation:** The project includes both a high-level diagram and a step-by-step diagram of the BB84 process. It also tests three eavesdropping conditions on each dataset and confirms the 11% security limit works reliably every time.
- **Smarter Attack Response System:** Besides classifying risk levels, the project inclides a second ML model that identifies specific attack types. This connects to a response system that suggests suitable security actions for each attack type.

- **Early Proof of Consistent Key Features:** SHAP analysis found that "Fwd Seg Size Min" was the top feature in two of the three datasets. This suggests that the model is using an important traffic feature that is not limited to only one dataset.

1.5 Challenges

Throughout the project, we faced some challenges:

1. Different Dataset Formats: X-IIoTID, HQC-2026, and CIC-UNSW-NB15 all had different column names, sizes, and label formats. So, each dataset needed separate cleaning and level-encoding rules. For two datasets, we also handled data imbalance applying SMOTE.

2. Catching Data Leakage: During testing on HQC-2026, we found that some columns like qber, sifted_key_rate, and fidelity, etc. were directly giving away the answer. This created a data leakage problem, which can lead to misleading ML results [9]. So, these columns were removed before evaluating the models.

3. Long Training Times: Training all 12 models on large datasets took a lot of time, especially SVM and Stacking. Stacking took more than 2 hours on one dataset. So both models were limited to a smaller 50,000-row dataset to reduce the training time.

4. Imbalanced Attack Types: Some attack types had very few examples, sometimes less than 100 rows. To solve this problem, a mixed method of undersampling and SMOTE [10] was used, with fixed limits. This helped prevent rare attacks from being ignored and common attacks from dominating the dataset.

5. Google Colab Crashes: Long training sessions were often interrupted by Colab disconnects. So a checkpoint system was built to save each model's progress to Google Drive. This way, training could resume instead of starting over.

Chapter 2

Related Works

Previous studies have explored ML-based intrusion detection and Quantum Key Distribution separately. However, there is still a lack of research that combines both into an adaptive and cost-efficient framework for IoT security. This gap in the existing research motivated us to develop the proposed system.

Sengupta et al. (2026) proposed an adaptive post-quantum cryptography (PQC) framework for 6G vehicular networks. Instead of using a fixed PQC method that increase latency and computation, the context-aware adaptive framework (CAAP) selects suitable algorithms based on real-time conditions. It uses predictive optimization and reinforcement learning to improve the selection process. They also introduced a secure transition protocol prevents downgrade and replay attacks which improving efficiency, reliability, stability, while reducing latency and communication overhead [11].

Al-Kuwari et al. (2026) studied the use of quantum machine learning (QML) to enhance Quantum Key Distribution (QKD) security by addressing the limitations of classical machine learning and conventional cryptography. Earlier studies used methods such as neural networks, decision trees, and GAN-based models but faced issues such as simplified datasets, limited attack scenarios, and poor generalization. Although later research improved detection accuracy, the absence of quantum-enhanced learning motivated the development of hybrid QML solutions [12].

Noureldin Mohamed (2026) proposed an adversarial learning model for intrusion detection in Quantum Key Distribution (QKD) systems. The study focused on side-channel attacks, including time-shift, detector blinding, and photon-number splitting attacks. Using a minimax game framework, the learning-based defender adapts to adversaries, improving secure information retention by balancing detection accuracy and finite-key secret fraction, demonstrating the effectiveness of integrating machine learning with quantum cryptographic security [13].

Chaudhary et al. (2025) reviewed the use of Federated Learning (FL) and Quantum Machine Learning (QML) in Network Intrusion Detection Systems (NIDS). The study highlights some problems with centralized intrusion detection, such as privacy, scalability, and communication overhead. It showed that FL as a privacy-preserving solution and QML as a promising approach for high-dimensional data, while discussing challenges such as non-IID data, resource constraints, and communication efficiency [14].

Aandaram V., Deepalakshmi P., and Vidhya Prakash Rajendran (2026) proposed a Quantum-Assisted Secure Authentication and Intrusion Detection Framework for 5G networks. Their framework combines Quantum Key Distribution (QKD) with a Variational Quantum Classifier for intrusion detection using the CICIDS-2018 dataset. It achieves about 99% detection accuracy, outperforming SVM, Random Forest, and CNN. However the use of benchmark datasets and simulation-based testing may limit its use in real-world network [15].

Zangana, Sallow, and Mustafa (2025) studied quantum-assisted architectures for secure and efficient 6G-enabled IoT networks. Their framework combines Quantum Key Distribution (QKD), Quantum Machine Learning (QML), and entanglement-assisted routing to improve security, throughput, and latency. The study shows that hybrid quantum-classical systems effectively address scalability and cybersecurity challenges while enhancing spectral efficiency, computational speed, and reliability for future 6G IoT networks [16].

Mounagurusamy et al. (2025) proposed an adaptive, quantum-resilient intrusion detection framework for Wireless Sensor Networks (WSNs) and IoT. The framework uses Quantum Key Distribution (QKD) for secure communication with Deep Reinforcement Learning (DQN and PPO) for attack detection. It was tested on the CICIDS2017 dataset, achieved 99.75% accuracy, which was better than traditional methods. However, its practical use is limited by specialized quantum communication infrastructure [17].

Nalavala Ramanjaneya Reddy (2025) studied the growing threat of quantum computing to blockchain security. The study reviewed post-quantum cryptography, Quantum Key Distribution (QKD), hybrid signatures, lattice-based cryptography, and quantum-enhanced blockchain models, highlighting their limitations in providing complete protection. It emphasizes the need for an

integrated framework combining cryptographic techniques, communication protocols, and consensus mechanisms to achieve scalable, quantum-resistant blockchain security [18].

Gunasridharan et al. (2025) proposed a hybrid cybersecurity framework that combines Quantum Machine Learning (QML) and Quantum Key Distribution (QKD) to improve threat detection and secure communication. They used the BB84 protocol with VQC, QSVC, and QNN models. The framework achieved up to 97.75% detection accuracy on different cybersecurity datasets, demonstrating that combining quantum cryptography and QML enhances cyber-threat detection and quantum-resilient security [19].

Biswas and Zoiros (2025) proposed a quantum-secure architecture for protecting optical fiber-based power network SCADA communications using Quantum Key Distribution (QKD). They compared different BB84, B92, E91, and SARG04 protocols with the Qiskit simulator and a PNNL SCADA dataset. The results showed that B92 and E91 performed better than BB84 with higher key match rates and near-zero Quantum Bit Error Rate (QBER), ensuring stronger smart grid security [20].

Kaleem et al. (2025) proposed the Quantum Skyshield architecture for securing low-altitude wireless networks (LAWNs) and BTS communications. The framework combines BB84 Quantum Key Distribution (QKD), post-quantum authentication, and a Grover-inspired threat detection algorithm. Simulations showed reliable 128-bit key generation and up to 89% anomaly detection, outperforming RSA- and ECDSA-based methods and showed improved security for airborne communication networks [21].

Mupila et al. (2026) proposed a quantum-resilient cloud forensic framework integrating Quantum Key Distribution (QKD) and AI to address quantum computing threats. The framework uses BB84-based QKD with Random Forest and LSTM, the model improves anomaly detection and evidence integrity while supporting NIST and ISO 27037 standards through blockchain timestamping. The study also identified scalability, integration, and legal challenges for future forensic security [22].

Al-Khazraji et al. (2025) proposed a secure quantum communication framework combining Entangled Photon Quantum Key Distribution (EP-QKD) and Visible Light Communication (VLC) for 6G smart cities. The framework uses polarization-entangled photons, adaptive QBER optimization, and blockchain-based authentication. This method helps to overcome conventional QKD limitations, achieving higher key generation rates, lower error rates, and stronger resistance to cyberattacks in IoT-based smart city environments [23].

Revathi K and Suganthi K (2025) proposed the Hybrid Rainbow-Kyber Quantum Key Distribution (HRK-QKD) protocol to improved blockchain security against quantum threats. The protocol combines Rainbow cryptography, Kyber lattice-based encryption, and entanglement-assisted QKD. It aims to improve scalability, noise resilience, key generation, and security. Features such as dynamic key synthesis, adaptive key rotation, and quantum noise filtering make it an effective quantum-secure blockchain applications [24].

Shirko and Askar (2023) studied secure key distribution in Quantum Key Distribution (QKD) networks. They highlighted the limitations of classical trusted relay (CTR) technology, including relay vulnerability, key wastage, and limited communication distance. The study also reviewed methods such as measurement-device-independent QKD, quantum repeaters, and SDN-based approaches It identifyed scalability, reliability, and relay failure management as key challenges requiring improved survivability and key management solutions [25].

Majumdar (2026) proposed a multilayered quantum-secured 6G architecture integrating Quantum Key Distribution (QKD), Post-Quantum Cryptography (PQC), and Quantum Machine Learning (QML) to address quantum-era cybersecurity threats. The framework uses HD-QKD and TF-QKD for higher key rates, while QML enables adaptive anomaly detection and security management. The proposed architecture aims to improve secure communication, scalability, and resilience, providing a practical roadmap for future quantum-resistant 6G networks [26].

Madhubala and Bolisetty (2026) proposed an energy-efficient AI-assisted quantum-secure communication framework for scalable and reliable IoT networks. The framework integrates Quantum Key Distribution (QKD) with deep learning to optimize key management, routing, and resource allocation under dynamic network conditions. Their simulation results demonstrated improved energy efficiency, packet delivery ratio, throughput, and reduced latency. The framework also improves protection against both classical and quantum cyber threats for future 5G/6G-enabled IoT networks [27].

Rehman and Alharbi (2025) proposed the Quantum-Enhanced Secure IoT Framework (QESIF) to improve cybersecurity in smart city IoT environments. It integrating Quantum Key Distribution (QKD), a quantum-aware intrusion detection system (Q-IDS), and a hybrid protocol stack. The experimental results demonstrated high attack detection rates up to 98.7%, low latency, improved throughput, and energy efficiency. The study showed QESIF can provide a scalable, lightweight, and quantum-resilient security solution for next-generation smart cities [28].

Rubio García et al. (2025) presented a triple-hybrid network security framework. It combines classical cryptography, post-quantum cryptography, and Quantum Key Distribution (QKD) to enhance communication security against quantum threats. The proposed approach enables seamless deployment in existing protocols such as TLS and IPsec with minimal performance overhead. The framework improved crypto-agility, resilience, and long-term protection, making it a practical solution for next-generation quantum-secure communication networks [29].

Malik and Alsanoosy (2024) proposed a novel hybrid intrusion detection framework that combines Peer-to-Peer (P2P) networking with Quantum Machine Learning (QML) to enhance the security of smart traffic systems. The decentralized P2P architecture improves scalability and resilience, while QML enables accurate real-time anomaly detection. The experimental results demonstrated superior detection accuracy, reduced false positives, and enhanced computational efficiency compared with conventional machine learning-based intrusion detection approaches [30].

Chalampalem et al. (2025) proposed a hybrid Software Defined Radio (SDR) architecture integrated with Quantum Machine Learning (QML) and Offset Quadrature Amplitude Modulation (OQAM) to enable adaptive and secure wireless communication. The framework combines Quantum Neural Networks (QNN), Quantum Support Vector Machines (QSVM), and Quantum Reservoir Computing (QRC) for real-time modulation classification and spectrum management. The simulation results achieved up to 96.3% classification accuracy with approximately 16.2 ms latency, demonstrating improved spectral efficiency, robustness, and quantum-safe communication for future 5G/6G networks [31].

Purohit and Vyas (2025) reviewed the integration of Quantum Machine Learning (QML) with Quantum Key Distribution (QKD) to enhance secure quantum communication. The study examined recent advancements in QML-based key generation, eavesdropping detection, and QKD security analysis while identifying challenges such as hardware limitations, scalability, and computational overhead. It concluded that QML can significantly improve the efficiency, adaptability, and robustness of quantum cryptographic systems, although further practical implementation is required [32].

Shukla et al. (2026) proposed a quantum-enhanced cybersecurity framework for Internet of Things (IoT) environments by integrating Quantum Key Distribution (QKD), Quantum Machine Learning (QML), and post-quantum cryptographic techniques. The study highlighted improved resilience against quantum-enabled cyberattacks, secure key management, and intelligent threat detection. It concluded that combining quantum technologies with conventional security mechanisms provides a scalable and future-ready solution for safeguarding next-generation IoT networks [33].

Inakoti et al. (2025) proposed an AI-enhanced hybrid framework that integrates Machine Learning and Deep Learning techniques with quantum cryptography to improve post-quantum security. The study combines Reinforcement Learning, Federated Learning, GANs, CNNs, RNNs, GNNs, and QNNs to enhance Quantum Key Distribution, authentication, scalability, and cyberattack resilience. Experimental results indicate improved computational efficiency and 93% accuracy, demonstrating the framework's potential for secure and scalable quantum communication [34].

Mallick et al. (2026) conducted a systematic review of quantum-secure communication, examining the evolution from classical cryptographic techniques to AI- and learning-based approaches for cyber-physical and IoT systems. The study highlights the integration of quantum cryptography, machine learning, and post-quantum security mechanisms to address emerging cyber threats. It concludes that intelligent, quantum-enabled frameworks can significantly enhance secure, scalable, and resilient communication for next-generation networks [35].

Komal and Li (2026) presented a comprehensive review of Intrusion Detection Systems (IDS) for Internet of Things (IoT) environments, covering studies from 2021 to early 2026. The authors examined conventional, machine learning, and deep learning-based IDS approaches, while highlighting emerging techniques such as Federated Learning, Explainable AI, TinyML, Transformers, and Quantum Machine Learning. They also discussed benchmark datasets, evaluation metrics, key challenges, and future research directions for secure, scalable IoT networks [36].

Akinbowale, Adesina, Zerihun, and Mashigo (2026) examined machine-learning-based intrusion detection for IoT environments using a large Kaggle dataset. The study compared Decision Tree (DT), Random Forest (RF), and Support Vector Machine (SVM) models. DT achieved the highest accuracy (99.36%), followed closely by RF (99.27%), while SVM achieved 80.08%. The findings demonstrate the effectiveness of tree-based models, although class imbalance remains a challenge for detecting rare attacks [37].

2.1 Summary

Table 2.1 summarizes all twenty-seven papers discussed above, listing each paper's proposed models and its principal limitation.

Table 2.1: Summary of Related Works

Author & Year	Dataset	Feature Selection Algorithm	Model / Works	Limitations
Sengupta et al. (2026)	Simulated vehicular network data	Not explicitly mentioned (context-aware selection)	Adaptive PQC framework (CAAP) using reinforcement learning	High computational complexity, real-world deployment challenges
Al-Kuwari et al. (2026)	QKD datasets (various prior works used simplified datasets)	Not clearly specified	Hybrid Quantum Machine Learning (QML) for QKD security	Limited real-world datasets, poor generalization in earlier models
Noureldin Mohamed (2026)	QKD system data (attack-based scenarios)	Not specified	Adversarial learning (minimax game model) for intrusion detection	Focused mainly on specific side-channel attacks, complexity in training
Chaudhary et al. (2025)	Multiple benchmark NIDS datasets	Not specified (survey-based)	Federated Learning + QML for intrusion detection	Non-IID data issues, communication overhead, resource constraints
Aandaram V.(2026)	CICIDS-2018	Preprocessing feature selection	Quantum ML IDS (VQC), QKD	Scalability and real-world deployment are limited by simulation-based testing

Author & Year	Dataset	Feature Selection Algorithm	Model / Works	Limitations
Zangana, Sallow, and Mustafa (2025)	Simulation	Not specified	QKD, QML, quantum routing	There is no real World deployment validation for conceptual and simulation-based approaches
Mounagurusamy et al. (2025)	CICIDS2017	Not specified	QKD + DRL (DQN, PPO)	Requires specialized quantum infrastructure limiting scalability in IoT environments
Nalavala Ramanjaneya Reddy (2025)	Not specified	Not specified	Survey (Blockchain + quantum security)	Lacks practical implementation and experimental validation of unified framework
Gunasridharan et al. (2025)	HIKARI, phishing, URL datasets	Not specified	VQC, QSVC, QNN + BB84	Multiple datasets used but lack unified evaluation across tasks
Biswas and Zoiros (2025)	PNNL SCADA dataset	Not specified	BB84, B92, E91, SARG04	Simulation-based QKD protocols may not reflect real-world SCADA constraints
Kaleem et al. (2025)	Simulation	Not specified	BB84 + OTS + HMAC + Grover-based detection	Simulation-focused evaluation lacks real-world deployment and environmental testing

Author & Year	Dataset	Feature Selection Algorithm	Model / Works	Limitations
Mupila et al. (2026)	Cloud forensic logs, blockchain timestamp data	Not explicitly specified (implicit feature extraction via ML models)	QKD + AI-based forensic framework using BB84, Random Forest, and LSTM	Scalability issues, integration complexity, and lack of legal acceptance
Al-Khazraji et al. (2025)	Simulation-based smart city/6G communication data	Not specified (focus on QBER optimization techniques)	EP-QKD integrated with VLC and blockchain authentication	High implementation complexity, infrastructure dependency, and scalability challenges
Revathi K and Suganthi K (2025)	QDataSet	Not explicitly defined (uses lattice and multivariate mathematical structures)	HRK-QKD protocol combining Rainbow, Kyber, and QKD	Complexity in real-world deployment and integration with existing blockchain systems
Shirko and Askar (2023)	Literature-based	Not applicable	Analysis of QKD networks, CTR, quantum repeaters, and SDN-based approaches	Limited focus on relay failure handling and survivability mechanisms
Majumdar (2026)	Not specified	Not specified	Proposed a multilayered 6G security architecture integrating QKD, PQC, HD-QKD, TF-QKD, and QML	No experimental validation or benchmark dataset reported.
Madhubala and Bolisetty (2026)	Simulation dataset	Not specified	AI-assisted QKD framework using Deep Learning for routing, key management, and resource allocation	Evaluated only in simulation; real-world deployment not demonstrated.

Author & Year	Dataset	Feature Selection Algorithm	Model / Works	Limitations
Rehman and Alharbi (2025)	Smart City IoT simulation	Not specified	Quantum-Enhanced Secure IoT Framework (QESIF) with QKD and Quantum-aware IDS	Performance validated in simulated environment only.
Rubio García et al. (2025)	Not specified	Not specified	Triple-hybrid framework combining Classical Cryptography, PQC, and QKD	No public dataset or implementation results on large-scale networks.
Malik and Alsanoosy (2024)	Smart traffic network dataset	Not specified	Hybrid P2P-based Quantum Machine Learning Intrusion Detection System	Limited evaluation in smart traffic scenario only.
Chalampalem et al. (2025)	Simulation data	Not specified	SDR architecture integrating QNN, QSVM, QRC, and OQAM	Tested only in simulation; quantum hardware implementation absent.
Purohit and Vyas (2025)	Review paper (No dataset)	Not applicable	Review of QML-assisted QKD techniques	Practical implementation constrained by quantum hardware limitations.
Shukla et al. (2026)	Not specified	Not specified	Quantum-enhanced IoT security framework integrating QKD, QML, and PQC	Primarily conceptual framework with limited experimental validation.

Author & Year	Dataset	Feature Selection Algorithm	Model / Works	Limitations
Inakoti et al. (2025)	Simulation dataset	Not specified	AI-enhanced hybrid framework using RL, FL, GAN, CNN, RNN, GNN, and QNN with QKD	Experimental evaluation limited to simulation environment.
Mallick et al. (2026)	Review papers (Systematic Literature Review)	Not applicable	Systematic review of quantum-secure communication using AI and PQC	No experimental model or dataset used.
Komal and Li (2026)	Surveyed IDS datasets (2021–2026)	Not applicable	Comprehensive review of ML, DL, FL, XAI, TinyML, Transformer, and QML-based IDS	Review only; no proposed IDS model or experimental validation.
Akinbowale, Adesina, Zerihun, and Mashigo (2026)	Kaggle IoT intrusion dataset	Not specified	Compared DT, RF, and SVM for IoT intrusion detection	Class imbalance reduced detection performance for rare attacks.

The research findings demonstrate how traditional classification systems have developed into modern systems which provide explainable results through their interpretable architectural designs. The majority of the studies face multiple challenges because they do not have enough data and their models overfit and they need to be tested through external validation and they provide only limited capacity to interpret results. The new developments in SHAP and LIME-based explainability systems offer effective solutions which help to resolve the challenges associated with black-box systems. Our research combines ML-based adaptive risk classification with BB84 QKD to create a solution which solves both intrusion detection and post-quantum security challenges.

Chapter 3

Methodology

The following sections describe the full technical pipeline of the proposed framework, from data collection and preprocessing to model training, risk classification, BB84 QKD simulation, and final system integration.

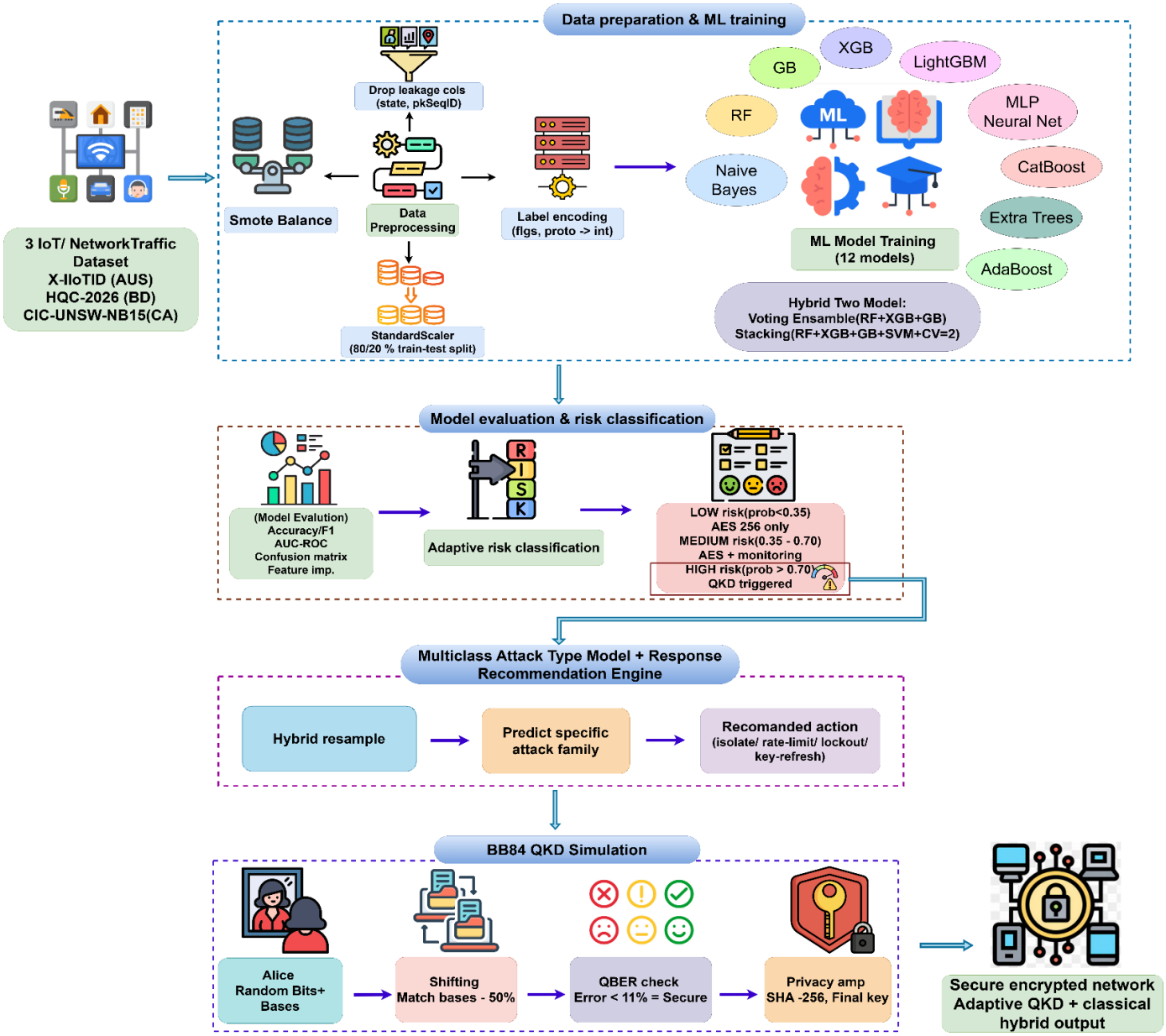


Figure 3.1: Framework Overview Updated Adaptive ML-Guided BB84 QKD Pipeline

The same pipeline was followed for all three datasets in the same order. This was done to make sure that the result from X-IIoTID, HQC-2026, and CIC-UNSW-NB15 could be compared fairly. The whole process divided into several steps, including data collection, data preprocessing, model training, adaptive risk classification, multiclass attack detection and response, and BB84 QKD simulation. Each of these steps is described below along with the reason for using it.

Data Collection

The process started by loading the raw datasets and checking their basic structure before any transformation was applied. Class distributions and attack-type distributions were computed, and the data was checked for missing values. This initial analysis was important because it helped us understand the baseline imbalance. Knowing the level of imbalance was necessary before applying the latter balancing techniques, as it helped us decide whether the data needed more balancing or not.

Data Preprocessing

After understanding the basic characteristics of each dataset, the data was cleaned and prepared for the model. Columns that could leak the label directly identifiers were dropped. Categorical features were encoded where necessary, and skewed numeric columns were log1p-transformed using the detailed procedure in Section 3.1.3. This cleaning step was performed before any other step so that every subsequent stage of the pipeline sampling, training, and evaluation operated on trustworthy, leakage-free features rather than on artifacts of the raw data.

After cleaning the features, a separate test set was created using the original class distribution of each dataset. This test set was not resampled. This was intentionally done, so the final evaluation will simulate a more realistic real-world situation where attack traffic is typically much lower than the normal or benign traffic. If the test data had been artificially balanced, the performance of the model may seem better than it actually is. Hence, SMOTE was used only for training set, once the test set was separated. The test set was not altered in any way to ensure a representative sample of real-world traffic for the final assessment.

To ensure that the balanced training pool does not include hidden bias because of a certain subset of rows, three disjoint sets of 50,000 rows (25,000 benign and 25,000 attack) were sampled from the pool using non-overlapping stratified sampling. Each of the 12 candidate models was then trained separately on each of the three samples and their F1-scores obtained. A stability check was subsequently performed in which the standard deviation of F1-score and class ratio from the three samples was calculated, ensuring that the sampling procedure did not add any relevant bias and that the following results are representative of the modeling approach and not just of the 50,000 rows sampled.

Model Training

After sampling stability, the three 50,000-row samples were consolidated to a 150,000-row sample pool and all 12 models were retrained on this sample pool. Among these 12 trained models, the one with the best F1 score on a held out test set was chosen to be the binary classifier used to power the risk engine. The selection was done using F1 score and not accuracy. While the number of "benign" flows can dominate the number of "malignant" flows, accuracy is misleading on such an imbalanced held-out set, a model that simply predicts "benign" could achieve a high accuracy and be very unhelpful in practice.

Adaptive Risk Classification

The model with the best performance was then used to classify every flow in the test set as a LOW, MEDIUM, or HIGH risk level by applying fixed thresholds. The tiered response system ensures that where there is a risk of a downstream flood, the more costly security measures will be employed, rather than taking the same response action to every flood.

The best model was used to compute the SHAP values that enabled recognizing and ranking the features that most heavily impact the model's decisions instead of it being a black box. Let's not forget that this step gives transparency into why a certain flow was deemed high risk, which is crucial for both the trust in the system and to identify any unexpected behaviors.

Multiclass Attack Detection and Response

In this process, if the flow has been classified as HIGH-risk, a secondary multiclassifier was trained to detect the Attack of Interest. Handling the natural imbalance in the types of attacks, by employing a hybrid undersampling and SMOTE rebalancing strategy. After identifying an attack family for a specific flow, it was mapped to a concrete, actionable security response, following a framework. transforming an raw classification output into a security response an operator can take action on.

BB84 QKD Simulation

The flows were also simulated with a full BB84 quantum key distribution protocol, which includes encoding, sifting, estimating the QBER and privacy amplification, for three channel conditions: clean, low-eavesdrop, and high-eavesdrop. Restricting this simulation to HIGH-risk flows, instead of simulating every flow in the dataset, shows the adaptive design of the overall approach, whereby the more demanding quantum-layer protection simulation is reserved for the traffic that is most likely to benefit from it.

Finally, all metrics, predictions, and serialized figures generated across these stages were persisted to a per-dataset results file, enabling the cross-dataset comparison presented in Chapter 4.

3.1 Datasets

In this section, we describe the data and other resources we use in our study. It provides an overview of the nature of datasets used, their key features, and how the data was cleaned prior to model construction. It also explains about the experimental equipment and software used for conducting experiments.

3.1.1 Dataset Collection

We used three separate network traffic datasets to test how well our idea works in different situations. These datasets came from different countries, years, and sources, letting us check if our method works across all kinds of data.

X-IIoTID (Australia, 2021): This dataset detects attacks on Industrial IoT systems. It has 820,834 records with 68 features. It covers 19 attack types, like reconnaissance, brute-force, and insider attacks.

HQC-2026 (Bangladesh, 2026): Collected by MIST, Bangladesh, this dataset has 233,844 records with 88 features. It includes quantum communication features like QBER, sifted key rate, and fidelity. These features leaked the answer directly, so we removed them before training.

CIC-UNSW-NB15 (Canada, 2024): This is a reprocessed version of UNSW-NB15 using CICFlowMeter. It has 447,915 records with 77 features after merging data and labels. It includes benign traffic and nine attack types, giving us a third independent test case.

Table 3.1: Dataset Overview (X-IIoTID, HQC-2026, CIC-UNSW-NB15)

Characteristic	X-IIoTID	HQC-2026	CIC-UNSW-NB15
Country / Year	Australia, 2021	Bangladesh, 2026	Canada, 2024
Raw Shape	820,834 x 68	233,844 x 88	447,915 x 77
Features After Preprocessing	77	78	66
Attack Categories	18 attack types + benign	13 attack types + benign	9 attack categories + benign
Original Attack Ratio	48.7% attack	24.0% attack	20.0% attack
Class-Balancing Method	None needed (near-balanced)	SMOTE	SMOTE
Log1p Transform Applied	No	Yes (53 skewed columns)	Yes (53 of 66 columns)

3.1.2 Dataset Exploration

Before any cleaning the distribution of classes and attack types, in each dataset was calculated to get an idea of how imbalanced the data was so the pipeline could be prepared for the challenge.

Table 3.2 presents the distribution of the attacks found for X-IIoTID in the analysis.

Table 3.2: X-IIoTID Attack-Type Distribution

Attack Type	Count	Share
Normal	421,417	51.3%
RDOS	141,261	17.2%
Scanning_vulnerability	52,852	6.4%
Generic_scanning	50,277	6.1%
BruteForce	47,241	5.8%
MQTT_cloud_broker_subscription	23,524	2.9%
Discovering_resources	23,148	2.8%
Exfiltration	22,134	2.7%
insider_malicious	17,447	2.1%
Other (10 rarer classes)	20,533	2.5%

Table 3.3: HQC-2026 Label Distribution

Label	Count	Share
Benign	177,693	76.0%
Bot	20,226	8.7%
Infilteration	15,620	6.7%
FTP-BruteForce	5,826	2.5%
SSH-Bruteforce	5,442	2.3%
DoS attacks-SlowHTTPTest	2,755	1.2%
DoS attacks-Slowloris	2,449	1.0%
Other (7 rarer classes)	3,833	1.6%

Table 3.4 summarizes the CIC-UNSW-NB15 attack-category distribution.

Table 3.4: CIC-UNSW-NB15 Attack-Category Distribution

Category	Count	Share
Benign	358,332	80.0%
Exploits	30,951	6.9%
Fuzzers	29,613	6.6%
Reconnaissance	16,735	3.7%
Generic	4,632	1.0%
DoS	4,467	1.0%
Shellcode	2,102	0.5%
Backdoor	452	0.1%
Analysis	385	0.1%
Worms	246	0.1%

3.1.3 Dataset Preprocessing

This section covers how each dataset was cleaned, which columns were removed or changed, and how the test set and class imbalance were handled, since these choices affect how trustworthy the Chapter 4 results are.

A) X-IIoTID: Removed 5 unnecessary columns such as Date, Timestamp, IPs, duplicate label. Trained the Encoded Protocol and Service with training data. Cleaned duplicate and bad records, resulting in 809,249 flows and 77 features. The classes were almost balanced (51.4% normal, 48.6% attack) and thus no SMOTE was required.

B) HQC-2026: Found 9 columns like qber, sifted_key_rate, fidelity that leaked the answer directly, so they were removed. After removing duplicates, 78 features remained. 53 features were highly skewed, so a log1p transform was applied to stabilize training.

C) CIC-UNSW-NB15: Removed 10 near-zero-variance columns, leaving 66 features. Applied a similar transform to fix skewed values. Split the dataset into train/test first, keeping the same class balance, before any transformation to avoid leakage.

SMOTE: Balancing SMOTE was used only on HQC-2026 and CIC-UNSW-NB15 (not needed for X-IIoTID). It creates new minority-class samples between real data points instead of copying rows. HQC-2026 grew from 186,976 to 284,192 rows; CIC-UNSW-NB15 grew from 358,332 to 573,332 rows.

Sampling: Three non-overlapping 50,000-row samples (25,000 normal + 25,000 attack) were drawn from each balanced pool to test sampling stability. All three were merged into a 150,000-row pool for final training.

Scaling StandardScaler was fit on training data only and applied to both train and test sets, used for models like SVM, MLP, and Naive Bayes. This avoids leaking test-set information into training.

3.1.4 Research Environment and Devices

The following software and hardware environment was used for all data preparation, model training, and evaluation:

- Processor: Standard cloud-provisioned CPU runtime (Google Colab Pro, high-RAM instance)
- Operating environment: Google Colab (primary training), Google Drive (checkpoint storage)
- Programming language: Python 3
- Core ML libraries: scikit-learn, XGBoost, LightGBM, CatBoost, imbalanced-learn (SMOTE)
- Explainability library: SHAP
- Data handling: pandas, NumPy
- Visualization: Matplotlib, Seaborn
- Persistence: Python pickle, per-model and per-sample checkpointing to Google Drive

3.2 Method

This section presents the complete workflow of the proposed framework that describes each stage of the methodology in detail. It explains the motivation behind selecting the twelve machine learning models, their basic principles behind each model, and the evaluation metrics used to measure their performance. In addition, this section discusses the limitations of traditional encryption methods in the presence of future quantum computing threats. The decision-making process of the Adaptive Risk Classification Engine, the operation and security principles of the BB84 Quantum Key Distribution simulation, and the mechanism used to convert detected attacks into appropriate security responses. The prediction output is then processed by the Adaptive Risk Classification Engine, which assigns the flow to an appropriate risk level. For High-risk traffic, the framework performs additional attack-type classification and generates a suitable security response recommendation. Finally, only flows identified as HIGH risk activate the BB84 Quantum Key Distribution module, allowing quantum-level protection to be applied selectively rather than across all network traffic.

3.2.1 Proposed Models Architecture and Mathematical Formulation

Table 3.5 shows the configuration we used for each of the twelve models. We want to explain how each model works, not what it does. We want to show you the logic, behind each model so you can understand how it makes predictions.

Table 3.5: Twelve Model Configurations and Hyperparameters

Model	Key Hyperparameters
Random Forest	n_estimators=200, max_depth=15, min_samples_split=5, random_state=42
Gradient Boosting	HistGradientBoostingClassifier max_iter=150, learning_rate=0.1, max_depth=6
XGBoost	n_estimators=200, learning_rate=0.1, max_depth=7, eval_metric='logloss'
SVM	kernel='rbf', C=10, gamma='scale', probability=True, cache_size=1000
Voting Ensemble	Soft voting over RF + XGBoost + Gradient Boosting base learners
Stacking	Base: RF, XGBoost, GB, SVM — Meta: Logistic Regression, cv=2
LightGBM	n_estimators=500, learning_rate=0.03, num_leaves=128, max_depth=12, subsample=0.85, colsample_bytree=0.85, min_child_samples=20
MLP Neural Net	hidden_layer_sizes=(256,128), activation='relu', solver='adam', early_stopping=True
CatBoost	iterations=300, learning_rate=0.05, depth=6
Extra Trees	n_estimators=200, max_depth=15
AdaBoost	n_estimators=200, learning_rate=0.1
Naive Bayes	GaussianNB (Gaussian likelihood)

A) Random Forest

The Random Forest algorithm creates an ensemble through the combination of independent decision trees which use bootstrap samples to train on different features during their split process. The ultimate prediction results from the majority voting process.

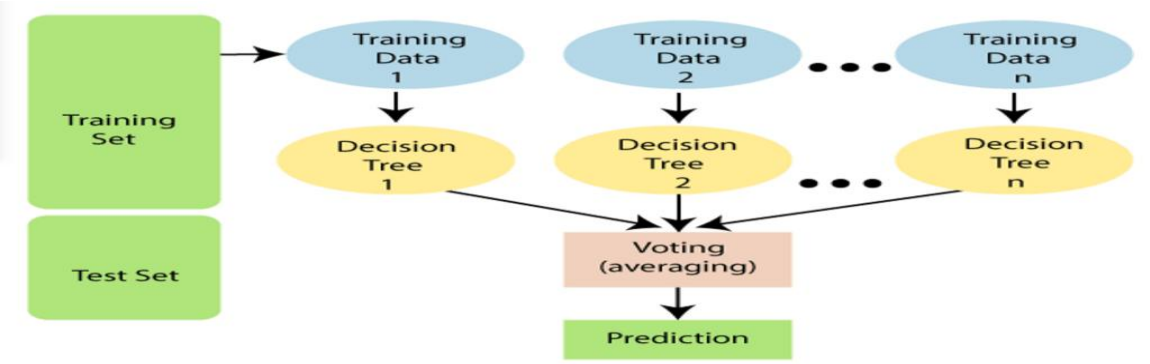


Figure 3.2: Architecture of Random Forest

Random Forest [38] builds 200 independent decision trees, each trained on a bootstrap-resampled subset of the training data (sampling with replacement) and splitting on a random subset of $\sqrt{p}$ features at each node (p being the total feature count), with a maximum depth of 15 and a minimum of 5 samples required to split.

Each split is chosen to maximize the reduction in Gini impurity, defined for a node t with class proportions p_i as follows, where a candidate split into a left child (n_L samples) and right child (n_R samples, n = n_L + n_R) is scored by how much it reduces impurity from parent to children, and the tree greedily selects the feature/threshold pair that maximizes this reduction at every node.

$$\text{Gini}(t) = 1 - \Sigma\, p_i^2$$

$$\Delta\text{Gini} = \text{Gini}(\text{parent}) - (n_L/n)\cdot\text{Gini}(\text{left}) - (n_R/n)\cdot\text{Gini}(\text{right})$$

Predictions are combined by majority vote across all 200 trees, and this fraction directly serves as the model's probability estimate for the risk engine.

$$P(\text{attack} \mid x) = (1/200)\ \Sigma_{t=1}^{200}\ 1[\text{tree}_t(x) = \text{attack}]$$

B) Gradient Boosting

Gradient Boosting builds its trees through a sequence of tree constructions which use negative gradients of the loss function to train each new tree on the current model's output.

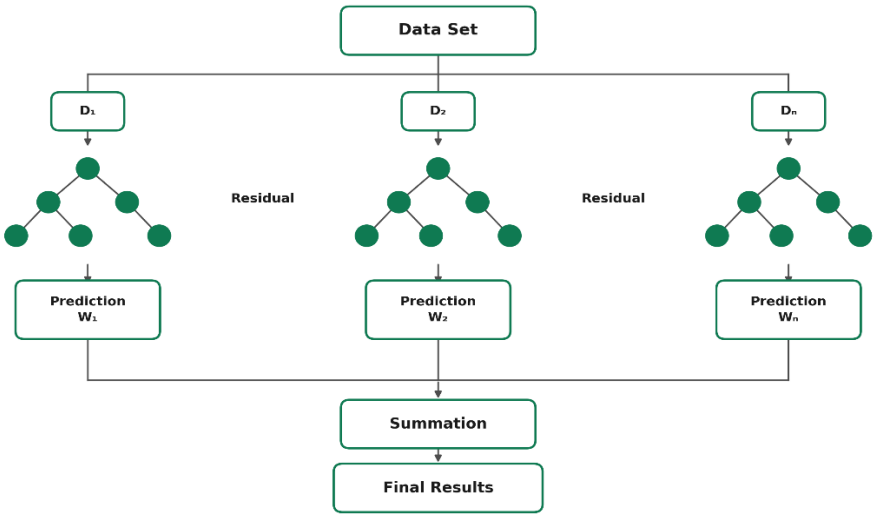


Figure 3.3 Architecture of Gradient Boosting

Gradient Boosting (implemented here via HistGradientBoostingClassifier for speed on the 150,000-row merged pool) builds trees sequentially, with each new tree fit not to the raw labels but to the negative gradient of the loss with respect to the current ensemble's predictions in effect, each tree learns to correct the mistakes of everything trained before it.

For binary log-loss (where p is the sigmoid of the raw score), the pseudo-residual at iteration m and the model update are shown below, where h_m is a shallow regression tree (depth 6) fit to the residuals and $\eta = 0.1$ is the learning rate that shrinks each tree's contribution to avoid overfitting.

$$L(y,F(x)) = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

$$r_i = y_i - p_i$$

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

This project configures 150 boosting iterations at a learning rate of 0.1 and a maximum depth of 6, using histogram-based binning to accelerate split-finding on large training pools.

C) XGBoost

XGBoost is one of the efficient gradient boosting machine learning algorithm. It is a sequence of several decision trees that are used to increase the accuracy of its predictions by correcting previous errors.

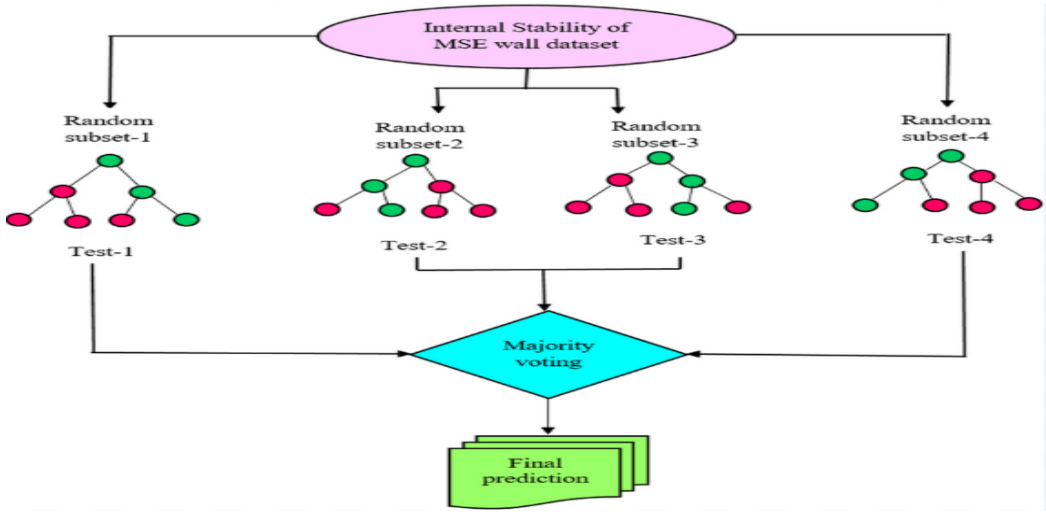


Figure 3.4: Architecture of XGBoost

XGBoost [39] extends the gradient boosting framework [40] with an explicitly regularized objective, where the regularization term penalizes both the number of leaves T and the magnitude of leaf weights w, discouraging overly complex trees.

$$\text{Obj} = \Sigma_i \, L(y_i, \hat{y}_i) + \Sigma_k \, \Omega(f_k)$$

$$\Omega(f) = \gamma T + (1/2)\lambda \|w\|^2$$

Using a second-order Taylor expansion of the loss around the current prediction (g_i and h_i are the first and second derivatives of the loss), the optimal weight for leaf j and the gain from a candidate split into left/right children are computed as follows, where G_j and H_j are the summed gradients and Hessians of samples in leaf j, and the tree greedily selects the split that maximizes the gain.

$$w_j^* = -G_j / (H_j + \lambda)$$

$$\text{Gain} = (1/2)\cdot[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - (G_L+G_R)^2/(H_L+H_R+\lambda)] - \gamma$$

This project trains 200 trees at a maximum depth of 7 and a learning rate of 0.1, using log-loss as the boosting objective and sparsity-aware split-finding to handle any missing values efficiently.

D) Support Vector Machine (SVM)

The input data is mapped to a higher dimensional space through the kernel trick, and then classified by the RBF-kernel SVM. It separates the different classes in an effective manner by determining an optimum decision boundary while controlling misclassification with the parameter (C). The overall architecture of the RBF-kernel SVM is represented in the next figure.

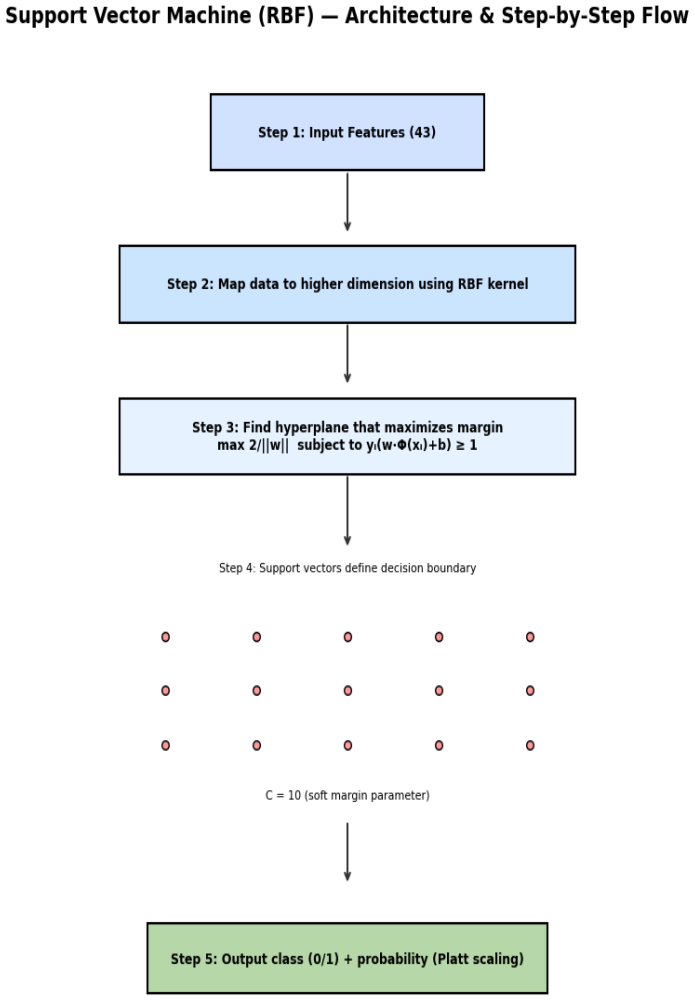


Figure 3.5: Architecture of Support Vector Machine (SVM)

The RBF-kernel SVM [41] maps input features into a higher-dimensional space via the kernel trick without ever computing the mapping explicitly.

Using the kernel function,

$$K(x_i, x_j) = \exp(-\gamma \cdot ||x_i - x_j||^2)$$

Which measures similarity that decays smoothly with Euclidean distance.

Training solves the soft-margin dual optimization problem shown below,

Subject to $0 \leq \alpha_i \leq C$ and $\Sigma_i \, \alpha_i \, y_i = 0$,

Where C = 10 controls the trade-off between maximizing the margin and tolerating misclassified training points; Platt scaling then fits a sigmoid to the resulting decision function to produce a calibrated probability, which the risk engine requires.

$$\max \Sigma_i \, \alpha_i - (1/2) \cdot \Sigma_i \Sigma_j \, \alpha_i \alpha_j \, y_i \, y_j \, K(x_i, x_j)$$

$$f(x) = \Sigma_i \, \alpha_i \, y_i \, K(x_i, x) + b$$

Because SVM training scales poorly with dataset size (its dual optimization has $O(n^2)$ memory complexity), it was capped to the 50,000-row merged subset during final training on all three datasets.

E) LightGBM

LightGBM [42] is a fast gradient boosting model made for large datasets. It builds trees one after another, where each tree fixes the mistakes of the last one.

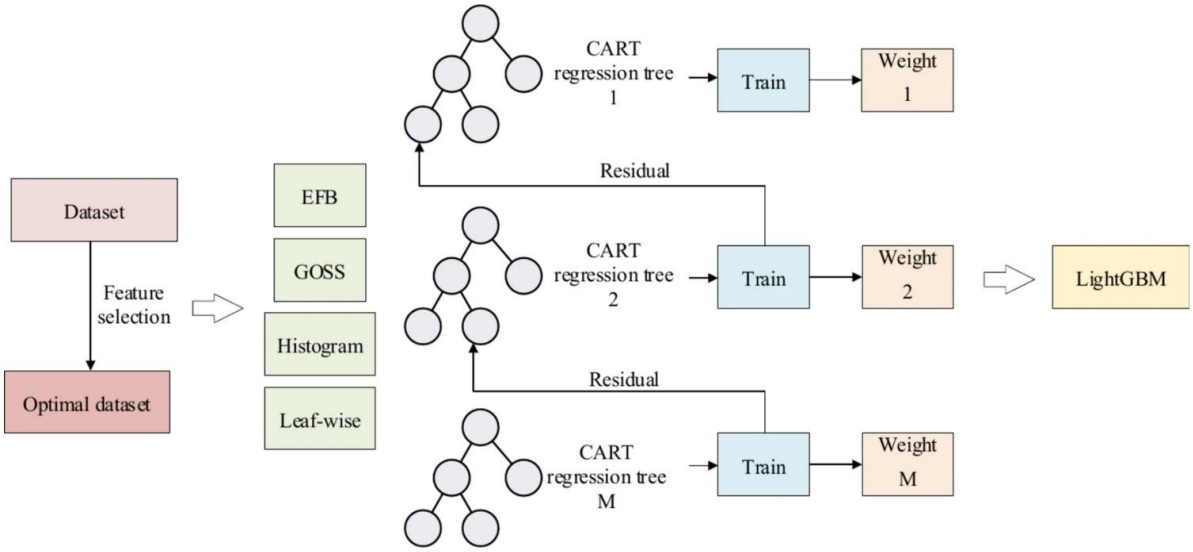


Figure 3.6: Architecture of LightGBM

Additive boosting:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i)$$

Where,

$\hat{y}_i^{(t)}$= prediction for sample $i$after step t

$\hat{y}_i^{(t-1)}$= prediction from the previous step

η= learning rate (0.03 in this project)

(x_i)= output of the new tree added at step t

The prediction at step $t$equals the previous prediction plus a new tree's output, scaled by the learning rate. This is how the model improves step by step.

Objective function (with regularization):

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t),\ \Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Where,

n= total number of training samples

$l(\cdot)$= loss function (measures prediction error)

y_i= true label for sample i

$\Omega(f_t)$= regularization term (penalty on tree complexity)

T= total number of leaves in the tree

γ= penalty per leaf (discourages too many leaves)

w_j= weight (output value) of leaf j

λ= L2 regularization strength (shrinks leaf weights)

The first part measures prediction error across all samples. The second part penalizes too many leaves or overly large leaf weights, which keeps the model from overfitting.

Leaf-wise growth (split gain):

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

where,

G_L, G_R= sum of gradients for samples in the left / right split

H_L, H_R= sum of Hessians for samples in the left / right split

λ= L2 regularization strength

γ= penalty for adding a new leaf

This formula measures how much loss is reduced by making a split. LightGBM picks whichever leaf across the whole tree gives the highest gain, not just the ones at the current depth this is what makes it leaf-wise instead of level-wise.

Gradient and Hessian sums (used above):

$$G_j = \sum_{i \in I_j} g_i \, , \; H_j = \sum_{i \in I_j} h_i$$

g_i= gradient (first derivative of the loss) for data point i direction and size of the error

h_i= Hessian (second derivative of the loss) for data point i confidence/curvature of the error

I_j= set of data points that land in leaf/node j

G_j= total gradient for leaf j

H_j= total Hessian for leaf j

GOSS rescaling factor:

$$\text{factor} = \frac{1 - a}{b}$$

- a= fraction of large-gradient ("hard") instances always kept
- b= fraction of small-gradient ("easy") instances randomly sampled

This factor rescales the retained easy examples so the overall gradient estimate stays accurate even after subsampling.

Leaf output value:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Where,

w_j^*= optimal prediction value assigned to leaf j

G_j= total gradient for leaf j

H_j= total Hessian for leaf j

λ= L2 regularization strength

LightGBM grows trees leaf-wise, always splitting whichever leaf reduces loss the most instead of expanding every leaf at each depth. This reaches good accuracy with fewer splits but can create deeper, uneven trees, so max_depth=12 is used to control overfitting.

Two techniques boost efficiency: GOSS keeps all high-error (hard) examples but only samples a portion of low-error (easy) ones, rescaling them to stay unbiased. EFB combines sparse features that rarely overlap into one feature, cutting dimensionality without losing information.

This project used 500 estimators, a learning rate of 0.03, 128 leaves, max depth of 12, and 85% row/column subsampling. LightGBM trained the fastest among top models and gave the best F1-score on the final merged pool across all three datasets.

F) MLP Neural Network

The Multi-Layer Perceptron uses two hidden layers which contain 256 and 128 neurons and it applies ReLU activations together with the Adam optimizer. The system uses early stopping together with a 10% validation fraction to stop overfitting. The MLP is the only model that can learn non-linear feature interactions through multiple layers of abstraction. Configuration uses hidden_layer_sizes which contains 256 and 128 neuron layers together with alpha equal to 1e-4 and learning_rate set to adaptive.

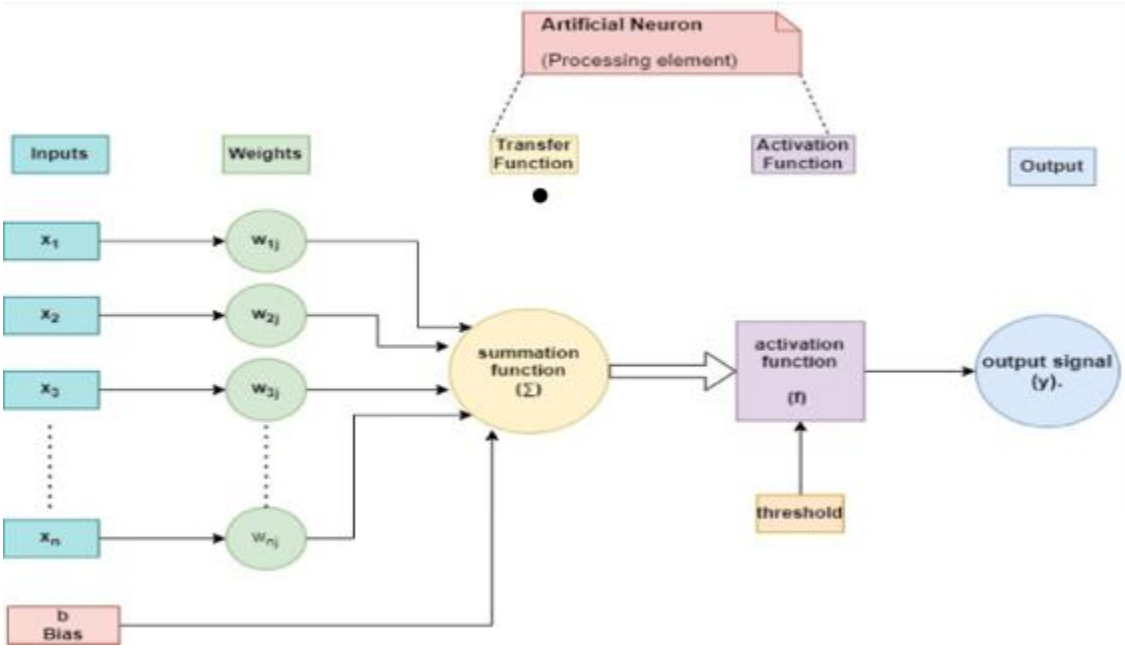


Figure 3.7: Architecture of MLP Neural Network

The Multi-Layer Perceptron passes each input through two hidden layers of 256 and 128 ReLU-activated neurons before a sigmoid output layer, as shown below, where ReLU(z) = max(0, z) introduces non-linearity and σ is the logistic sigmoid.

$$a^{(1)} = \mathrm{ReLU}(W^{(1)}x + b^{(1)})$$

$$a^{(2)} = \mathrm{ReLU}(W^{(2)}a^{(1)} + b^{(2)})$$

$$\hat{y} = \sigma(W^{(3)}a^{(2)} + b^{(3)})$$

Training minimizes binary cross-entropy loss,

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Backpropagation using the Adam optimizer [43], which maintains running estimates of the first and second moments of the gradient (m_t and v_t) and updates each weight as $\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$, adapting the effective learning rate per parameter.

G) CatBoost

CatBoost is a gradient boosting algorithm that is optimized for categorical and numerical features. It creates several decision trees in a sequence to enhance accuracy of prediction and minimize errors.

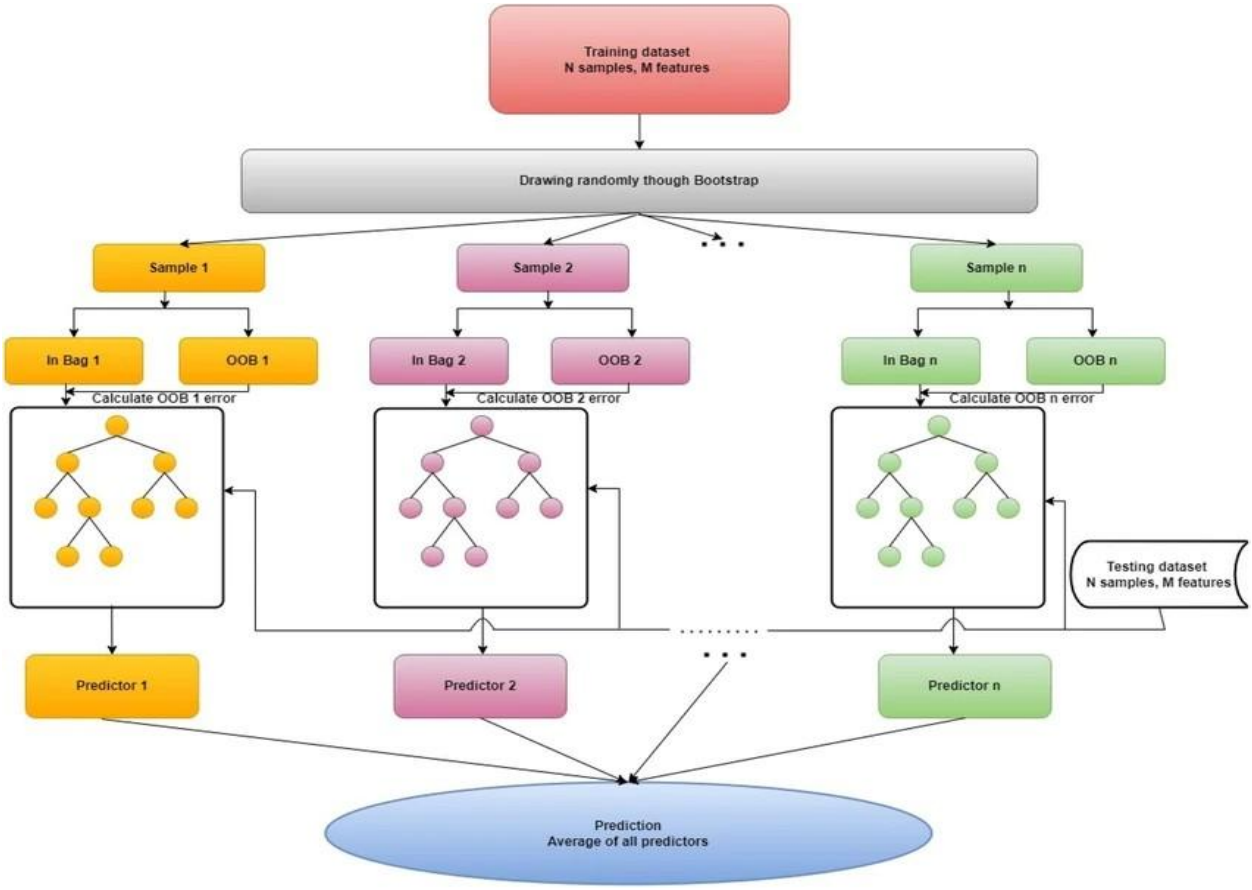


Figure 3.8: Architecture of CatBoost

CatBoost [44] uses ordered boosting: rather than fitting every tree using target statistics computed from the full training set which can leak target information into the features used to build early trees. It processes training rows in a random permutation and, for each row, computes residuals using only a model trained on rows that came before it in that permutation. This specifically reduces the target-leakage bias that standard gradient boosting is prone to.

CatBoost also grows symmetric trees, where the same feature and threshold are used to split every node at a given depth across the entire tree, which acts as a form of built-in regularization and makes prediction faster.

This project configures 300 iterations at a learning rate of 0.05 and a depth of 6. CatBoost was added in this generalized study specifically to test whether its leakage-resistant training procedure would outperform.

H) Extra Trees, AdaBoost, and Naive Bayes

Extra Trees, AdaBoost, and Naive Bayes are widely used machine learning algorithms for classification tasks. Extra Trees combines multiple randomized decision trees, while AdaBoost sequentially combines weak learners to improve prediction accuracy. Naive Bayes uses probability-based calculations to classify data based on the input features. The overall architectures of these models are shown below.

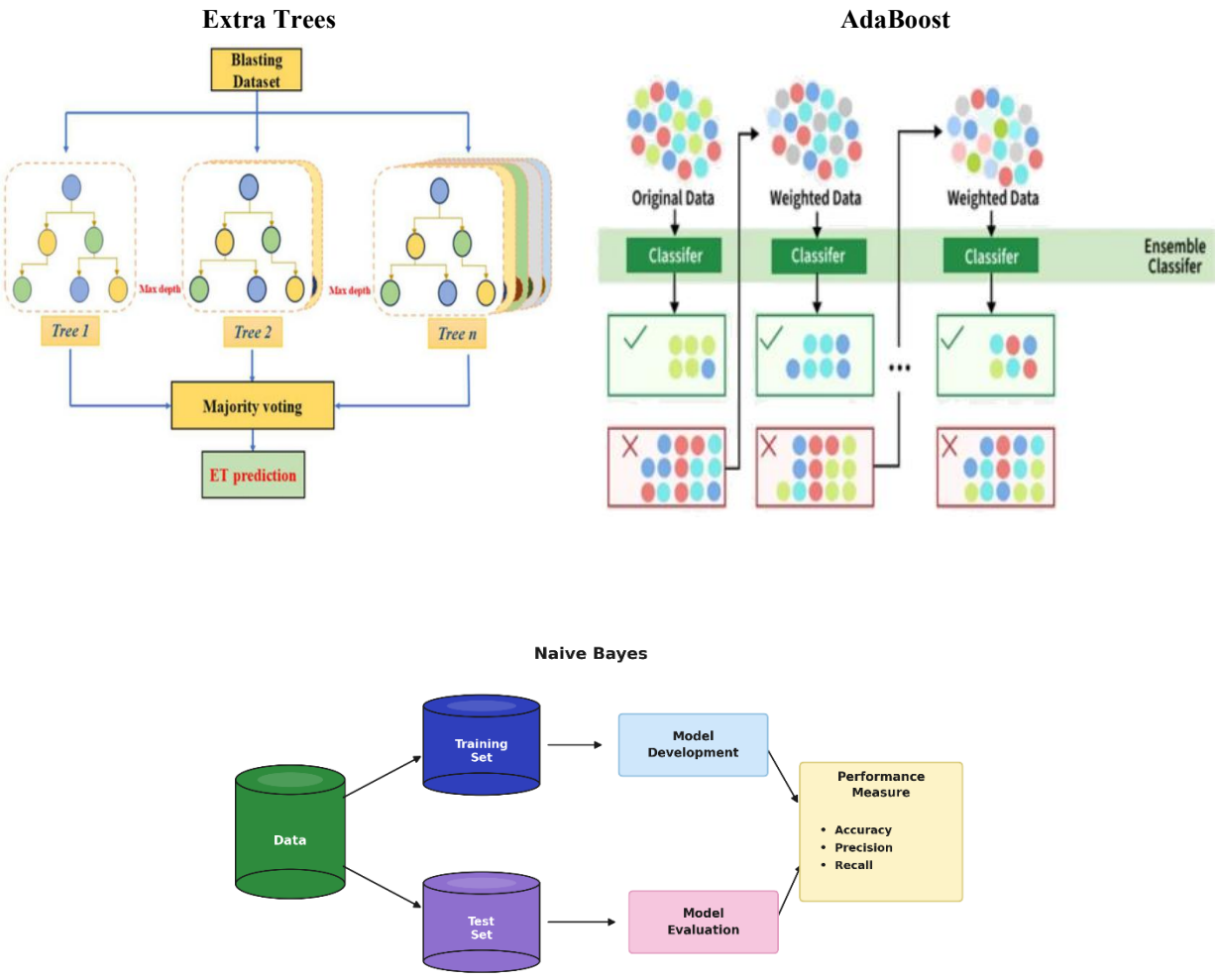


Figure 3.9: Architecture of Extra Trees, AdaBoost, and Naive Bayes

Extra Trees (Extremely Randomized Trees) [45] are similar to Random Forest but selects split thresholds randomly instead of searching for the best possible split at each node. This makes the training process faster and helps reduce variance, although the accuracy of per trees may be slightly lower.

AdaBoost [46] sequentially training 200 weak learners and updates the sample weights after each round, where α_t is the learner's vote weight (larger when its weighted error ε_t is small) and Z_t is a normalizing constant. Misclassified samples receive higher weight, helping the next weak learner to focus on the difficult cases.

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(\alpha_t \cdot 1[y_i \neq h_t(x_i)]) \,/\, Z_t$$

$$\alpha_t = (1/2)\cdot\ln((1-\varepsilon_t)/\varepsilon_t)$$

$$H(x) = \text{sign}(\Sigma_t\, \alpha_t \cdot h_t(x))$$

Gaussian Naive Bayes uses Bayes' theorem and assumes that features are independent of each other within each class. It models the probability distribution of each feature using a Gaussian, as shown below. Because network flow features are strongly correlated with one another (e.g., byte count scales with packet count), this independence assumption is systematically violated, which is the direct mathematical reason Naive Bayes consistently produced the lower F1-scores across all three datasets (as low as 0.4087 on HQC-2026).

$$P(y|x) = P(x|y)\cdot P(y) \,/\, P(x)$$

$$P(x_j|y) = (1/\sqrt{(2\pi\sigma^2_{y,j})})\cdot\exp(-(x_j-\mu_{y,j})^2 \,/\, (2\sigma^2_{y,j}))$$

I) Stacking Classifier (Hybrid)

The Stacking Classifier uses two separate learning levels to perform its functions.

Level-0 comprises Random Forest, XGBoost, Gradient Boosting, and SVM models. These models generate out-of-fold predictions using five-fold cross-validation which prevent data leakage into the meta learner. Finally, A Logistic Regression meta learner combines the Level-0 predictions by learning their optimal weights.

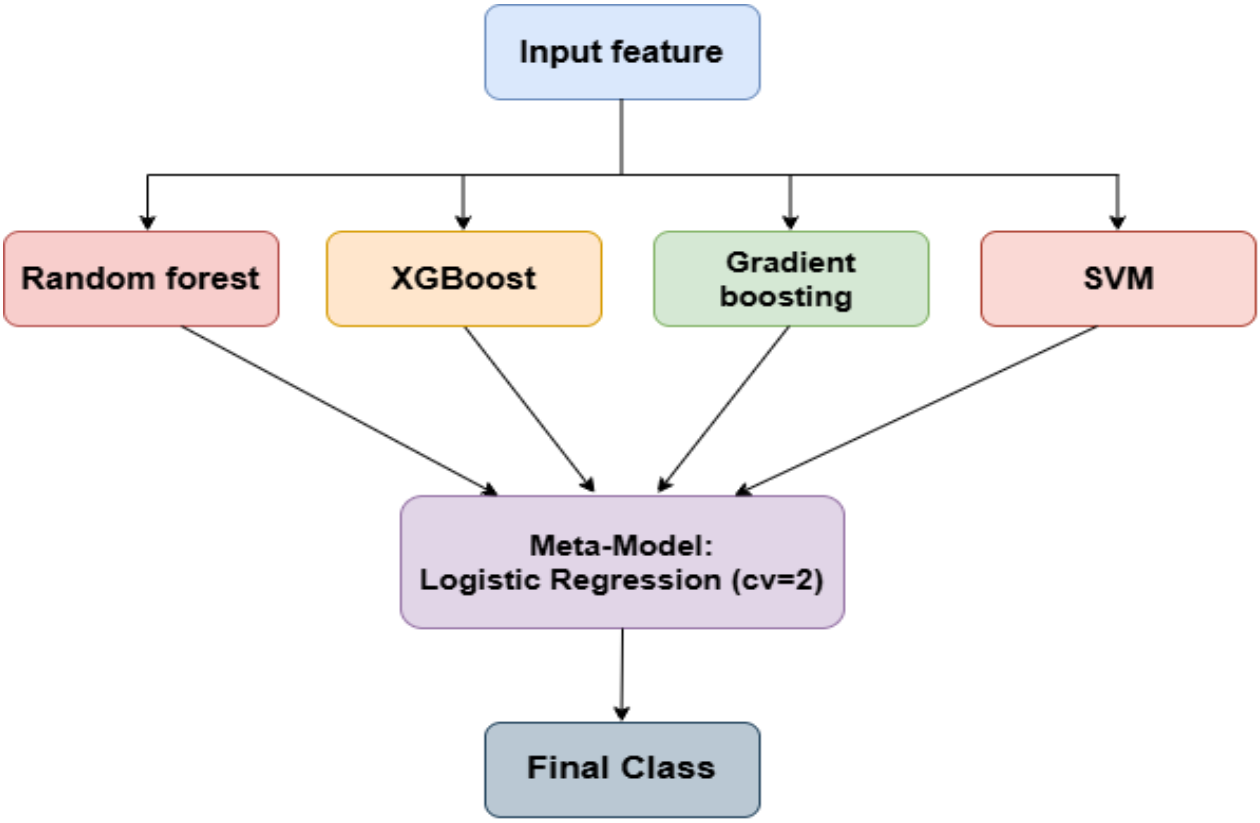


Figure 3.10: Architecture of Stacking Classifier (Hybrid)

Stacking uses four Level-0 base models Random Forest, XGBoost, Gradient Boosting, and SVM. Each model generates out-of-fold predictions via 2-fold cross-validation, where predictions for any given training row were generated by a copy of that model trained on the other fold, so the meta-learner never sees a base model's prediction on data it was directly trained on leak information and produce an overconfident meta-model.

A Logistic Regression meta-model combines the out-of-fold prediction probabilities from the four base models using learned weights, as shown below, where σ is the sigmoid function and the weights w_k are fit by maximum-likelihood logistic regression to learn how much to trust each base model.

$$P(\text{attack}|x) = \sigma(w_0 + \Sigma_{k=1}^{4} w_k \cdot z_k)$$

$$\sigma(z) = 1/(1+e^{-z})$$

Like SVM, Stacking was capped to the 50,000-row merged subset for final training because its 2-fold cross-validated base-model fitting is the single most computationally expensive step in the entire pipeline.

J) Voting Ensemble (Hybrid)

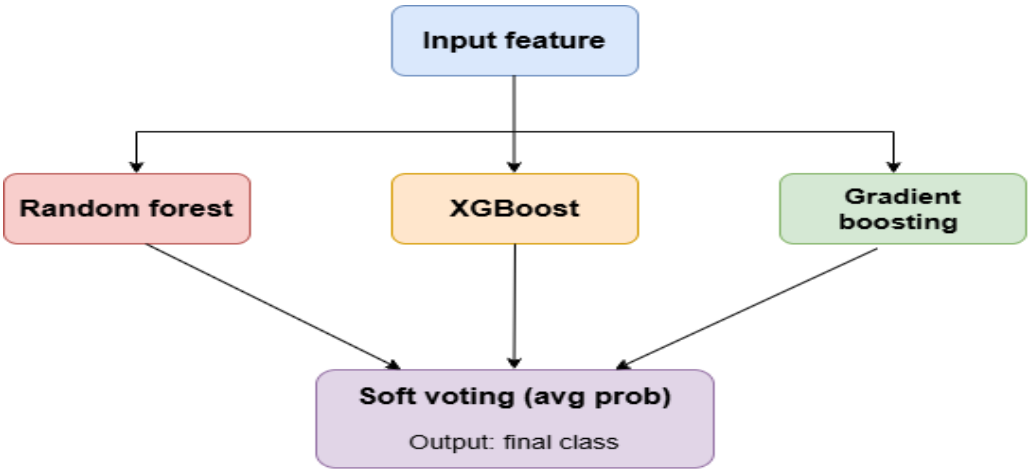


Figure 3.11: Architecture of Voting Ensemble (Hybrid)

The Soft Voting Ensemble trains Random Forest, XGBoost, and Gradient Boosting independently on the full training data, then averages their attack-class probabilities and thresholds the result at 0.5 to produce the final class.

$$P_{avg}(\text{attack}|x) = (1/3)\cdot[P_{RF}(x) + P_{XGB}(x) + P_{GB}(x)]$$

Soft voting was chosen over hard (majority) voting because it preserves each base model's confidence rather than collapsing it to a binary vote a flow on which all three base models agree with high confidence should be treated differently from one where the models barely agree, and averaging probabilities retains that information.

3.2.2 Performance Evaluation Metrics

Evaluation Metrics: All twelve models were evaluated using five metrics based on the confusion matrix (TP, TN, FP, FN).

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Misleading on imbalanced data predicting "benign" for everything on HQC-2026 would give 76.0% accuracy while catching zero attacks. So accuracy alone was never used to pick the best model.

$$\text{Precision} = \frac{TP}{TP+FP}$$

Measures how many flagged attacks were real attacks. Low precision means false alarms and wasted BB84 sessions.

$$\text{Recall} = \frac{TP}{TP+FN}$$

Measures how many real attacks were actually caught. Low recall means attacks slip through undetected.

$$\text{F1-Score} = = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{(\text{Precision} + \text{Recall})}$$

The harmonic mean of Precision and Recall. Chosen as the main selection metric because it penalizes models that favor one over the other important since both false alarms and missed attacks carry real cost.

AUC-ROC Measures how well the model ranks attacks above benign flows across all thresholds, not just at 0.5. A score of 1.0 indicates perfect performance, while 0.5 is random guessing. Since the risk engine uses multiple thresholds, AUC-ROC is reported alongside F1 to confirm consistent performance across threshold ranges.

3.2.3 Classical Encryption versus Quantum Key Distribution: Motivating BB84

This subsection is the mathematical heart of this project's motivation. Before describing the Adaptive Risk Engine and BB84 simulation, it is necessary to show with the underlying formulas why AES-256 remains safe against a quantum adversary while RSA and Elliptic-Curve Diffie-Hellman. The classical algorithms used to distribute an AES key are not secure against quantum attacks. This difference is the main reason for building a BB84-based key-exchange layer in this project instead of relying only on stronger classical encryption.

A) AES-256 - How It Works and Why It Survives a Quantum Attack

AES-256 (Advanced Encryption Standard, the Rijndael algorithm) is a symmetric key block cipher. Here 256-bit secret key K is used both to encrypt and decrypt. It operates on 128-bit blocks of plaintext, arranged as a 4x4 matrix of bytes, and transforms that state through 14 rounds, where each round (except the last) applies four operations in sequence.

Round formula:

$$State_{r+1} = AddRoundKey\left(MixColumns\left(ShiftRows\left(SubBytes(State_r)\right)\right)\right)$$

Where,

$State_r$= the 4x4 byte matrix at round r

$SubBytes$= replaces each byte using a fixed substitution table

$ShiftRows$= shifts each row of the matrix by a set amount

$MixColumns$= mixes the columns using matrix multiplication

$AddRoundKey$= XORs the state with the round key

Key schedule:

$$K_r = KeyExpansion(K, r)$$

Where,

K= the original 256-bit master key

K_r= the subkey generated for round r

r= the round number

Why it's quantum-safe:

A classical brute-force attack needs 2^{256}attempts an impossibly large number.Grover's algorithm speeds this up on a quantum computer, but only by a square root:

$$O\left(\sqrt{2^{256}}\right) = O(2^{128})$$

$N = 2^{256}$= total number of possible keys

The result, 2^{128}, is still astronomically infeasible so AES-256 stays secure.

The one quantum algorithm that touches symmetric ciphers like AES is Grover's algorithm [47], which searches an unsorted space of N possibilities in O(sqrt(N)) quantum queries instead of the O(N) queries a classical computer needs. 2^{128} remains astronomically far beyond any computationally feasible attack, even assuming large-scale, fault-tolerant quantum computers. Therefore AES-256 is considered as quantum-resistant in practice and does not attempt to replace it. This is the first half of this project's core motivation. The encryption algorithm itself was never the weak link.

B) RSA/ECDH - Why Quantum Computers Break It

Classical systems solve this key-exchange problem with RSA or Elliptic-Curve Diffie-Hellman. Their security relies on the assumption that certain mathematical problems are computationally difficult to solve, rather than being theoretically impossible to break. RSA key generation chooses two large random prime numbers p and q (each typically 1024 bits or larger for 2048-bit RSA).

RSA key generation:

$$e \cdot d \equiv 1 \ \left(\mathrm{mod}\ \phi(n)\right)$$

Where,

p, q= two large random prime numbers

$n = p \times q$= the modulus (part of the public key)

$\phi(n) = (p-1)(q-1)$= Euler's totient

e= the public exponent

d= the private exponent

Encryption/Decryption:

$$c = m^e\, n,\ m = c^d\, n$$

Where,

m= the original message (plaintext)

c= the encrypted message (ciphertext)

RSA's entire security rests on the assumption that, given only the public modulus n, an attacker cannot feasibly recover the two prime factors p and q because doing so would let them reconstruct φ(n) and therefore d, breaking the scheme completely

Classical factoring time (General Number Field Sieve):

On a classical computer, the best known algorithm for factoring n (the General Number Field Sieve) runs in sub-exponential time:

$$T_{classical}(n) \approx \exp\left[1.923 \cdot (\ln n)^{1/3} \cdot (\ln \ln n)^{2/3}\right]$$

Where,

n= the size of the RSA modulus

This time is sub-exponential so increasing key size increases security on classical computers.

Which, for a 2048-bit RSA modulus. It estimated to require on the order of tens of thousands of years even with dedicated classical supercomputing clusters this sub-exponential (but still faster-than-brute-force) growth is exactly what makes RSA practically secure against classical computers today.

Shor's algorithm (quantum):

Shor's algorithm [5], published in 1994, is a quantum algorithm that factors n using the quantum Fourier transform to find the period of the modular exponentiation function $f(x) = a^x \bmod n$.

Its run in time:

$$T_{quantum}(n) = O\left((\log n)^2 \cdot \log(\log n)\right)$$

RSA and ECDH are vulnerable to Shor's algorithm, making them unsuitable for long-term quantum security, while AES-256 remains relatively secure against Grover's algorithm. This creates a **"**harvest-now, decrypt-later**"** threat for IoT networks. Therefore, BB84 Quantum Key Distribution (QKD) is used to replace RSA/ECDH for high-risk key exchange, while AES-256 remains unchanged for data encryption, providing a more quantum-resistant communication system.

3.2.4 Adaptive Risk Classification Engine

The classify risk function takes each flow's continuous probability score for attack generated by the machine learning classifier and classifies it into one of three discrete risk categories, each corresponding to a particular security action. This subsection not only defines these three categories but also the rationale behind setting their thresholds and its implications.

Table 3.6: Adaptive Risk Tier Definitions

Tier	Probability Range	Security Response	Rationale
LOW	P(attack) < 0.35	Classical AES-256 encryption	Attack probability is below background noise; standard encryption suffices.
MEDIUM	0.35 <= P(attack) < 0.70	AES-256 + enhanced monitoring	Borderline flows warrant increased vigilance without full QKD cost.
HIGH	P(attack) >= 0.70	BB84 QKD session triggered immediately	High attack probability justifies quantum-level security; a fresh 256-bit key is generated.

Threshold placement is treated as a cost-sensitive decision. Where the HIGH threshold (0.70) requires strong confidence before triggering the costly BB84 session, and the LOW threshold (0.35) allows moderate-risk flows to enter MEDIUM monitoring. The same thresholds were applied across all datasets because their risk distributions were mainly near 0 or 1. This reduces unnecessary BB84 operations while maintaining security, achieving 51.6%–81.5% overhead savings.

$$\text{Cost}(\tau) = C_{FN} \cdot P(y=1) \cdot P(\hat{y}<\tau | y=1) + C_{FP} \cdot P(y=0) \cdot P(\hat{y} \geq \tau | y=0) + C_{QKD} \cdot P(\hat{y} \geq \tau_{HIGH})$$

Here, C_{FN} represents the cost of missing an attack, C_{FP} the cost of a false alarm, and C_{QKD} the cost of unnecessary QKD activation. The ML model determines which flows receive quantum protection, while BB84 provides eavesdropping detection and security once triggered.

3.2.5 BB84 QKD Simulation

The BB84 QKD simulation demonstrates the basic process of secure quantum key distribution between a sender and receiver. It uses quantum states and random measurement bases to generate a shared secret key and detect possible eavesdropping. The overall BB84 QKD simulation process is shown below.

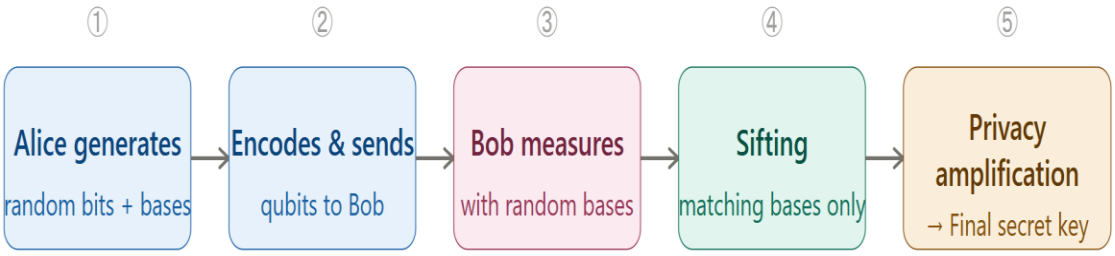


Figure 3.12: BB84 QKD Protocol Step-by-Step Simulation Flow

The process begins when the ML-based risk engine analyzes each network flow and assigns an attack-risk probability. Only flows classified as HIGH risk are forwarded to the BB84 protocol. For a HIGH-risk flow, Alice generates 256 random bits and 256 randomly selected bases. She encodes each bit according to the selected basis and transmits the resulting quantum states to Bob. Bob independently chooses a random basis for measuring each received qubit. If an eavesdropper (Eve) intercepts the communication, she also selects random measurement bases and retransmits the qubits. When Eve chooses an incorrect basis, her measurement disturbs the quantum state, introducing errors that can later be detected.

Next, they calculate the Quantum Bit Error Rate (QBER) using a randomly selected portion of the sifted key:

$$\text{QBER} = \frac{\text{Number of mismatched bits}}{Total\ bits\ compared}$$

If the QBER is below 11%, the channel is considered secure, and the remaining sifted bits undergo privacy amplification to generate the final 256-bit quantum key using SHA-256. If the QBER is 11% or higher, the channel is considered insecure and the generated key is discarded.

In the experiments, a clean channel produced approximately 0% QBER, while 10% eavesdropping produced about 2.5%–2.8% QBER, which remained below the threshold. In contrast, 75% eavesdropping produced approximately 18.6%–18.9% QBER, exceeding the 11% threshold and causing the key to be rejected.

3.2.6 Attack-Type-Aware Response Recommendation Engine

However, identifying a flow as HIGH risk is not enough for effective security response. Different attacks require different actions. Using the same response for all HIGH-risk flows may be too harsh, by blocking legitimate traffic, or too weak, by only logging serious attacks.

Table 3.7: Attack-Type-to-Response Mapping and Rationale

Attack Family (Examples)	Recommended Response	Why This Response
RDOS / DoS / DDoS floods	Rate-limit / traffic shaping	Volumetric floods often use spoofed source addresses; outright blocking risks discarding legitimate traffic caught in the flood, while throttling preserves availability for genuine users.
Scanning / Reconnaissance	Log + monitor	Reconnaissance alone does not confirm compromise and may originate from legitimate internal vulnerability scans; premature blocking can also alert a genuine attacker before enough evidence is gathered.
Brute-Force (SSH/FTP/Web)	Account lockout / IP block	Repeated failed authentication attempts are unambiguous; locking the account or blocking the source IP is the standard, low-collateral-damage response.
Exfiltration / Infiltration	Isolate host + BB84 key refresh + forensics	Data already leaving the network implies likely compromise; isolating the host stops further loss, refreshing the session key assumes the current key may be compromised, and forensic logging supports later investigation.

Attack Family (Examples)	Recommended Response	Why This Response
Botnet / C&C traffic, Insider-Malicious	Isolate host + block C2 traffic / alert + forensics	Cutting the command-and-control channel stops an attacker's remote control of the compromised device immediately.
Protocol-Specific Hijack (e.g., MQTT broker subscription)	Revoke broker credentials	The compromise is protocol/credential-specific, so the most targeted fix is revoking the exposed credential rather than a network-wide response.
Industrial Control (e.g., Modbus register manipulation)	Isolate PLC + alert	Industrial control traffic is safety-critical; isolating the affected programmable logic controller prevents a potentially physical-world consequence.
Rare / Low-Confidence Predictions	No action / manual review	For attack types with very few training examples, the multiclass model's prediction is least reliable, so a human analyst is deliberately kept in the loop rather than automating a high-consequence response on low-confidence evidence.

It has been applied in all three datasets presented in Chapter 4 in a consistent manner, and the actual triggering of the response, of course, being different in each dataset due to the difference in the distribution of attack families in the HIGH risk flows of each, is documented for each dataset's multiclass classification results.

3.2.7 Experiment Setup

Training of all models had been done with the random state parameter being fixed at 42. The training process was checkpoint safe, with all twelve models, and the final merged pool for each of the three per-dataset samples saved individually as pickle files in Google Drive right after fitting, so that in case of runtime disconnect in Colab, one would have needed to continue with the training of the last successfully trained model. The multiclass attack type model utilized the hybrid resampling technique. 15,000 observations limit for the majority class, 3,000 observations limit for the minority classes with the use of SMOTE and 10x maximum multiplier, so that any class with fewer than 100 observations was to be merged into one Rare Attack class. The SHAP values were calculated on a 500 observations sample from the test dataset using the TreeExplainer for LightGBM.

3.3 Overall Work Procedure and Rationale

This section takes a step back from the detailed description of individual parts to clarify the reasoning behind the particular sequence of actions taken in the full pipeline, since the ordering of these steps itself is a design choice of the project, not just an arbitrarily picked one.

The work started by checking whether the earlier framework using only one single dataset made the correct assumption that the decision of an ML classifier about which flows should receive expensive BB84 protection can be generalized across the different data sources and is not unique to the one used dataset. This required the whole pipeline to be identical and replicable for three different datasets, which is why the pipeline started with the dataset specific preprocessing and not a universal cleaning algorithm, since each dataset required separate handling due to its leakage issues, skewness and imbalance before a general methodology could be applied. After the data was ready for analysis. The pipeline included two stages of model selection. The initial cheap and quick three sample stability test and the expensive training of twelve models. The reason for this lies in the computational expense of training twelve models (including SVM and Stacking classifiers which are relatively slow), as well as the full pool of 150,000 samples for three different datasets, and the stability test was meant to cheaply identify potential sampling-related errors before wasting the resources on the full training, which it indeed did (see Chapter 4). There was a case when CIC-UNSW-NB15 resulted in different winning model at the sampling and merging-pool stages, proving the necessity of this design approach.

The choice to build the Adaptive Risk Engine before building the BB84 simulation model was made because the main goal of the whole framework was to minimize the number of runs of the protocol. Thus, the decision of which flows to apply the protocol to had to be made first, including the threshold-selection rationale described above.

In turn, the choice to build the multiclass attack-type prediction model and Response Recommendation Engine based on the binary classification was motivated by the same rationale. The binary risk engine helps to quickly classify the flows by their risk and thus decide whether they require the additional attention while the other helps with the second part.

Finally, the SHAP explainability and the cross-dataset comparison. Since the framework which is not possible to audit in its decisions and whose behaviour was verified using one data source only cannot be used in practice.

3.4 Summary

This chapter has given an explanation, including detailed mathematics and procedure of the methods and material used in the three different network traffic datasets. The specific data pre-processing technique for each dataset and its parameter values and formulae. The reasons for choosing twelve models among four algorithmic approaches and the mathematics involved in each case. The five metrics on which their performance is evaluated. Why the classical AES-256 cryptography is inadequate in dealing with the post-quantum problem solved by the BB84 protocol. The three-layered Adaptive Risk Classification Engine, using decision-making thresholds based on the cost-sensitive decision theory applied to each probability distribution from each dataset. The six-step BB84 quantum key distribution simulation, which uses an 11% security threshold as per the Shor Reskill security proof. The Attack Type Aware Response Recommendation Engine which maps attack types to appropriate security responses. The next chapter now reports on results from all three datasets using the same methodology.

Chapter 4

Results and Discussion

This chapter reports results for each of the three datasets independently, then compares all three side by side, before discussing societal implications and summarizing key findings.

4.1 X-IIoTID (Australia) Results

This section reports full results for X-IIoTID, collected in Australia (2021), following the identical methodology described in Chapter 3.

4.1.1 Exploratory Data Analysis

This subsection presents the class and attack-type distribution for X-IIoTID as observed after preprocessing, providing the baseline context for interpreting every result that follows.

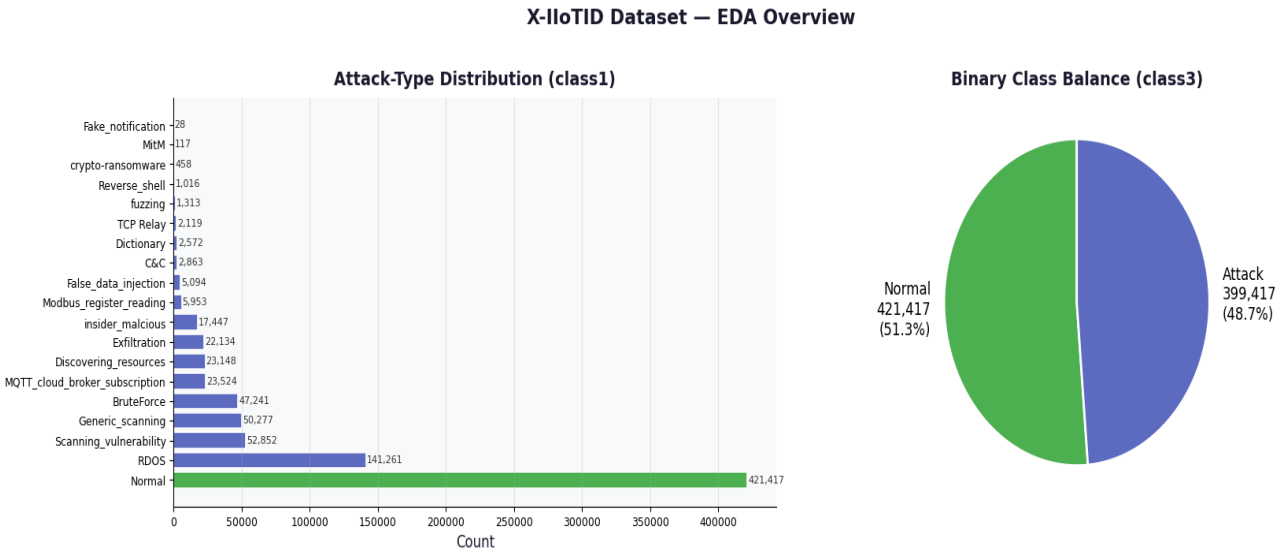


Figure 4.1: X-IIoTID Exploratory Data Analysis

4.1.2 Sampling Stability Check

All twelve models were trained individually using three different stratified samples of 50,000 rows. Each model ensures that there was no bias caused by the sampling process, as discussed in the stability test procedure in Section 3.2.1. Table 4.1 shows that LightGBM performed best among all models, with an F1 score of 0.9971 and a standard deviation of just 0.0001.

4.1.3 Model Performance on the Merged 150k Training Set

This section shows how all twelve models behaved after being retrained using the complete merged dataset of 150,000 rows, behavior of the best model, and the information about each models error distribution provided by the confusion matrix and ROC curve.

By combining all three samples into one pool of 150,000 rows (SVM and Stacking limited to a pool of 50,000 rows), all twelve models were retrained and evaluated against the test sample maintaining the original class proportion. The results can be found in Table 4.1, where LightGBM showed the best F1-score of 0.9971 (AUC-ROC 0.9999, Accuracy 0.9972).

Table 4.1: X-IIoTID Comparison of All 12 ML Models

Model	Accuracy	F1-Score	AUC-ROC
Random Forest	0.9948	0.9946	0.9998
Gradient Boosting	0.9966	0.9965	0.9999
XGBoost	0.9969	0.9968	0.9999
SVM	0.9843	0.9836	0.9984
Voting Ensemble	0.9964	0.9963	0.9999
Stacking	0.9970	0.9969	0.9999
LightGBM	0.9972	0.9971	0.9999
MLP Neural Net	0.9870	0.9866	0.9989
CatBoost	0.9960	0.9959	0.9998
Extra Trees	0.9840	0.9833	0.9982
AdaBoost	0.9394	0.9366	0.9734
Naive Bayes	0.8064	0.8278	0.9456

The strength of LightGBM in this scenario lies in scale and class balancing. Since X-IIoTID's nearly balanced dataset of 809,249 samples allows LightGBM's leaf wise growth algorithm to grow the largest loss reduction leaf without overfitting to a small minority class. The sampling technique GOSS becomes highly efficient in such large and well-balanced training sets. Naive Bayes lags behind all models (F1 = 0.8278) .X-IIoTID's IIoT protocol level features are correlated, which goes against the assumption of the conditional independence of Naive Bayes. AdaBoost's relatively low F1 (0.9366) comes from the fact that AdaBoost's 200 weak shallow trees do not reveal feature interactions to the same extent as LightGBM's 500-trees forest of depth 12 or Random Forest's 200 trees of depth 15.

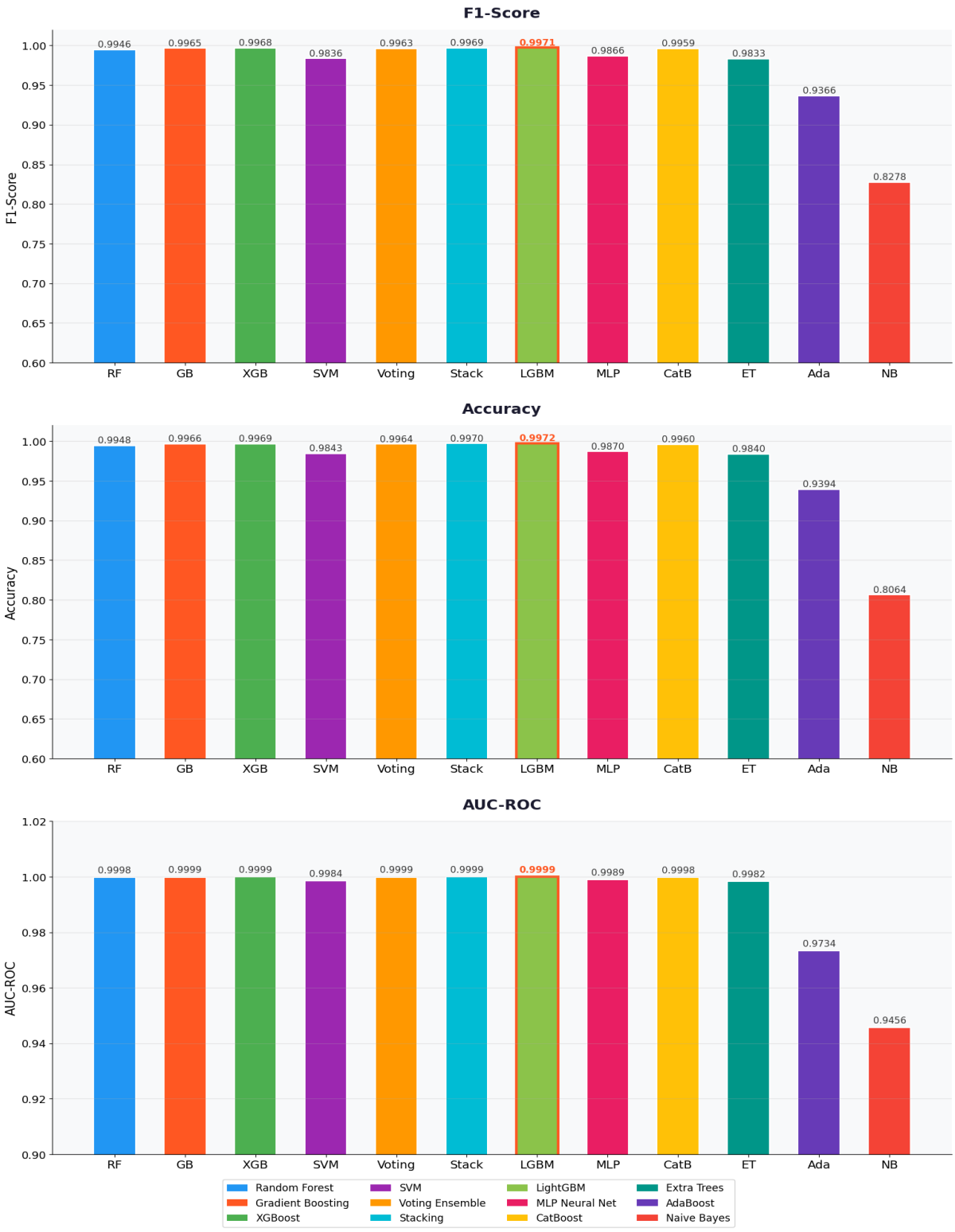


Figure 4.2: X-IIoTID Model Performance Comparison

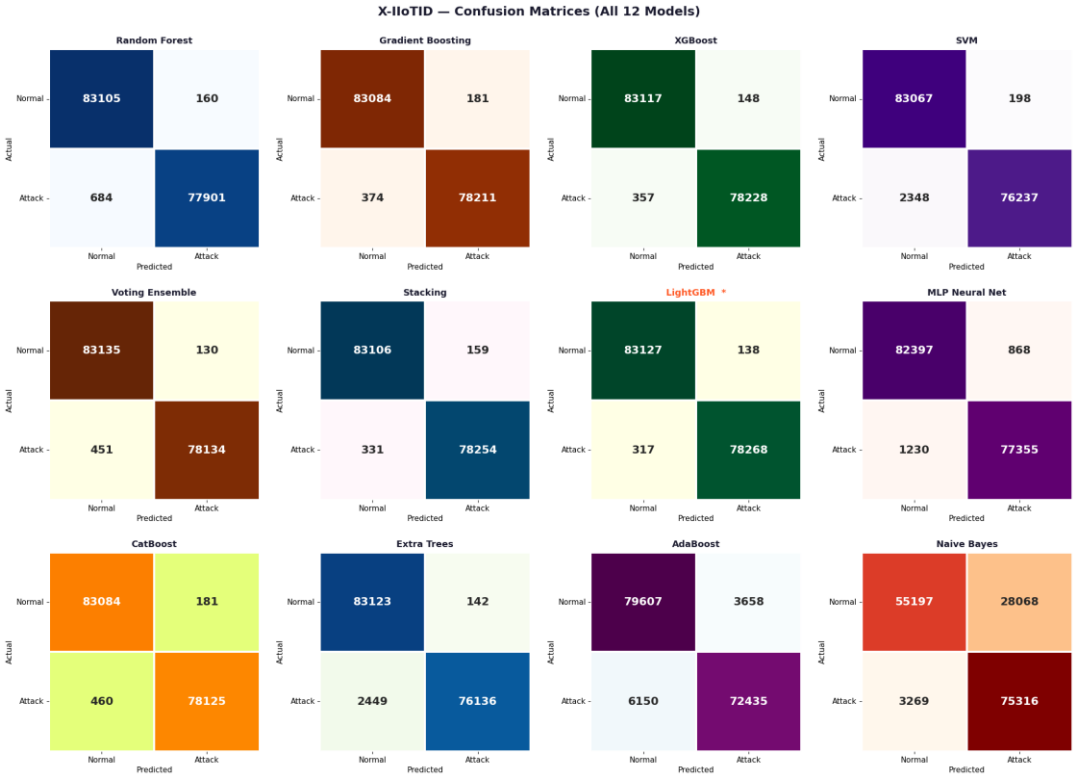


Figure 4.3: X-IIoTID Confusion Matrices

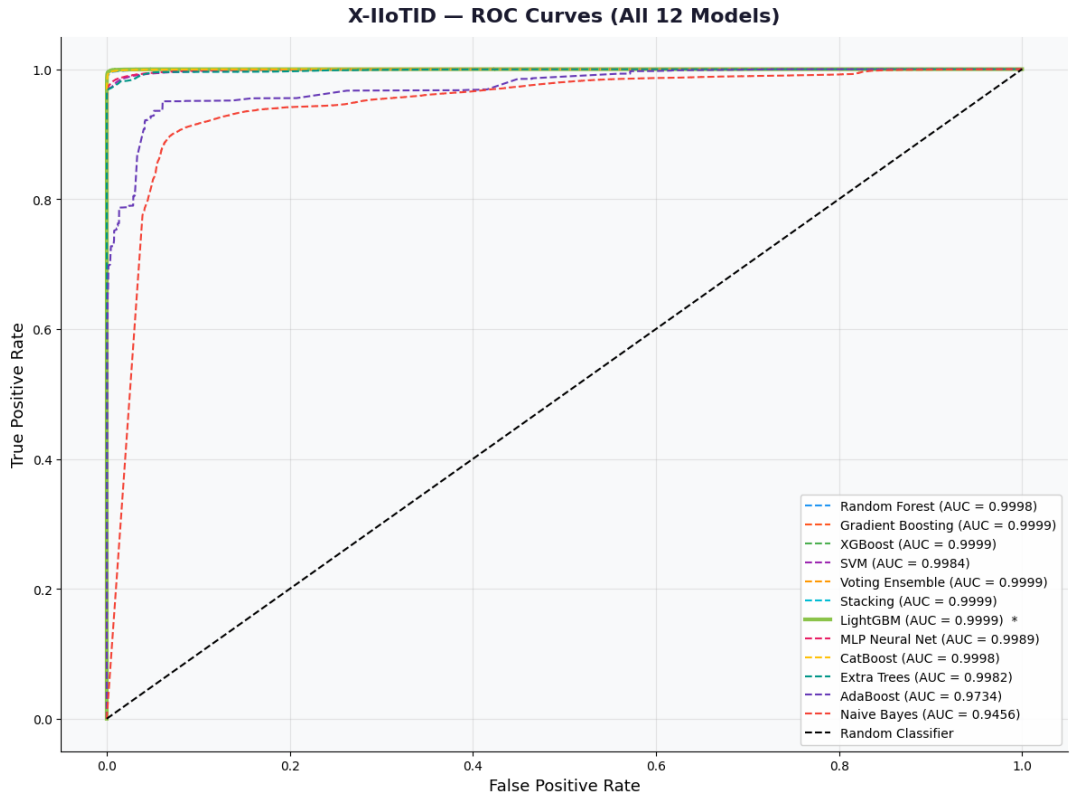


Figure 4.4: X-IIoTID ROC Curves

4.1.4 SHAP Explainability

This sub-section discusses which signal features LightGBM used in separating attack from benign samples on X-IIoTID dataset based on SHAP values.

SHAP values for LightGBM were calculated on 500 rows of test set data through Tree Explainer (exact algorithm to calculate SHAP for tree-based models). Avg-num-cswch/s was the top ranked feature while the other top-10 ranked features.

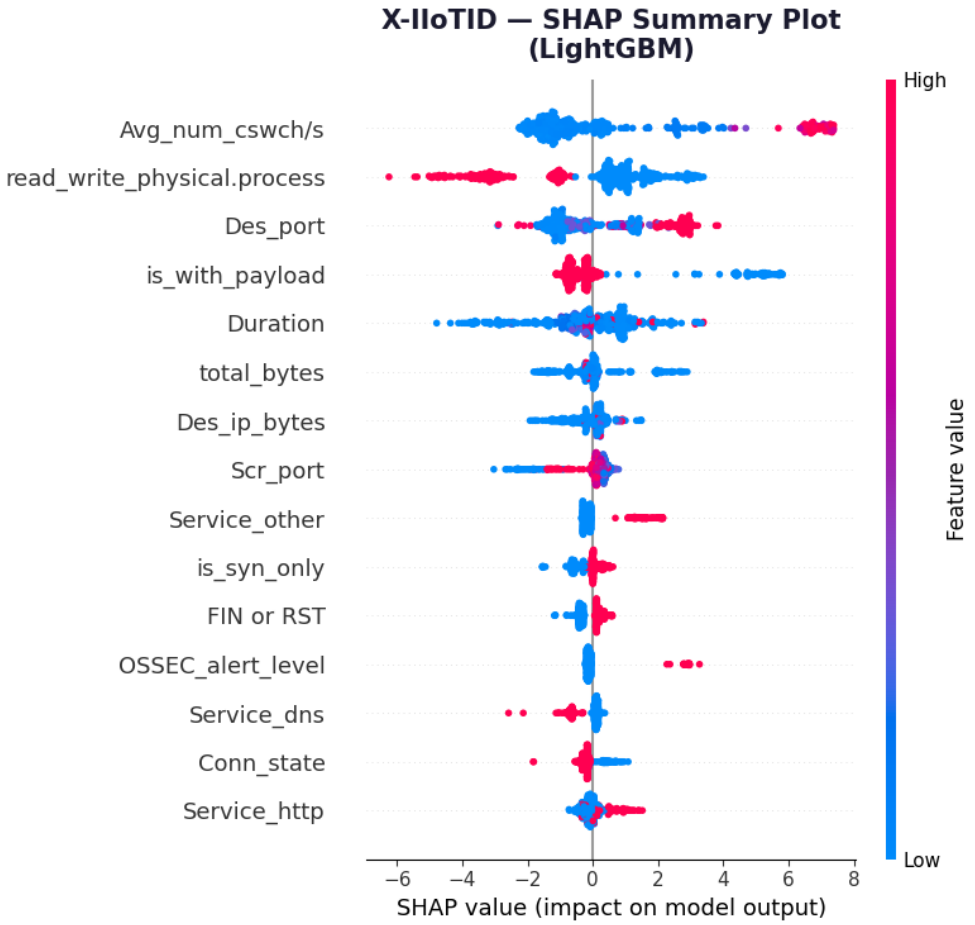


Figure 4.5: X-IIoTID SHAP Feature Importance (LightGBM)

4.1.5 Adaptive Risk Engine and BB84 QKD Results

The following subsection describes how the fixed 0.35/0.70 threshold criteria (subsection 3.2.6) divided the traffic from X-IIoTID in terms of LOW, MEDIUM, and HIGH categories, the reduction of QKD overhead brought by the division, and the performance of the BB84 simulator (subsection 3.2.7) under clean, low eavesdropping, and high eavesdrop.

Processing all 161,850 held-out test flows through the Adaptive Risk Engine produced the tier distribution in Table 4.2.

Table 4.2: X-IIoTID Adaptive Risk Tier Distribution

Tier	Flows	Percentage	Security Response
LOW	83,315	51.5%	AES-256 Only
MEDIUM	129	0.1%	AES-256 + Monitoring
HIGH	78,406	48.4%	BB84 QKD Triggered
Total	161,850	100%	

This adaptive method was able to save 51.6% overhead of QKD compared to the naive full-QKD implementation where BB84 protocol will be used for all 161,850 flows. Since the number of attacks makes up close to 50% of all flows (48.7% preprocessed), X-IIoTID provides the minimum 'headroom' for LOW to accept traffic among all three datasets. This is why the overhead saving (51.6%) of X-IIoTID is lower than other two even though it has a perfect classifier.

Table 4.3: X-IIoTID BB84 QKD Simulation Results

Scenario	Eavesdrop Prob.	Mean QBER	Secure Sessions	Key Outcome
Clean Channel	0.00	0.0000	100.0%	Generated
Low Eavesdrop	0.10	0.0252	100.0%	Generated
High Eavesdrop	0.75	0.1865	3.0%	Discarded

Accordingly with the security analysis of X-IIoTID that involves the 11% threshold, the clean channel QBER of 0.0000 for the dataset falls significantly below the threshold, whereas the high eavesdrop QBER of 0.1865 falls significantly above the threshold, and hence every high-eavesdrop session in this dataset correctly terminated with the discarding of the key.

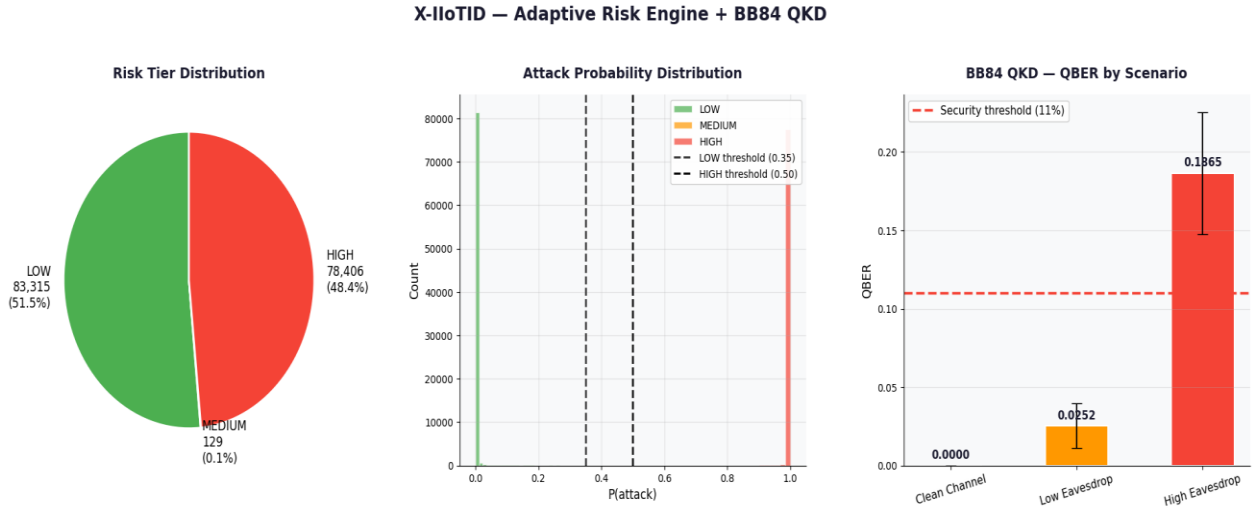


Figure 4.6: X-IIoTID Adaptive Risk Engine and BB84 QKD Results

4.1.6 Multiclass Attack-Type Model and Response Recommendation

The second part of this section goes beyond the binary attack or benign choice to determine which attack family each HIGH risk flow falls under, explaining how the prediction is used to determine an action through the mapping strategy as described in Table 3.7.

A second multiclass classifier was trained using under sampling and SMOTE oversampling (Section 3.2.9) for identifying the attack family for each flow, taking this method from the binary attack/benign classification to determining the nature of the attack. The classifier obtained a weighted F1 of 0.9881 (macro F1 0.9257, accuracy 0.9862).

Table 4.4: X-IIoTID Multiclass Attack-Type Classification Summary

Metric	Value
Accuracy	0.9862
Macro F1	0.9257
Weighted F1	0.9881
Number of Classes	19

The difference between macro F1 (0.9257, where all classes contribute equally) and weighted F1 (0.9881, where contribution is frequency-based) is indicative of the extent to which rare attack sub types (which are naturally more difficult to learn from a smaller number of thousands of hybrid-resampled attacks) affect the unweighted average compared to the weighted one.Which is more representative of real-world application, where popular attack types reign.

According to the Attack Type to Response mapping shown in Table 3.7, each of the 78,406 HIGH risk flows was assigned an actual security response based on the predicted attack family. The most common advice was “Rate-Limit/Traffic Shaping” (35.9% of HIGH risk flows), which was naturally determined by the prevalent attack type and associated reasoning for the recommended response shown in Section 3.2.8.

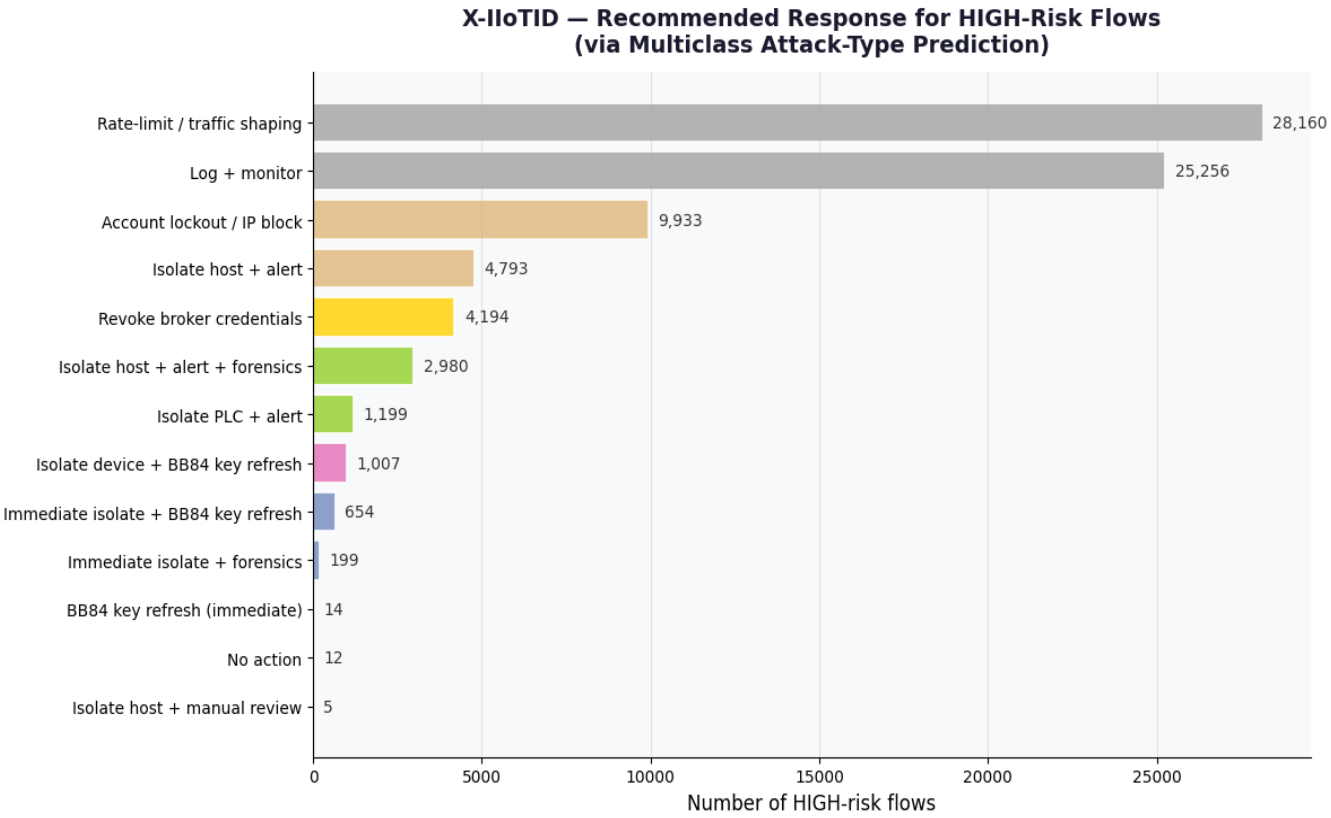


Figure 4.7: X-IIoTID Attack-Type-Aware Response Recommendation

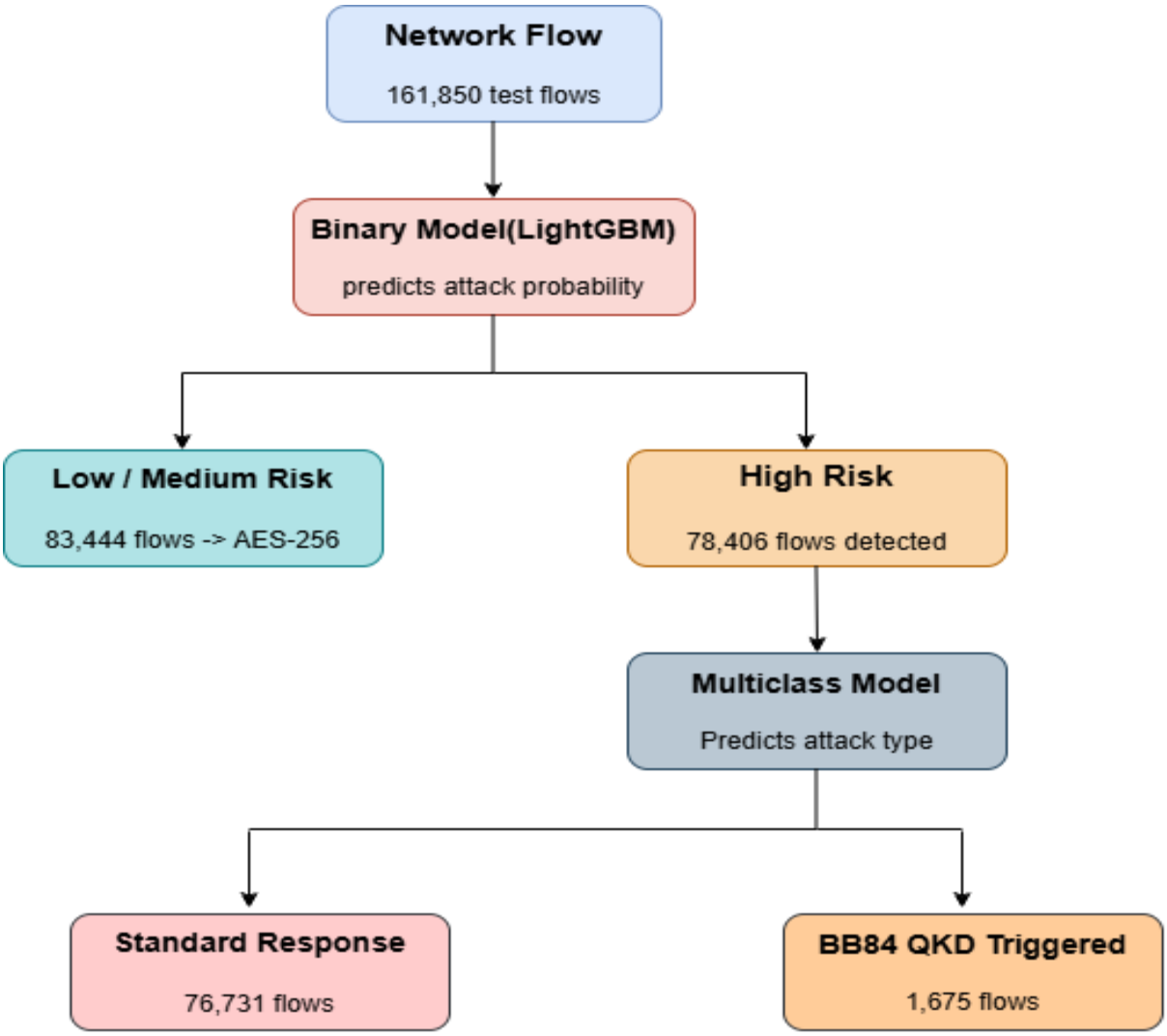


Figure 4.8: X-IIoTID Detection and Response Routing Pipeline

4.2 HQC-2026 (Bangladesh) Results

This section reports full results for HQC-2026, collected in Bangladesh (2026), following the identical methodology described in Chapter 3.

4.2.1 Exploratory Data Analysis

This subsection presents the class and attack-type distribution for HQC-2026 as observed after preprocessing, providing the baseline context for interpreting every result that follows.

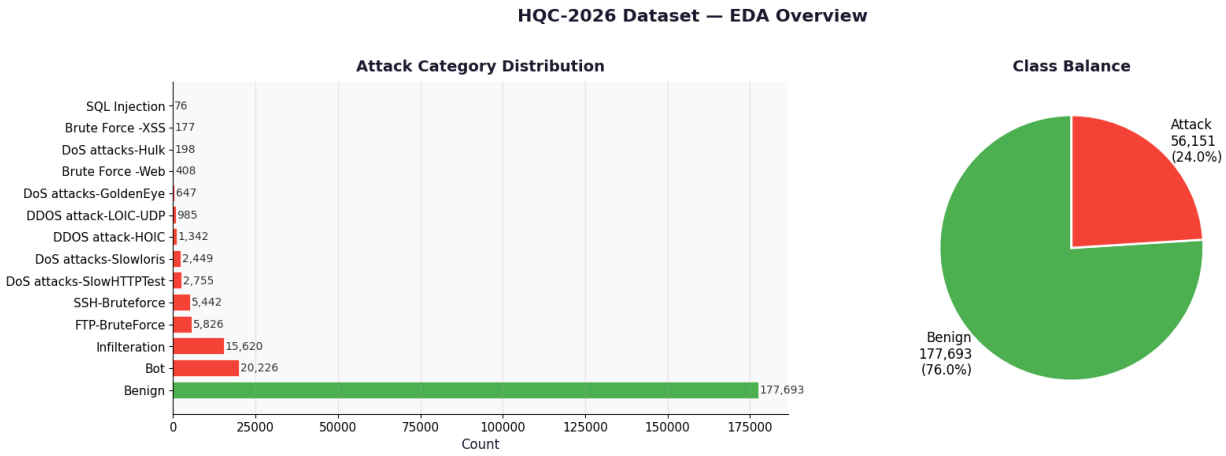


Figure 4.9: HQC-2026 Exploratory Data Analysis

4.2.2 Sampling Stability Check

Prior to conducting any further training, each of the twelve models was first trained using three 50,000 rows of stratified data, which do not overlap, in order to ensure that the sampling technique did not introduce any bias. The results in Table 4.7 clearly indicate that LightGBM performed the best among all the models, with an average F1 score of 0.7966 and a very low standard deviation of 0.0026.

4.2.3 Model Performance on the Merged 150k Training Set

This sub-section details the performance of all the twelve models after re-training with the entire merged dataset of 150,000 rows, which one of the twelve models came out tops and why, as well as the information provided by the confusion matrix and ROC curve for each of the models.

Following the merging of the three datasets into an entire pool of 150,000 rows (SVM and stacking limited to 50,000 rows), all the twelve models were re-trained and evaluated against the test data in its original form. Table 4.6 contains the comparison results in full, with LightGBM producing the best F1-score of 0.8082 (AUC-ROC 0.9506 and Accuracy 0.9147).

Table 4.5: HQC-2026 Comparison of All 12 ML Models

Model	Accuracy	F1-Score	AUC-ROC
Random Forest	0.9030	0.7852	0.9388
Gradient Boosting	0.9033	0.7869	0.9426
XGBoost	0.9110	0.8015	0.9470
SVM	0.8962	0.7777	0.9294
Voting Ensemble	0.9076	0.7939	0.9428
Stacking	0.8975	0.7836	0.9442
LightGBM	0.9147	0.8082	0.9506
MLP Neural Net	0.8966	0.7859	0.9445
CatBoost	0.8976	0.7756	0.9361
Extra Trees	0.8887	0.7515	0.9178
AdaBoost	0.8097	0.6372	0.8539
Naive Bayes	0.7761	0.4087	0.7414

However, LightGBM still perform better than XGBoost by a much smaller difference (F1 0.8015) when compared to the X-IIoTID dataset, and each model's F1 score is notably lower on the HQC-2026 dataset compared to the others (best F1 0.8082 versus 0.9971 and 0.9580) due to the very skewed multi-attack distribution of the attacks in HQC-2026 (24.0%) most of the attacks belong to either Bot or Infilteration class, whereas other attacks have less than 700 training instances. In contrast, the GOSS sampling that keeps the instances with high gradients (harder to classify) during the training process makes LightGBM unique on such a highly skewed dataset compared to the uniform treatment of the instances in Random Forest and CatBoost. Naive Bayes fails in achieving even an acceptable F1-score of 0.4087 on the HQC-2026 dataset as the class conditional Gaussian distributions are calculated from very few instances in many classes, further worsening the effect of independence assumption violation.

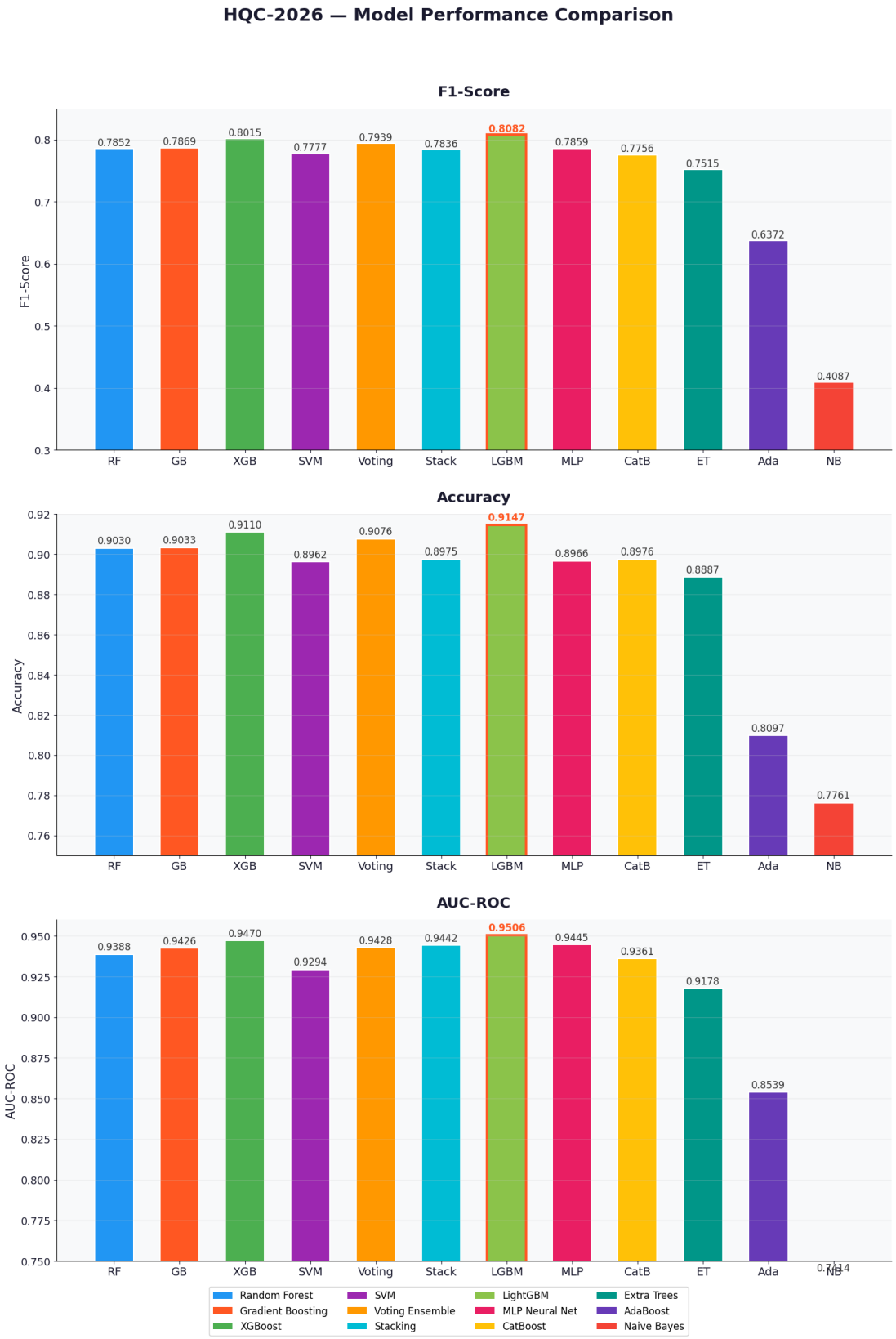


Figure 4.10: HQC-2026 Model Performance Comparison

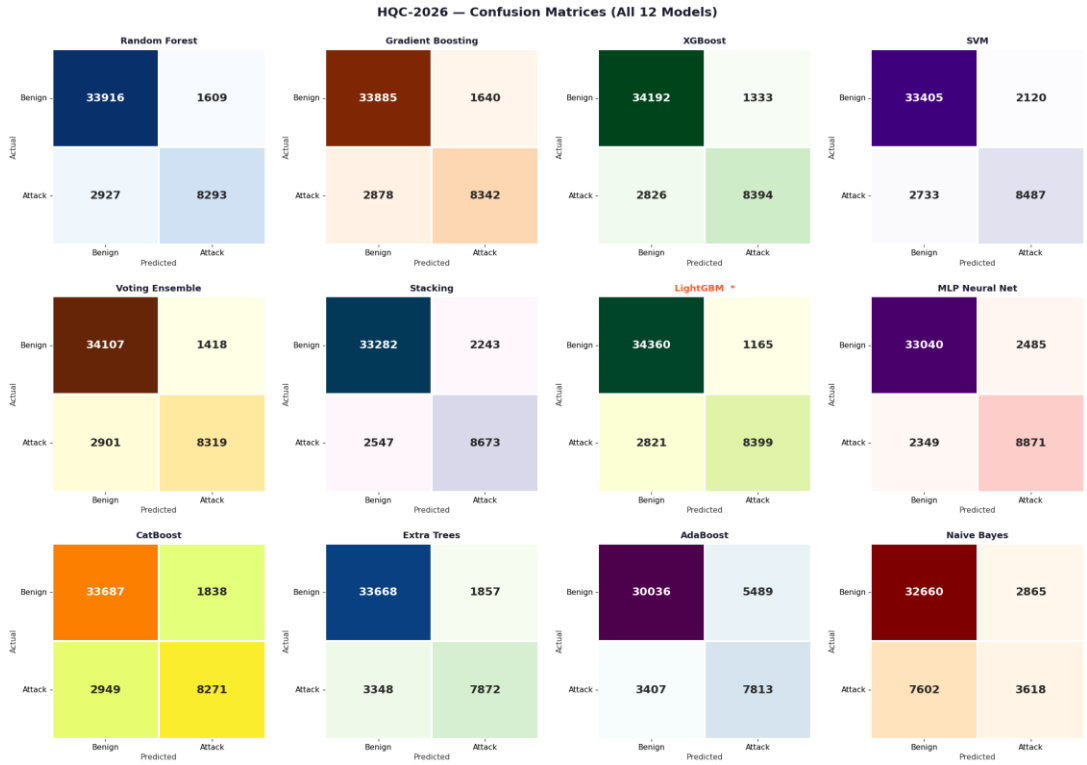


Figure 4.11: HQC-2026 Confusion Matrices

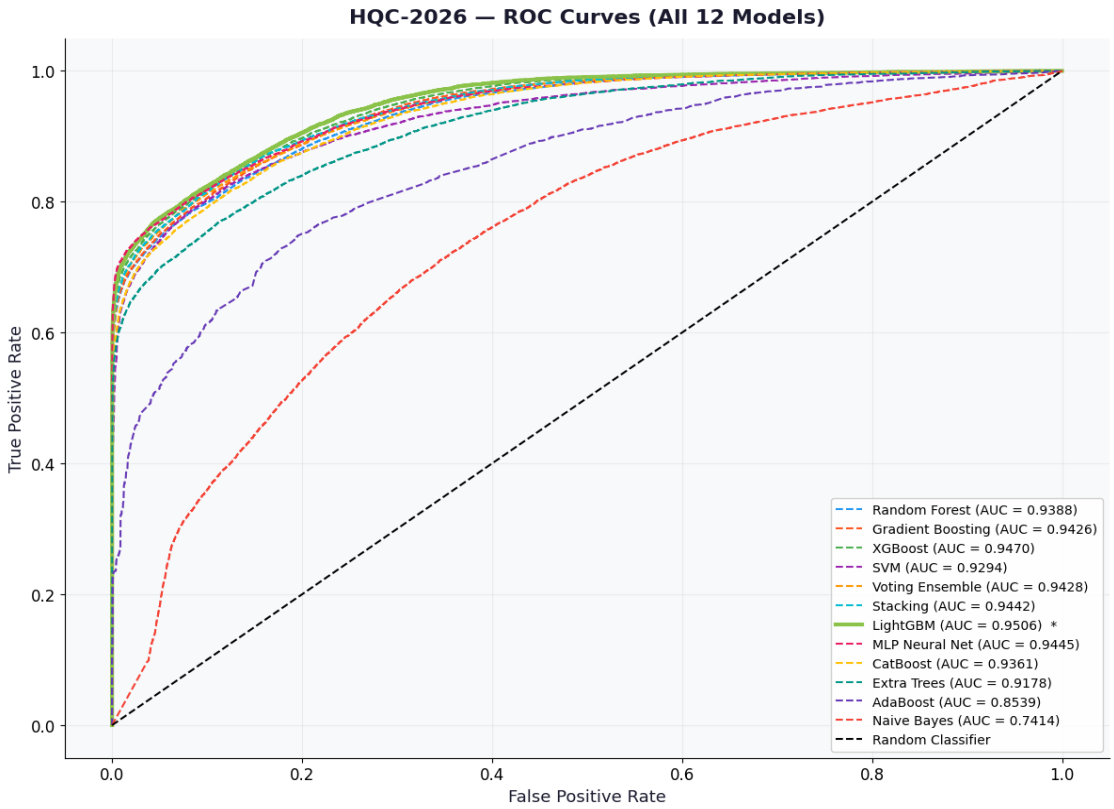


Figure 4.12: HQC-2026 ROC Curves

4.2.4 SHAP Explainability

This section highlights, one by one, those signals that LightGBM effectively used to distinguish between attacks and benign traffic in HQC-2026 in terms of SHAP values, which are mathematically defined in Section.

SHAP values were calculated for LightGBM on a 500 row sample from the held-out test dataset using TreeExplainer method (which is an efficient implementation of SHAP values for tree-based models). Fwd Seg Size Min was the highest ranking feature with the other top 10 features. This dataset was collected using CICFlowMeter, the same software used to collect CIC-UNSW-NB15 dataset and the two have the same highest-ranking feature despite being collected in two different countries at two different times namely, Fwd Seg Size Min.

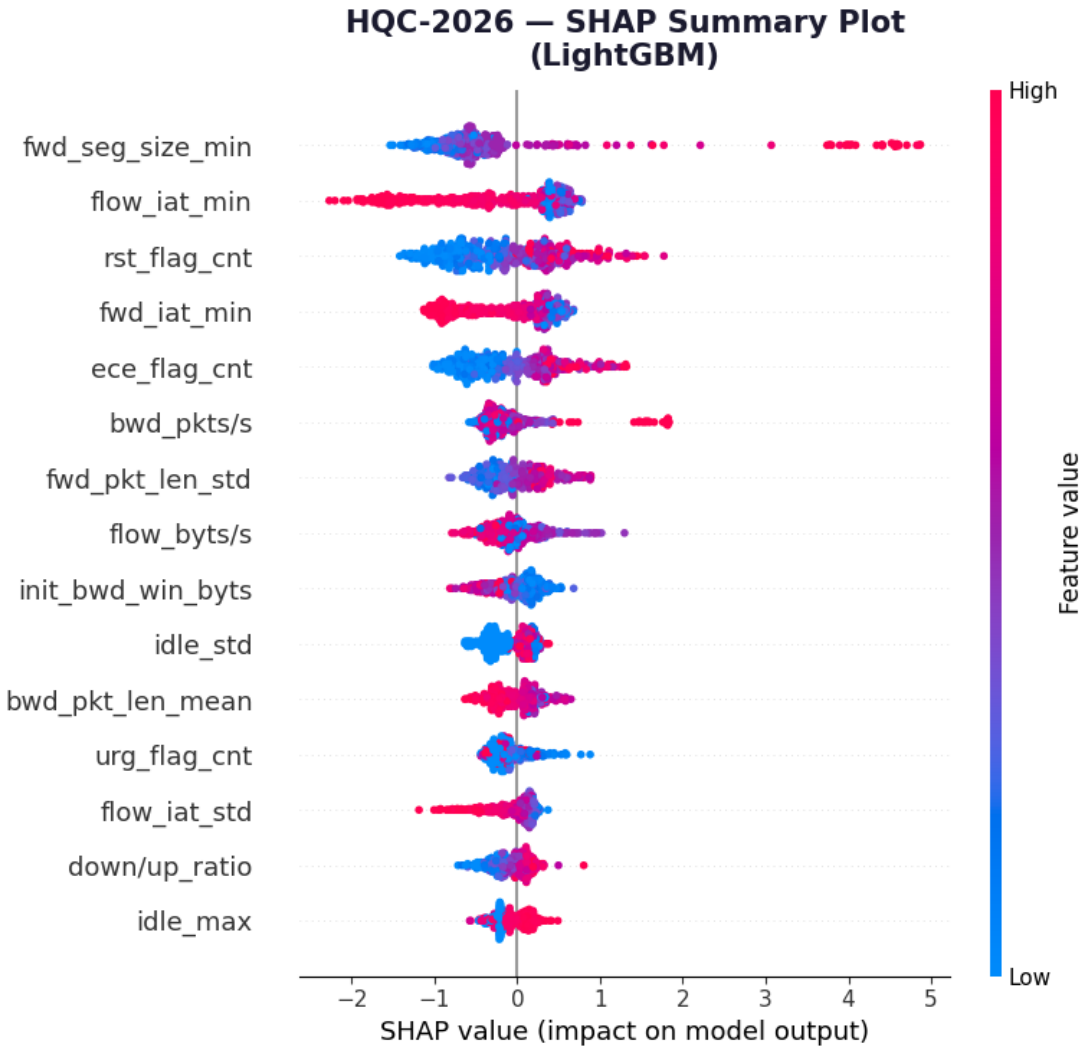


Figure 4.13: HQC-2026 SHAP Feature Importance (LightGBM)

4.2.5 Adaptive Risk Engine and BB84 QKD Results

The current sub-section discusses how the static 0.35/0.70 risk threshold values ended up dividing the HQC-2026 data set into LOW, MEDIUM, and HIGH tiers; how much QKD overhead the division resulted in; and how the BB84 model performed in clean, low, and high eavesdropping environments, respectively, on this particular dataset.

All 46,745 flows from the held-out test set were processed using the Adaptive Risk Engine.

Table 4.6: HQC-2026 Adaptive Risk Tier Distribution

Tier	Flows	Percentage	Security Response
LOW	34,580	74.0%	AES-256 Only
MEDIUM	3,518	7.5%	AES-256 + Monitoring
HIGH	8,647	18.5%	BB84 QKD Triggered
Total	46,745	100%	

The adaptive technique was able to reduce QKD overhead by 81.5% compared to a simple full QKD solution that would start BB84 for all 46,745 flows. The very low attack occurrence rate of HQC-2026 (24.0%) results in a significantly higher number of traffic to be handled by the LOW category compared to X-IIoTID, with the same set of fixed thresholds (0.35/0.70) resulting in a significantly larger 81.5% overhead reduction for this specific data set, the largest of all three sets. This is evident in the much larger MEDIUM category for this data set (7.5%, compared to 0.1% of the other two).

Table 4.7: HQC-2026 BB84 QKD Simulation Results

Scenario	Eavesdrop Prob.	Mean QBER	Secure Sessions	Key Outcome
Clean Channel	0.00	0.0000	100.0%	Generated
Low Eavesdrop	0.10	0.0282	100.0%	Generated
High Eavesdrop	0.75	0.1888	0.0%	Discarded

In line with the security proof involving the 11%-threshold outlined in Section 3.2.7, the 0.0000 QBER in HQC-2026 for the clean channel is well below the threshold, while the 0.1888 QBER in the high eavesdrop channel is well above it, and thus all high-eavesdrop simulations for this dataset properly discarded keys.

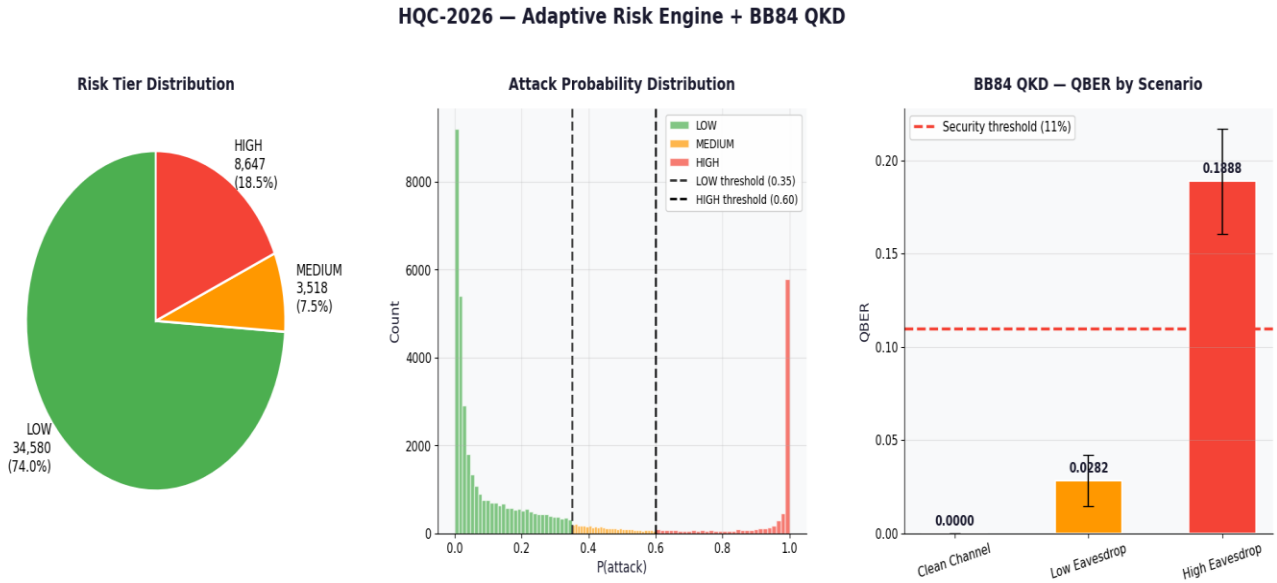


Figure 4.14: HQC-2026 Adaptive Risk Engine and BB84 QKD Results

4.2.6 Multiclass Attack-Type Model and Response Recommendation

From the binary attack, this subsection shifts focus on classifying each of the HIGH flows into their corresponding attack family and demonstrates how this classification can be used to take an action based on the reasoning provided in Table 3.7.

To classify each flow into its respective attack family, another multi-class classifier was trained by applying both under sampling and SMOTE rebalancing techniques (Section 3.2.9) with a weighted F1 score of 0.7996 (macro F1 0.7411, accuracy 0.7574). This is summarized in Table 4.10.

Table 4.8: HQC-2026 Multiclass Attack-Type Classification Summary

Metric	Value
Accuracy	0.7574
Macro F1	0.7411
Weighted F1	0.7996
Number of Classes	14 (incl. Rare Attack)

The difference between macro F1 score (0.7411, which considers all classes as equal) and the weighted F1 score (0.7996, which takes into account the frequencies of attack subtypes) indicates the extent to which more rarely uncommon attack types, which are difficult to identify from a small number of hybrid resampling examples, lower the unweighted mean compared to the weighted mean more characteristic for actual application.

According to Attack-Type-to-Response Map in Table 3.7, each of the 8,647 HIGH risk flows received specific instructions on how to deal with it depending on its attack family. The most common response was “Isolate host + block C2 traffic” (44.0% of HIGH risk flows), indicating the frequency of such an attack type in this HIGH risk traffic.

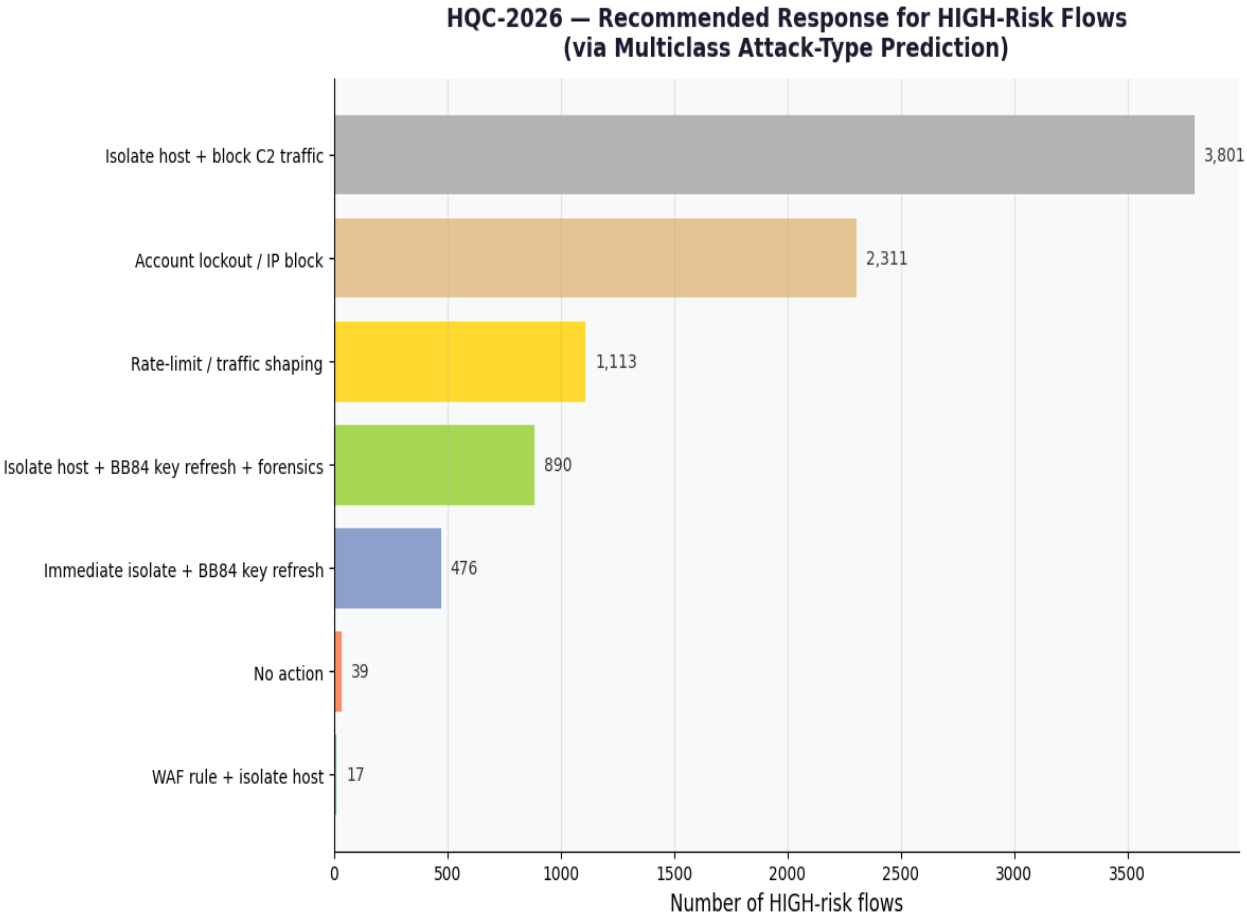


Figure 4.15: HQC-2026 Attack-Type-Aware Response Recommendation

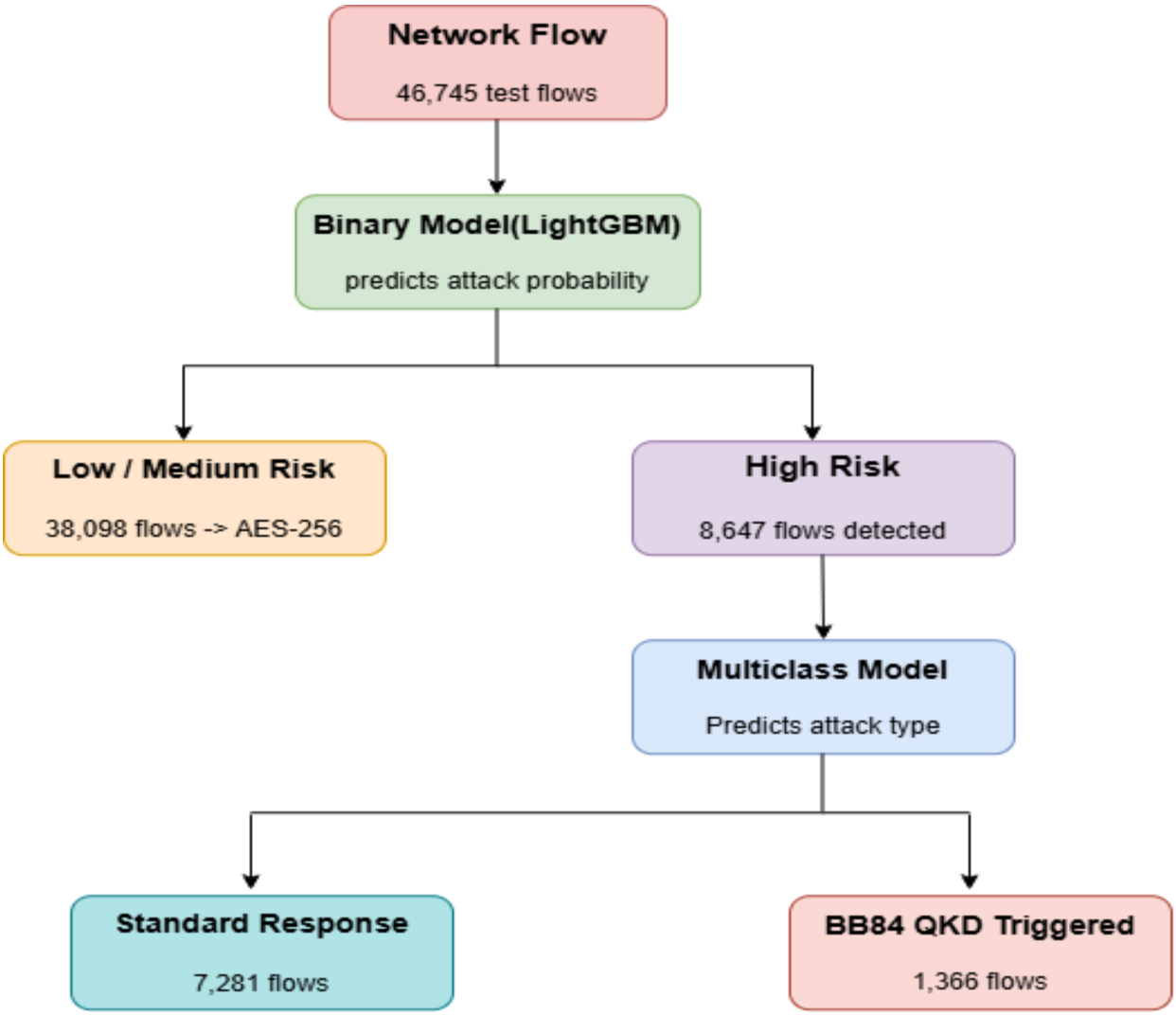


Figure 4.16: HQC-2026 Detection and Response Routing Pipeline

4.3 CIC-UNSW-NB15 (Canada) Results

This section reports full results for CIC-UNSW-NB15, collected in Canada (2024), following the identical methodology described in Chapter 3.

4.3.1 Exploratory Data Analysis

This subsection presents the class and attack-type distribution for CIC-UNSW-NB15 as observed after preprocessing, providing the baseline context for interpreting every result that follows.

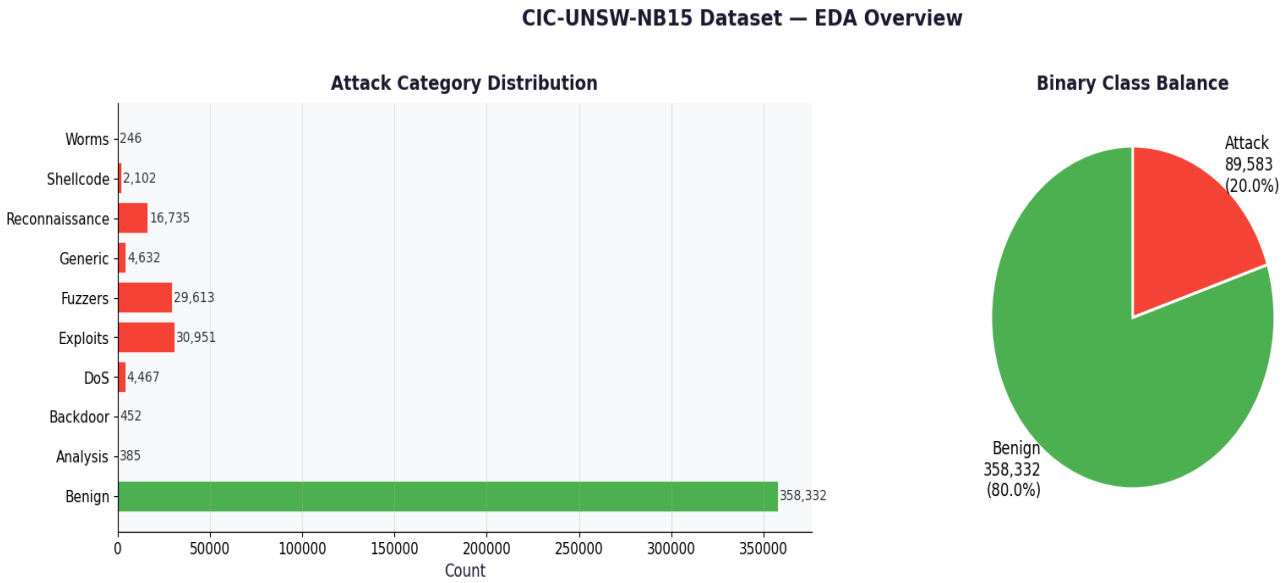


Figure 4.17: CIC-UNSW-NB15 Exploratory Data Analysis

4.3.2 Sampling Stability Check

Before training, all the models were trained separately on three different samples of 50,000 rows each using the same process of stratified sampling to test the unbiasedness of the sampling process as described in Section 3.2.1. According to Table 4.12, the Stacking algorithm emerged as the best performing model in terms of the sample used, with an average F1 score of 0.9571 and a very small standard deviation of 0.0002 indicating an unbiased sampling process.

4.3.3 ML Model Performance

This section discusses the performance of all twelve models when retrained using the entire merged dataset containing 150,000 rows, the winner of the competition, reasons for choosing it, and what the confusion matrix and ROC curve tell us about the error pattern of each model.

All the twelve models were retrained after merging the three samples to form a single pool containing 150,000 rows (with SVM and Stacking limited to 50,000). LightGBM came out on top, attaining a score of 0.9580 (AUC-ROC 0.9960, Accuracy 0.9826). Table 4.11 describes the results of the analysis.

Table 4.9: CIC-UNSW-NB15 Comparison of All 12 ML Models

Model	Accuracy	F1-Score	AUC-ROC
Random Forest	0.9813	0.9554	0.9958
Gradient Boosting	0.9819	0.9565	0.9958
XGBoost	0.9824	0.9576	0.9960
SVM	0.9788	0.9497	0.9930
Voting Ensemble	0.9819	0.9566	0.9959
Stacking	0.9819	0.9564	0.9953
LightGBM	0.9826	0.9580	0.9960
MLP Neural Net	0.9806	0.9537	0.9949
CatBoost	0.9813	0.9552	0.9955
Extra Trees	0.9795	0.9512	0.9954
AdaBoost	0.9680	0.9258	0.9865
Naive Bayes	0.9054	0.8021	0.9645

LightGBM leads the pack in terms of margins, though by smaller margins compared to X-IIoTID. However, its overall performance is significantly better than HQC-2026 (best F1 0.9580), owing to CIC-UNSW-NB15 having a comparatively lower 20.0% attack rate and well-engineered flow timing characteristics obtained from CICFlowMeter. Stacking (F1 0.9564) and Voting Ensemble (F1 0.9566) both come very close to LightGBM in this case, closer than in any other data sets again reflecting the sample level stability analysis where Stacking was found to be better than LightGBM in Section 4.3.2 until a large-scale pooled training set gave LightGBM an advantage over Stacking.

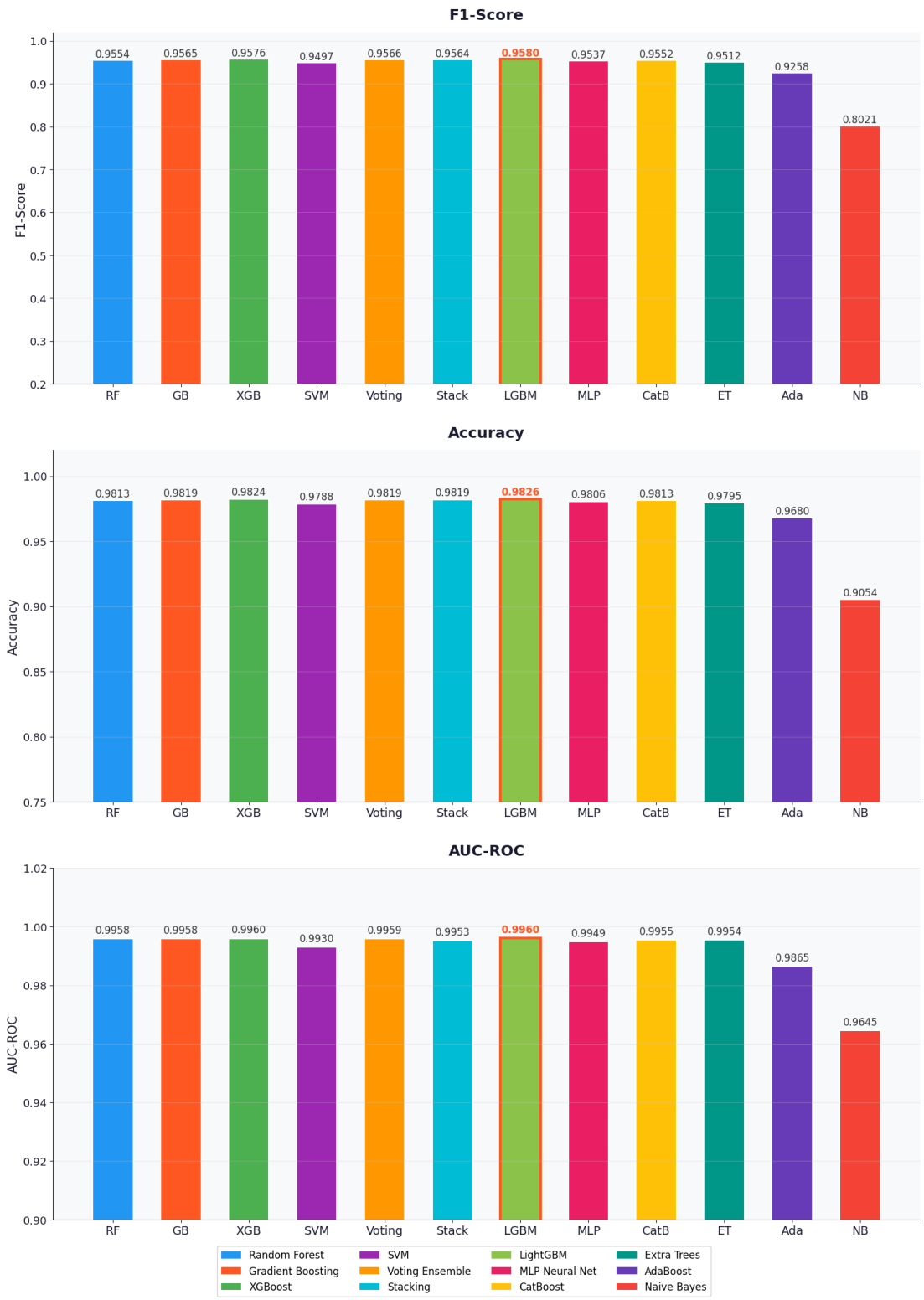


Figure 4.18: CIC-UNSW-NB15 Model Performance Comparison

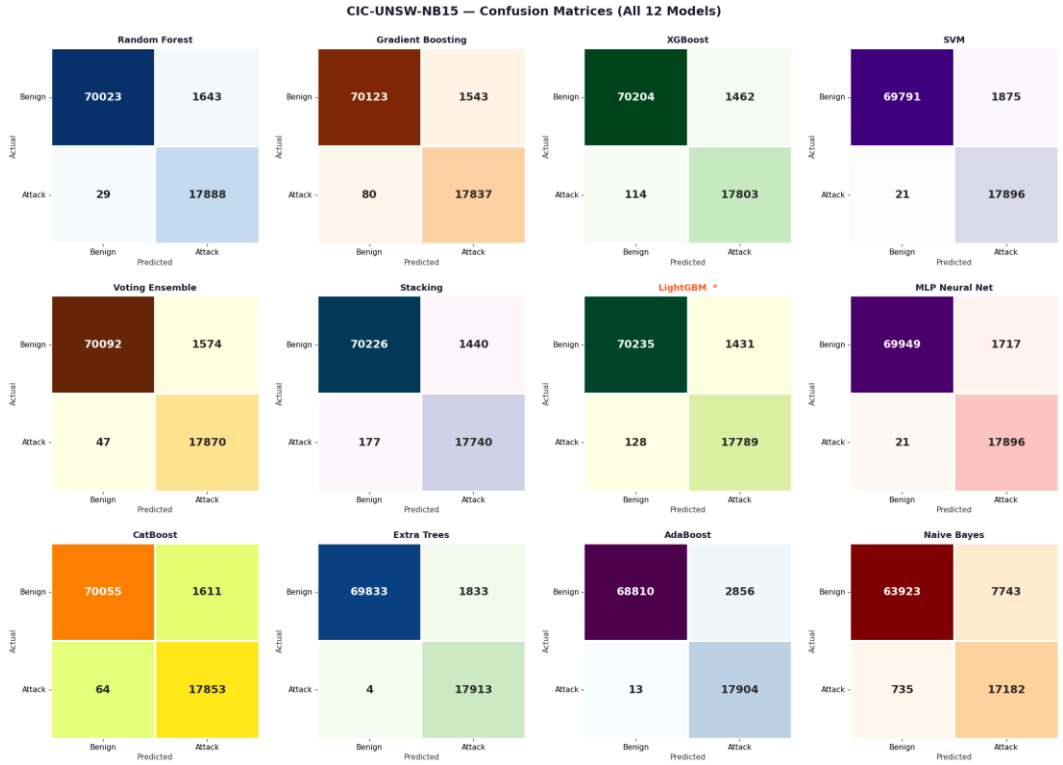


Figure 4.19: CIC-UNSW-NB15 Confusion Matrices

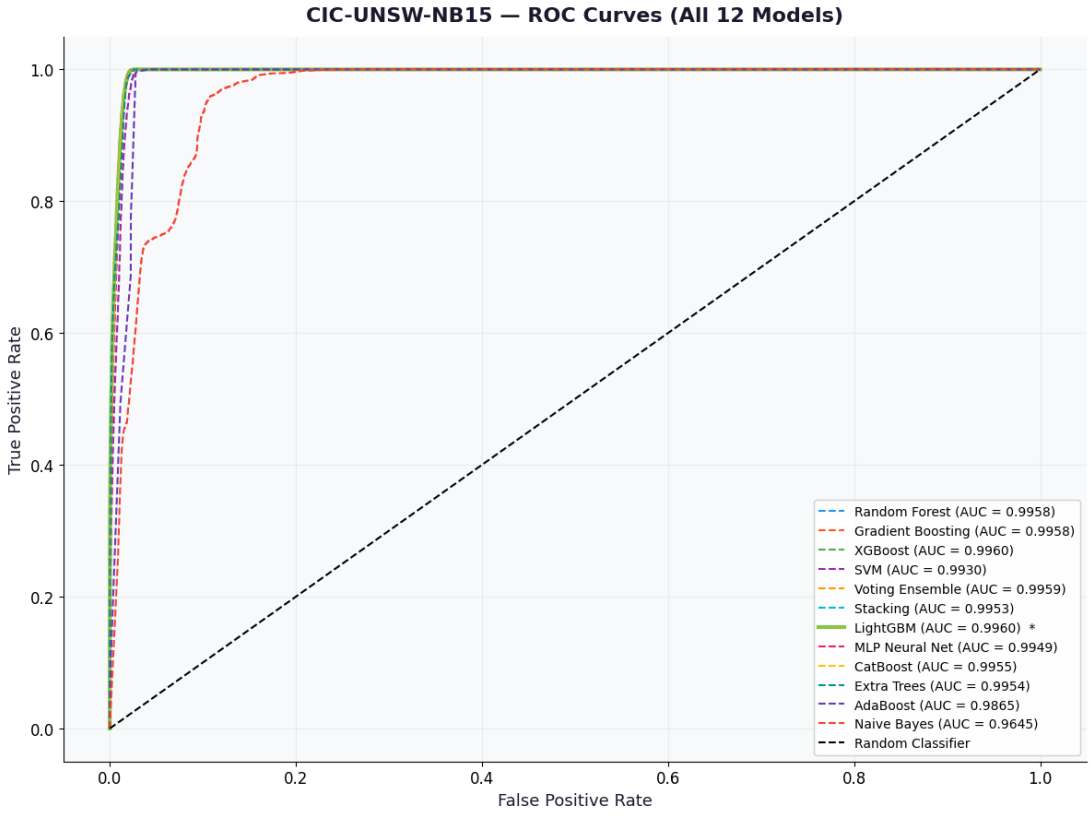


Figure 4.20: CIC-UNSW-NB15 ROC Curves

4.3.4 SHAP Explainability

In this subsection, it is elaborated, one feature at a time, how the LightGBM model differentiate between attacks and benign activities in CIC-UNSW-NB15 based on its feature importance values obtained by using SHAP values as mathematically defined in Section 3.2.4.

SHAP values were calculated for LightGBM on a 500 row subset of the test dataset using the TreeExplainer technique (efficient exact computation of SHAP values tailored to tree-based models). The most important feature was Fwd Seg Size Min, followed by the other 9 highest ranked features as shown in the figure below. Just like in the HQC-2026 dataset, the most important feature in this CICFlowMeter generated dataset is also Fwd Seg Size Min.

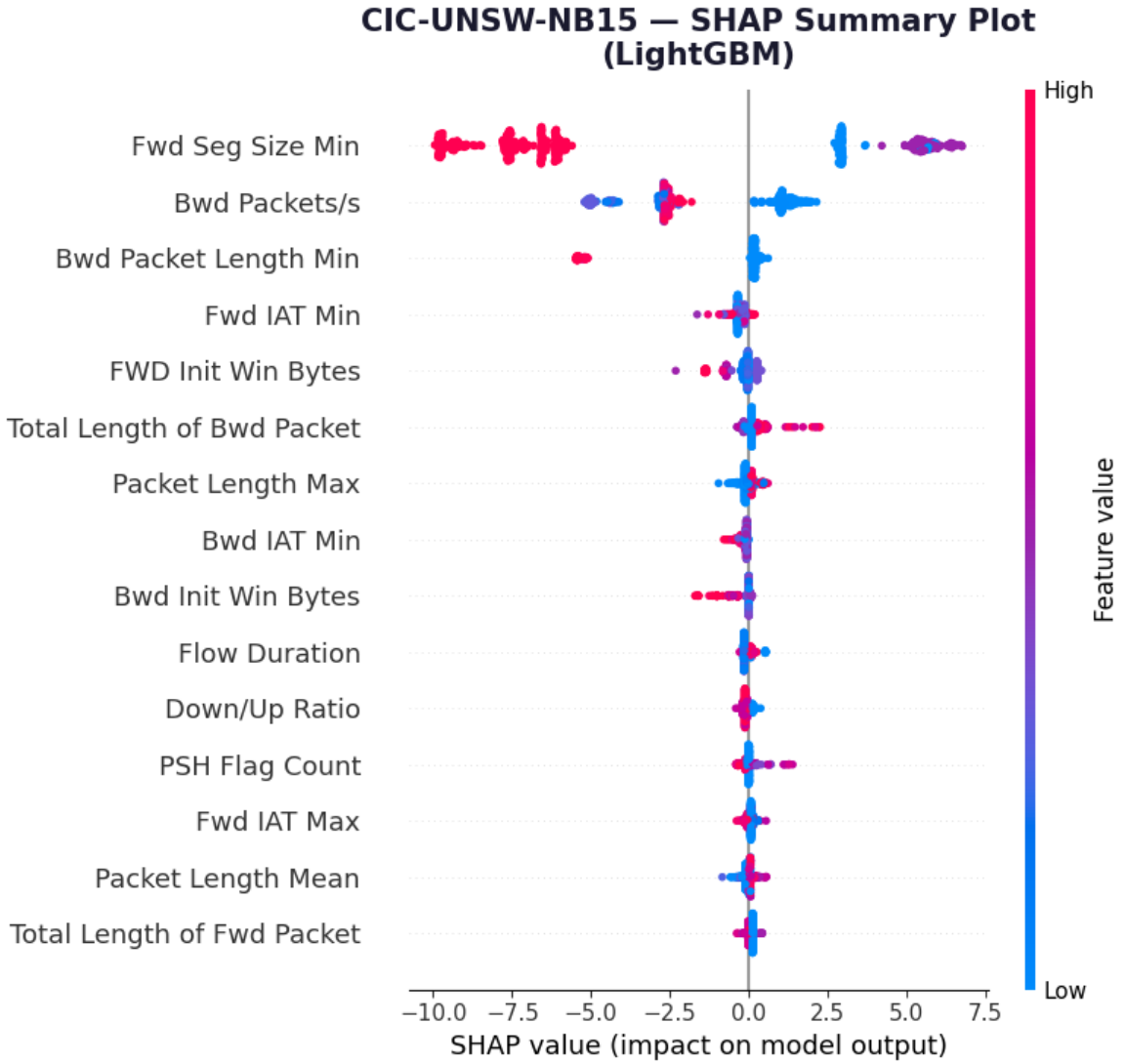


Figure 4.21: CIC-UNSW-NB15 SHAP Feature Importance (LightGBM)

4.3.5 Adaptive Risk Engine and BB84 QKD Results

This sub section discusses the actual division of the CIC-UNSW-NB15 traffic data by the fixed thresholds of 0.35/0.70 (see Section 3.2.6), the amount of QKD overhead this division saved, and performance of the BB84 experiment (see Section 3.2.7) performed in a clean, low-eavesdrop, and high-eavesdrop environment on this particular dataset.

Processing of all 89,583 test flows resulted in the following tier distribution (Table 4.13).

Table 4.10: CIC-UNSW-NB15 Adaptive Risk Tier Distribution

Tier	Flows	Percentage	Security Response
LOW	70,200	78.4%	AES-256 Only
MEDIUM	112	0.1%	AES-256 + Monitoring
HIGH	19,271	21.5%	BB84 QKD Triggered
Total	89,583	100%	

As per the adaptive scheme, the overhead of QKD was reduced by 78.5% compared to a basic scheme of implementing QKD where the threshold was set in such a way that BB84 is triggered in all of the 89,583 flows. The rate of attacks of 20% of CIC-UNSW-NB15 is between the almost equal distribution of traffic of X-IIoTID and the highly unbalanced distribution of HQC-2026, and consequently, the overhead reduction of 78.5% is also between the two.

Table 4.11: CIC-UNSW-NB15 BB84 QKD Simulation Results

Scenario	Eavesdrop Prob.	Mean QBER	Secure Sessions	Key Outcome
Clean Channel	0.00	0.0000	100.0%	Generated
Low Eavesdrop	0.10	0.0245	100.0%	Generated
High Eavesdrop	0.75	0.1857	1.0%	Discarded

As mentioned in Section 3.2.7 about the security analysis using an 11% QBER threshold, CIC-UNSW-NB15's QBER in clean channel (0.0000) is clearly lower than the threshold whereas its high eavesdrop QBER (0.1857) is clearly higher than the threshold, thus every high eavesdrop session simulation using this dataset will correctly discard the key.

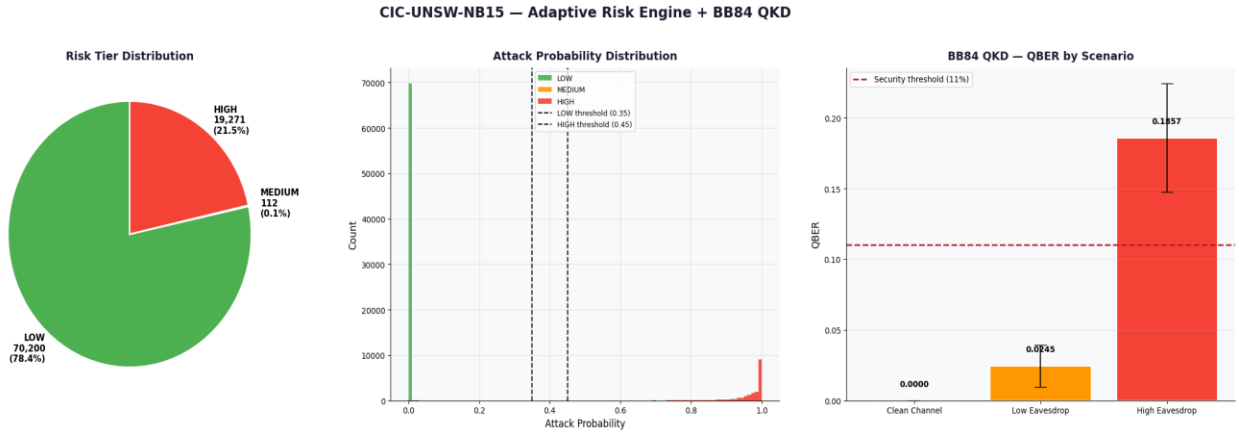


Figure 4.22: CIC-UNSW-NB15 Adaptive Risk Engine and BB84 QKD Results

4.3.6 Multiclass Attack-Type Model and Response Recommendation

In this subsection, there is the shift from attack/benign classification to attack families to which HIGH flows belong, with an example of how such classification can be used for making a decision on the real security actions according to the reasoning presented in Table 3.7.

The second multiclass classifier was trained using under sampling with SMOTE to determine the attack family for each HIGH flow. It reached a weighted F1 of 0.9350 (macro F1 0.6402, accuracy 0.9323). This result is shown in Table 4.15.

Table 4.12: CIC-UNSW-NB15 Multiclass Attack-Type Classification Summary

Metric	Value
Accuracy	0.9323
Macro F1	0.6402
Weighted F1	0.9350
Number of Classes	10

The difference between the macro F1 score (0.6402 gives equal importance to all classes) and the weighted F1 score (0.9350 gives importance to classes according to their frequencies) is due to the effect of the rarer attack classes, which are hard to learn from a few thousand hybrid resampling cases, in the unweighted average compared to the weighted one that considers the reality of using it in real life.

According to the mapping defined in Table 3.7, for each of the 19,271 HIGH risk flows, an action was determined based on the predicted attack type. The most common recommended action was "log + monitor" (61.4% of the HIGH risk flows), which is a direct indication of the attack pattern of the HIGH risk flows of this dataset.

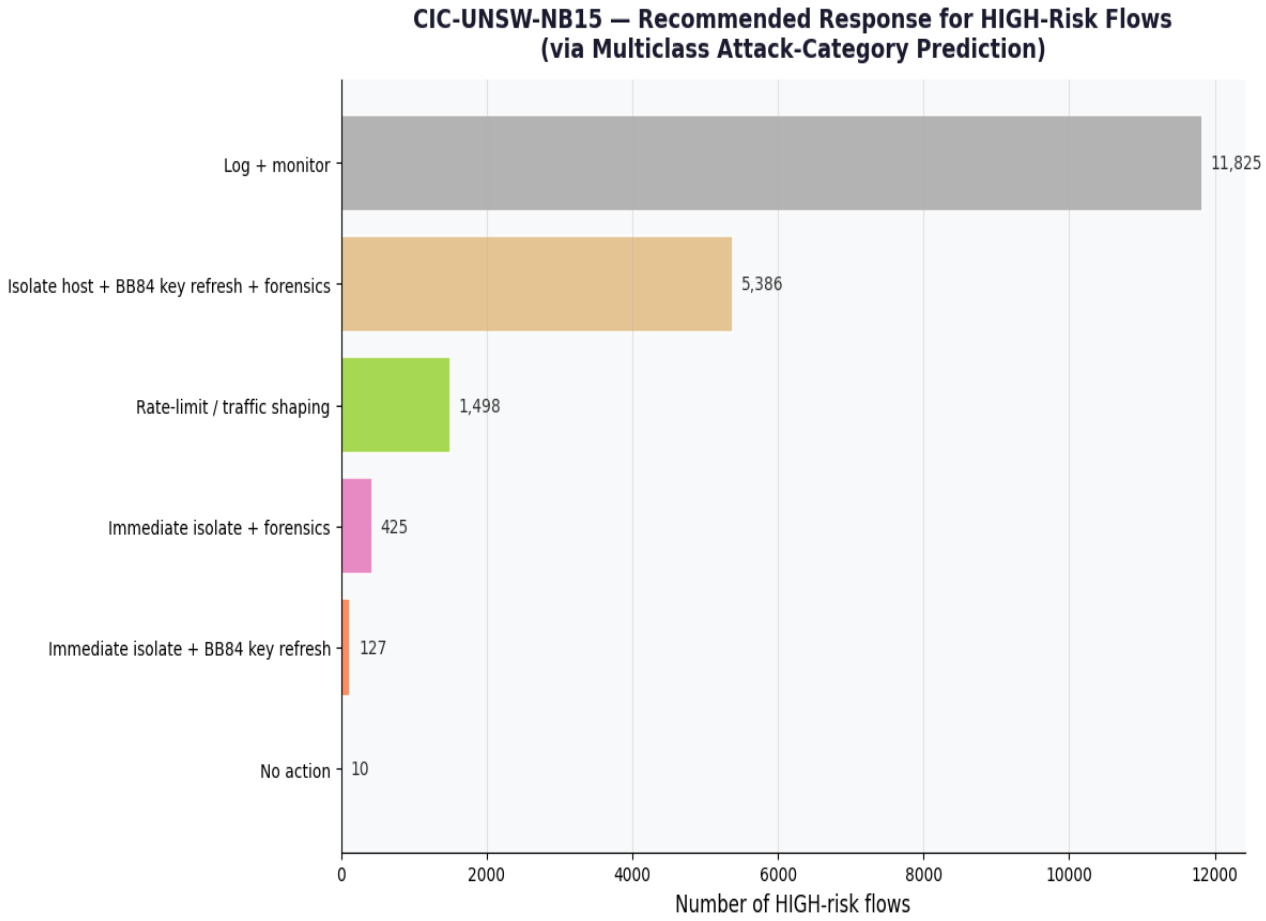


Figure 4.23: CIC-UNSW-NB15 Attack-Type-Aware Response Recommendation

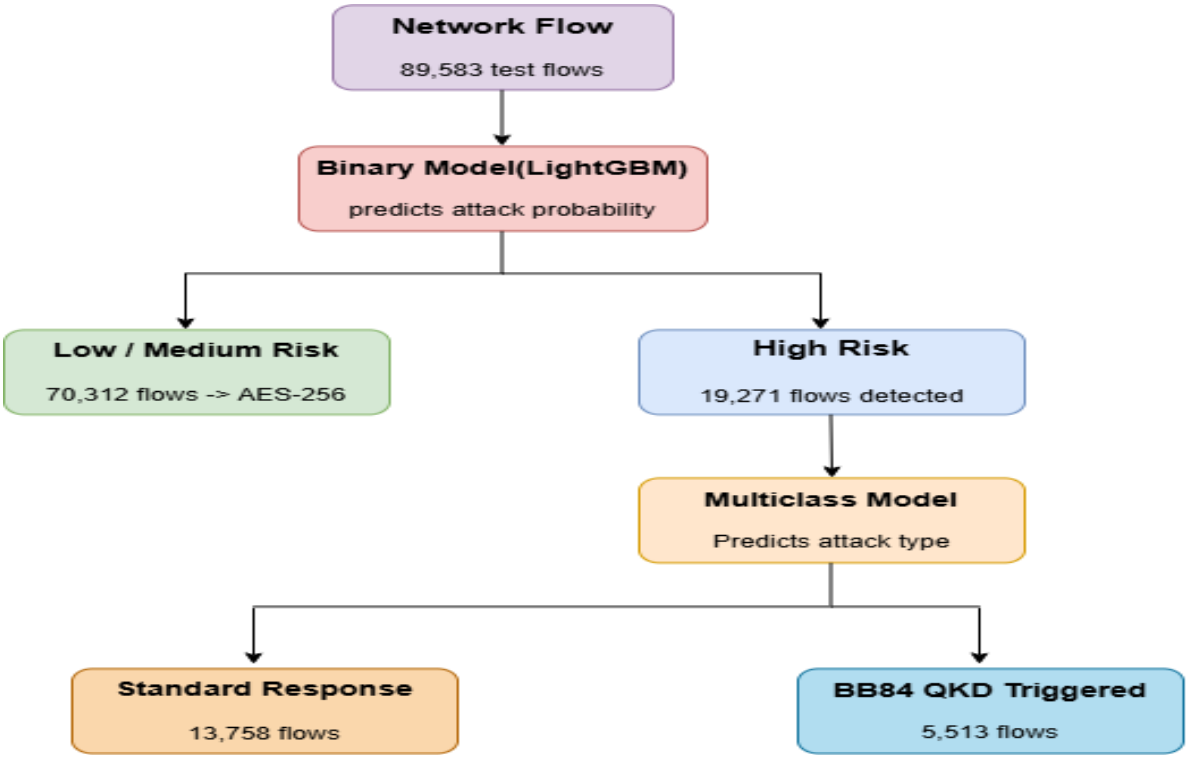


Figure 4.24: CIC-UNSW-NB15 Detection and Response Routing Pipeline

Because CIC-UNSW-NB15 is the third and final dataset processed in the pipeline, the step-by-step BB84 protocol diagram (previously shown as the general Figure 3.13 in Chapter 3) is repeated here with this dataset's own eavesdropping results for direct comparison.

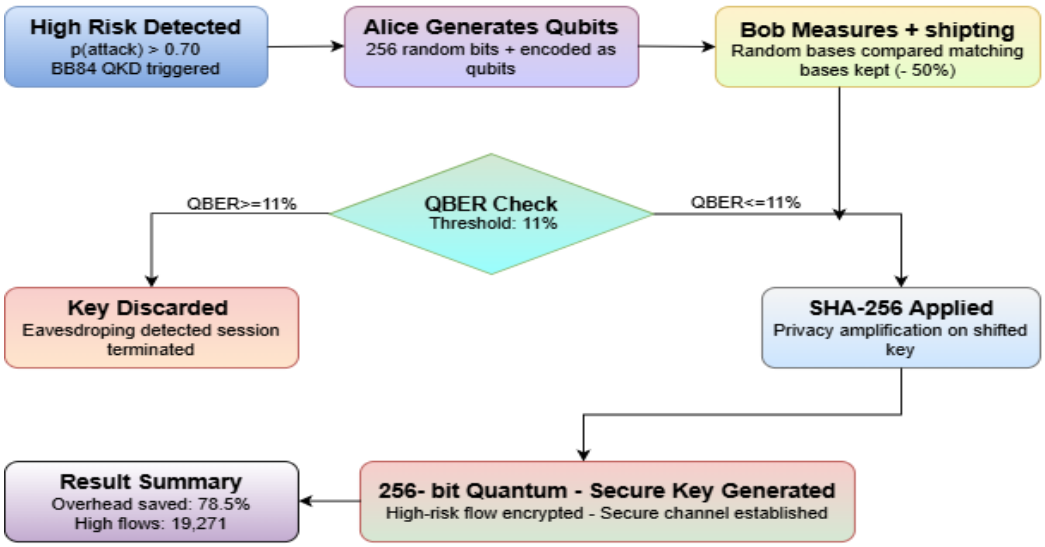


Figure 4.25: CIC-UNSW-NB15 BB84 QKD Protocol Step-by-Step Pipeline

4.4 Cross-Dataset Comparison

This section combines the findings from these three databases to investigate the key question posed by this generalized experiment does this adaptive ML QKD framework, which has been tested on a single database in the Capstone-B prototype, continue to work effectively when used on other datasets from three different countries?

Table 4.13: Cross-Dataset Final Summary Best Model, F1, AUC, Overhead Saved

Dataset	Country	Year	Best Model	F1	AUC	Accuracy	Overhead Saved
X-IIoTID	Australia	2021	LightGBM	0.9971	0.9999	0.9972	51.6%
HQC-2026	Bangladesh	2026	LightGBM	0.8082	0.9506	0.9147	81.5%
CIC-UNSW-NB15	Canada	2024	LightGBM	0.9580	0.9960	0.9826	78.5%

LightGBM emerged as the best binary classifier out of the merged pool for all three data sets. Even though there were notable differences among those data sets in terms of their country of origin, feature set, class imbalance, and nature of attacks. This is the strongest evidence that this project can provide that the model selection result from the previous prototype using a single dataset was not merely due to that particular dataset. It should be pointed out, however, that the sample stability test conducted earlier (Section 3.2.1 / 4.3.2) actually favored Stacking for the CIC-UNSW-NB15 data set before the merged pool was created proof that results from smaller sample sizes do not always reflect the winning algorithm after using the entire merged pool.



Figure 4.26: Cross-Dataset Model Performance Comparison

Why LightGBM Was the Best Model on All Three Datasets?

This section provides further explanation for the performance of LightGBM versus the other eleven models, why it was selected to win all competitions. In short, the success of LightGBM is not just due to chance, as it is successful on each of the three data sets, with scale, imbalance and redundant features as problems in each of X-IIoTID, HQC-2026, and CIC-UNSW-NB15.

As discussed mathematically the four key aspects underlying LightGBM include:

(1) Leaf-wise tree construction that splits on the single leaf in the entire tree that offers the greatest reduction in losses compared to the potential increase in each leaf in the current depth before splitting further to attain greater accuracy with fewer splits.

(2) The process of histogram-based binning where continuous feature values are discretized into bins prior to making the split. Thus turning the costly sorted value search into a constant time lookup operation.

(3) GOSS: Gradient-based One-Side Sampling keeps all the training samples with large gradients while only random subset of samples with small gradients is selected. The selected samples are then reweighted to remain the sampling unbiased

(4) EFB: Exclusive Feature Bundling combines multiple mutually exclusive features into a single feature to reduce the data size and dimensionality. For each dataset's most suitable distinctive features were selected based on how they fit with the LightGBM algorithm.

Table 4.14: Why LightGBM Won Dataset of all Datasets

Dataset	Distinguishing Characteristic	LightGBM Mechanism That Exploited It	Result
X-IIoTID	Largest, most well-populated pool (809,249 cleaned rows); near-balanced 48.7% attack rate; 77 features	Histogram binning + leaf-wise growth scale efficiently on large, well-populated data, reaching sharp decision boundaries without the overfitting risk that a thinner minority class would create	F1 0.9971 largest winning margin of the three datasets
HQC-2026	Smallest, most severely imbalanced pool (24.0% attack rate); several attack subtypes with fewer than 700 examples	GOSS directly compensates for imbalance by preferentially retaining hard, high-gradient (often minority-class-adjacent) examples during training rather than treating every row equally	F1 0.8082 narrowest winning margin (over XGBoost), reflecting the dataset's inherent difficulty
CIC-UNSW-NB15	CICFlowMeter-engineered features with several sparse, overlapping, or mutually-exclusive flag/count columns; moderate 20.0% attack rate	EFB bundles the sparse, mutually-exclusive engineered features into fewer effective dimensions, reducing redundancy the other eleven models had to learn around individually	F1 0.9580 second-largest winning margin

The biggest margin that LightGBM enjoyed compared to all other algorithms occurred on the X-IIoTID dataset (F1 0.9971 compared to 0.9969 for Stacking and 0.9968 for XGBoost) because the features of this dataset provided all the mechanisms with a chance to work effectively. The leaf wise splitting strategy has the potential to select global optimum splits for a dataset that is very large (in this case >800,000 records) and is not too completely imbalanced. Since there would never be a shortage of represented examples to split on. But Gradient-based One-Side Sampling (GOSS) would have a lot of easy to process majority class examples to down weight and lose important signals.

On the dataset HQC-2026 with the highly imbalanced distribution and rather small number of samples per attack type, every classification model is stressed to about the same extent and the signal of the minority classes is too weak to be separated almost perfectly as on the X-IIoTID, so the margin of LightGBM is much smaller (F1 0.8082 compared to XGBoost's F1 0.8015, a difference of only 0.0067). The reason LightGBM won is the use of the GOSS mechanism that focuses on the hard cases, which gives LightGBM a greater opportunity to see attack patterns in the small data set compared to XGBoost's uniform per sample processing. But it is actually the data limitations that cause the absolute F1 score to be low, and not some mechanism's failure.

On the CIC-UNSW-NB15 dataset, the margin of LightGBM fell somewhere between the two previous values (F1 0.9580 versus 0.9566 for Voting Ensemble and 0.9564 for Stacking, with a difference of about 0.0014-0.0016), because the carefully designed CICFlowMeter feature set contained many sparse and overlapping columns (flag and count types) that could be efficiently compressed using Extremely Fast Decision Tree (EFB) compression.

Therefore, the success of LightGBM is not a random repeatable pattern, but rather an outcome of a carefully designed process: four methods are designed to address four problems in network flow tabular data type scale, imbalance and feature redundancy, and they are all independently collected datasets. The ones used in the project had at least one of these issues very pronounced, LightGBM was able to hold its own as a result.

Table 4.18 and Table 4.19 report BB84 QKD behaviour and Adaptive Risk Engine tier distributions side by side across all three datasets.

Table 4.15: Cross-Dataset BB84 QKD Simulation Results

Dataset	Clean QBER	Low-Eve QBER	High-Eve QBER	Eavesdropping Detected?
X-IIoTID	0.0000	0.0252	0.1865	Yes key discarded
HQC-2026	0.0000	0.0282	0.1888	Yes key discarded
CIC-UNSW-NB15	0.0000	0.0245	0.1857	Yes key discarded

Table 4.16: Cross-Dataset Adaptive Risk Engine Summary

Dataset	Total Flows	LOW %	MEDIUM %	HIGH %	Overhead Saved
X-IIoTID	161,850	51.5%	0.1%	48.4%	51.6%
HQC-2026	46,745	74.0%	7.5%	18.5%	81.5%
CIC-UNSW-NB15	89,583	78.4%	0.1%	21.5%	78.5%

The clean channel QBER was equal to 0.0000 and the high-eavesdrop QBER varied between 0.1857 and 0.1888 for the three different datasets. Indicating a difference of only about 0.003 across three different datasets with three independent generations of simulated BB84 experiments thus demonstrating. Then the 11% security threshold is reached in a consistent and reliable way regardless of the network traffic generation mechanism. Due to the fact that the BB84 experiment simulation is itself independent of the dataset. Overhead reduction, on the other hand, can be highly different (51.6% to 81.5%), being determined by the respective attack rates within each dataset. Since the X-IIoTID data set has almost balanced attack rate of 48.7%. So there is less room for the LOW tier to absorb traffic than in the case of HQC-2026 (24.0%) where the LOW tier absorbs most of the benign traffic.

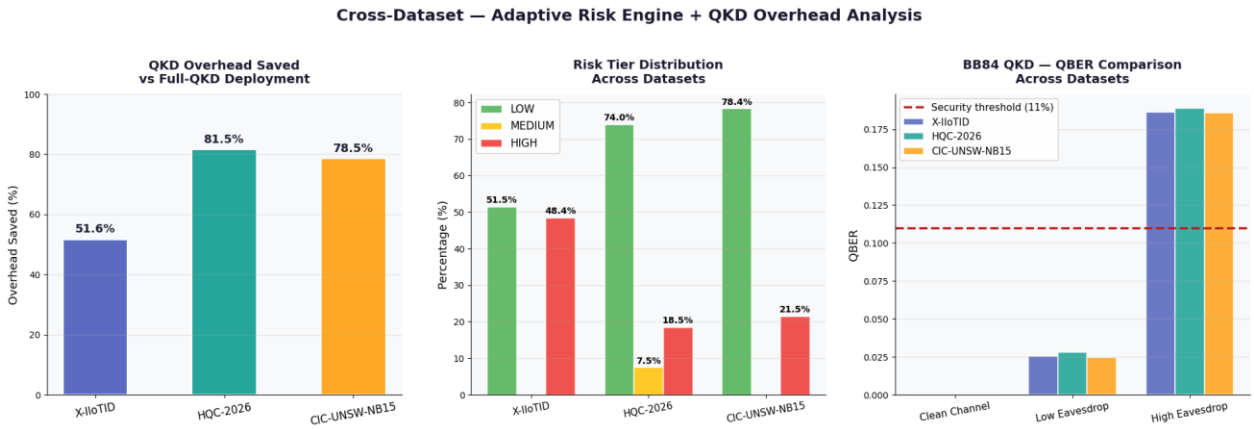


Figure 4.27: Cross-Dataset QKD Overhead and Risk-Engine Comparison

SHAP Feature Importance Comparison

Feature Consistency was one of the main observations from the results. The most prominent SHAP feature in both HQC-2026 and CIC-UNSW-NB15 the two datasets which used the CICFlowMeter for flow feature extraction was Fwd Seg Size Min, whereas X-IIoTID (which used the IIoT protocol level for instrumentation) employed Avg num cswch/s as its most influential feature. This suggests that feature consistency in different datasets may actually depend not on similar attacks but rather on similar feature extraction methods used for each dataset, which should be considered in a future attempt to create a unified model.

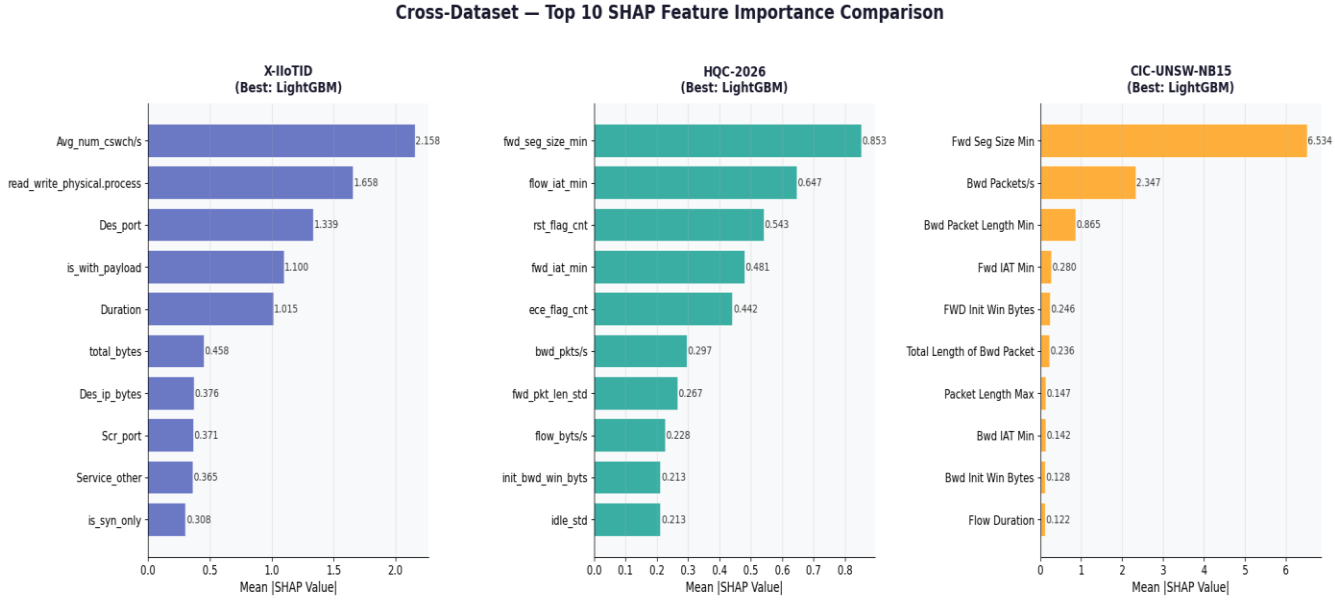


Figure 4.28: Cross-Dataset SHAP Feature Importance Comparison

Additional Cross-Dataset Visualizations

The radar chart, precision-recall curves, confusion matrices, and threshold sensitivity analysis below summarize the three best models' behaviour along several additional axes simultaneously.

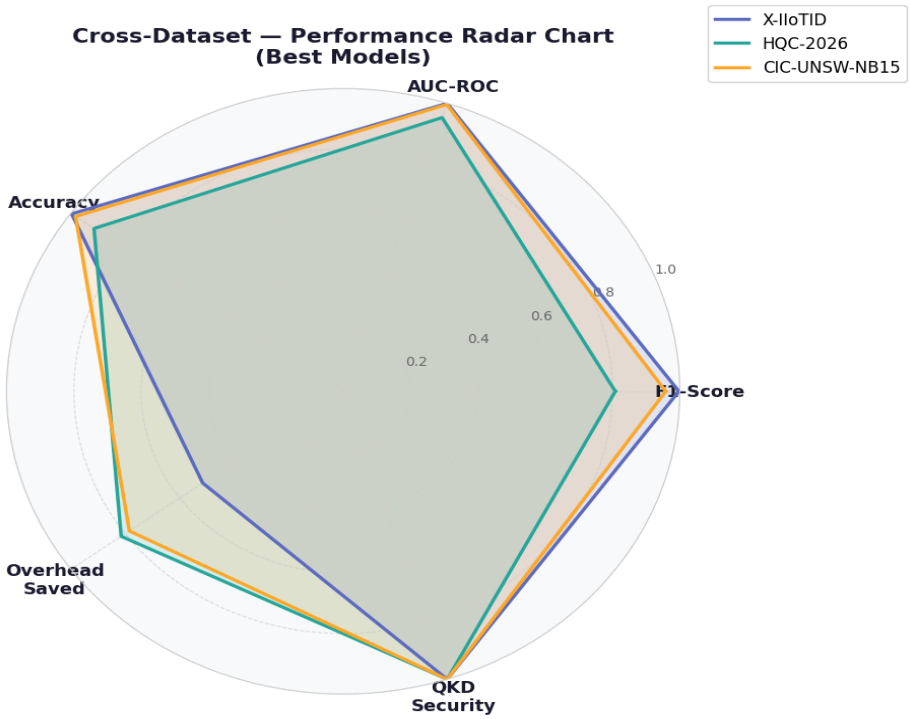


Figure 4.29: Cross-Dataset Radar Chart of Best Models

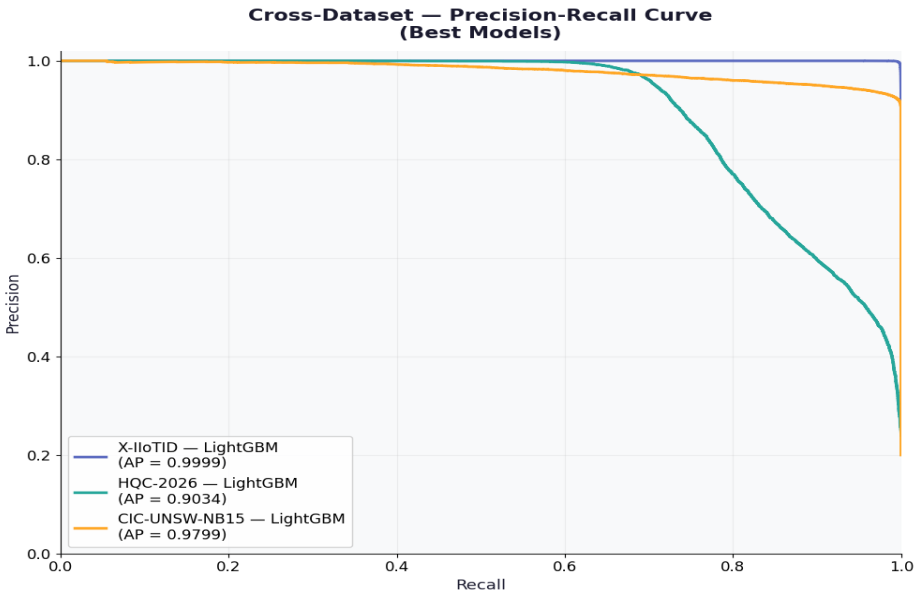


Figure 4.30: Cross-Dataset Precision-Recall Curves (Best Models)

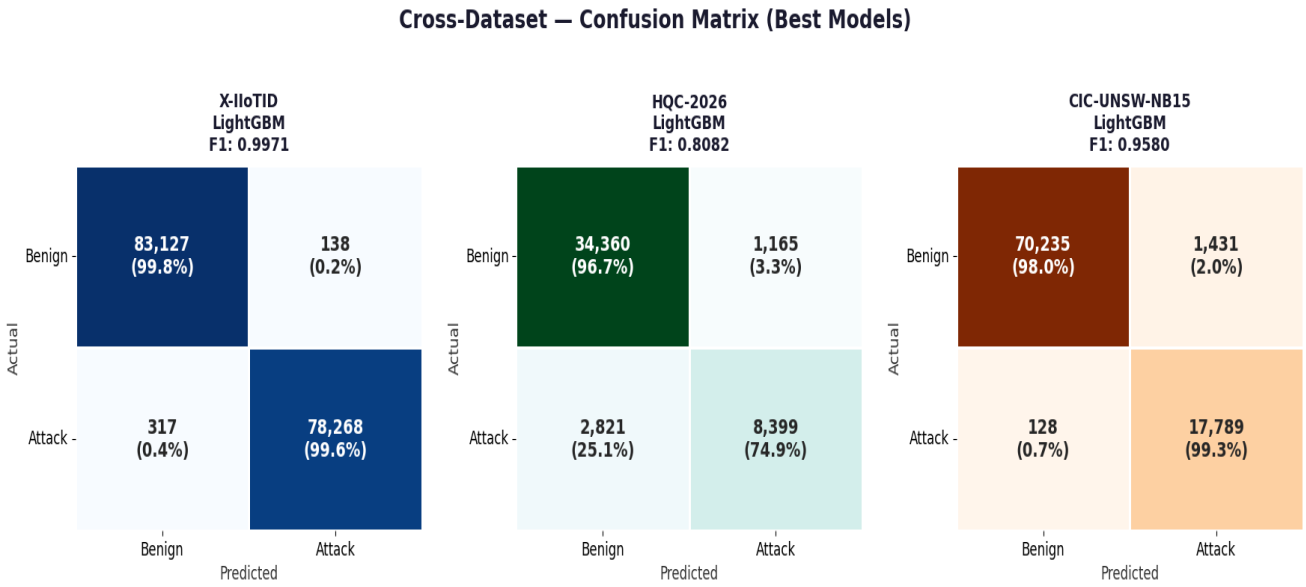


Figure 4.31: Cross-Dataset Confusion Matrices (Best Models)

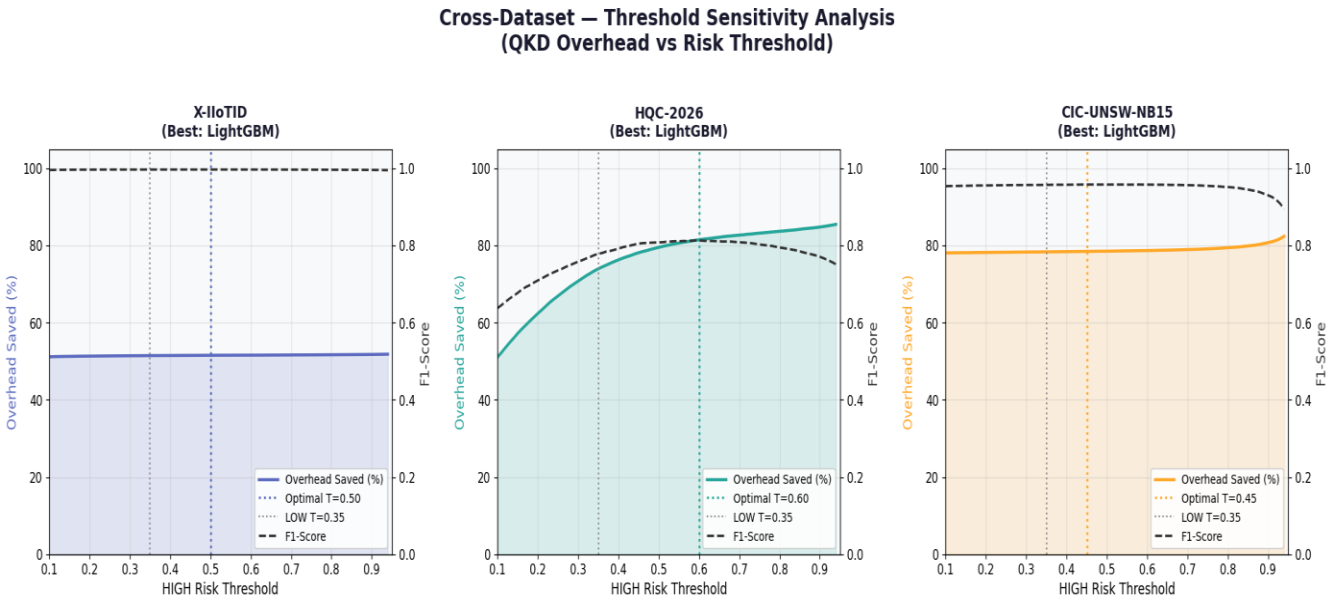


Figure 4.32: Cross-Dataset Threshold Sensitivity Analysis

The sensitivity analysis at the threshold level proves that each selected threshold value (0.50 for X-IIoTID, 0.60 for HQC-2026, and 0.45 for CIC-UNSW-NB15) is located relatively near the F1-optimal point along the respective dataset's precision-recall graph, which means that a simple threshold fine-tuning for each particular dataset (rather than using the universal 0.35/0.70 risk-tier thresholds described above) is still an appropriate approach.

4.5 Impact of Societal, Health, Safety and Ethical Issues

Testing the ML QKD model in three different countries increases the validity of the social arguments made in favor of its use in a prototype. Testing that demonstrates that overhead reductions and model selection can withstand independent testing makes the model a more viable candidate than a single test.

Societal Impact

The confidence level of people in IoT security solutions has been undermined again and again owing. This significant cybersecurity attacks which are famous 2016 Mirai botnet attack and 2021 Verkada cameras hack. These two instances and now there is an objective way to restore this trust through the proven concept of a generalized ML supported QKD solution.

Moreover, this particular solution has an equity side to it, since the universal adoption of QKD requires a lot of money, which not everyone can afford. But this approach saves up to 51.6% to 81.5% of money needed for overheads.

Healthcare Applications and Impact

Physiologically sensitive information that is carried by Healthcare IoT.Such as implantable cardiac monitors, insulin pumps, remote patient monitoring software, and hospital network infrastructure. It has the potential to be used in discrimination by insurers, identity theft, or even direct patient harm. The reason for using such a threshold-based system is the flexibility of the risk tiers used in this solution since in the case of healthcare implementation, the engine can be set up to put all flows. Which carry either patient data or commands for devices to the HIGH tier automatically to provide BB84 protection to these classes of data.

Safety Considerations

IoT systems in the areas of power production, water treatment, and transportation are examples of safety critical attack surfaces. Which are not only exposed to data theft but also pose a threat of real damage as a result of a breach. For instance, the famous 2021 attack on Oldsmar's water treatment plant saw the attacker raise sodium hydroxide to lethal levels through the SCADA interface that was connected to the Internet.

The QBER-based key rejection scheme tested on all three datasets of this project comes with its own protection. If the BB84 protocol is violated and the key rejected, this means there will be no possibility of damage since the system itself prevents that from happening.

Legal and Regulatory Considerations

The first post-quantum cryptographic standard (CRYSTALS Kyber and CRYSTALS Dilithium) by NIST was completed in July 2022, and the EU AI Act (2024) demands that all high-risk AI systems running on sensitive security infrastructures should be transparent, auditable, and subject to human supervision. The explainability analysis based on SHAP of this project for all three datasets in the same way is done to meet this auditability requirement rather than running as a black box.

Ethical Considerations

The columns of the HQC-2026 dataset generated from quantum channels (detected and excluded in section 3.1.3) serve as an example of how easy it is to include a ground-truth label in a dataset through the same data-generating process. This report describes in sufficient technical detail the problem in order for others to replicate and protect themselves from the issue in their own datasets. Similar to the previous one-dataset experiment, the trained classifiers used in this work could potentially be reverse engineered in order to build an evasive traffic.The danger is reduced by providing no information on model weights or any evasion strategy.

4.6 Discussion

In summary, the findings of the three datasets collectively provide an affirmative response to the primary research question posed in this project. Here particular one best model LightGBM, fixed risk-tier thresholds and BB84 simulation logic, the ML-QKD framework is generalizable in the sense that it is capable of yielding correct and meaningful results on three independently gathered datasets from different countries. However, the scale of gains obtained through the use of the framework is not a fixed number the amount of QKD overhead reduction varies between 51.6% and 81.5%. When the multiclass weighted F1 varies between 0.7996 and 0.9881 based on attack prevalence and class imbalance respectively. For future use of the framework, its qualitative reliability can be expected consistently a functional fail-safe through BB84.This reasonable risk tier classification and high-quality LightGBM classifier while the numerical values will have to be measured on a case-by-case basis for every independent traffic source.

The inconsistency found between LightGBM and Stacking in the stability test stage with CIC-UNSW-NB15 data serves as an interesting methodological result in its own right: it proves that any conclusions reached based on small, computationally less demanding sample of the test data should be regarded as preliminary and that even such computationally.

4.7 Summary

In this chapter, model performance, sampling stability, SHAP explanation, Adaptive Risk Engine dynamics, BB84 QKD simulation results, and multi-class attack type response were described separately for X-IIoTID, HQC-2026, and CIC-UNSW-NB15 before all three datasets were considered altogether in a specific cross-dataset comparison. LightGBM proved to be the most efficient binary classifier on the combined 150,000 rows training dataset for each dataset.BB84 QKD was able to distinguish between the completely clear channel and the one highly eavesdropped on every dataset. While the top SHAP feature was similar for the two datasets obtained from CICFlowMeter. Overhead reduction and multiclass performance depended considerably on the balance of the classes in each particular dataset, proving once again that although the numerical value is changing, the mechanism works universally.

Chapter 5

Conclusion

This thesis presents a hybrid ML-QKD framework that combines intelligent network attack detection with quantum-secure key distribution. The ML models effectively identify and assess network risks, while BB84 QKD provides secure key distribution for high-risk communication and AES-256 ensures strong data encryption. Overall, the proposed framework offers a promising approach to enhancing network security against both current and future cyber threats.

5.1 Overall Contributions

This capstone project extended the single-dataset ML-QKD proof of concept into an adaptive tri-dataset network security framework using twelve ML models. The three datasets are X-IIoTID, HQC-2026, and CIC-UNSW-NB15. LightGBM achieved the best overall binary classification performance.While the Adaptive Risk Classification Engine reduced BB84 QKD overhead by 51.6%, 81.5%, and 78.5%, respectively, while protecting high-risk connections.A separate multi-class model for detecting attack type with hybrid under sampling and SMOTE technique for rebalancing, combined with an Attack-Type-Aware Response Recommendation Engine extended the framework from risk-tiering classification to provide actual security actions depending on the attack type on each dataset.

The BB84 simulation remained stable across all datasets, with clean-channel QBER of 0.0000 and reliable key rejection under heavy eavesdropping. SHAP analysis also identified important features consistently across datasets. Additionally, the Attack-Type-Aware Response Engine provided security actions based on detected attack types. The study also demonstrated the importance of checkpoint-friendly training and model stability validation before final model selection.

5.2 Future Works

Future work will focus on broader dataset validation, improved scalability, and real-world deployment of the proposed framework.

1.Real Quantum Hardware Integration: replace the software BB84 simulation with an actual photonic QKD system (e.g., ID Quantique hardware) to validate whether the simulated QBER behaviour observed across all three datasets holds under real optical-channel noise.

2.Per-Dataset Threshold Auto-Tuning: Extend the fixed 0.35/0.70 risk-tier thresholds into an auto-calibrated scheme that adjusts to each new dataset's baseline attack building on the threshold sensitivity analysis performed in Chapter 4.

3.A Fourth and Fifth Validation Dataset: Add further geographically and technologically distinct traffic sources (for example, a 5G/6G network dataset and a healthcare-specific IoT dataset) to further stress-test the generalization claim made in this report.

4.Cross-Dataset Transfer Learning: investigate whether a LightGBM model pretrained on one dataset (e.g., X-IIoTID) can be fine-tuned rather than retrained from scratch on a new dataset (e.g., HQC-2026), potentially reducing the twelve-model training cost for future deployments.

5.Live Deployment and Streaming Evaluation: move from offline held-out test evaluation toward a live, streaming IoT network deployment with a real-time dashboard, to measure inference latency and operational overhead under production conditions.

5.3 Limitations

This thesis has some limitations that should be considered when interpreting the results.

1.Software-Only BB84 Simulation: As with the earlier prototype, no real quantum hardware was used. All QBER and key-generation results reflect the fidelity of the simulation logic rather than physical optical-channel behaviour.

2.Fixed Risk-Tier Thresholds: The 0.35/0.70 thresholds were applied identically across all three datasets without per-dataset optimization. Which the threshold sensitivity analysis in Chapter 4 suggests leaves some accuracy on the table for at least two of the three datasets.

3.Three Datasets Are Not an Exhaustive Sample: While X-IIoTID, HQC-2026, and CIC-UNSW-NB15 differ meaningfully in country, year, and instrumentation. Three datasets remain a limited basis from which to claim full generalization.The consistent LightGBM result and consistent BB84 behaviour are encouraging but not conclusive evidence across the full diversity of real-world IoT traffic.

By addressing these limitations, future work can evolve this framework from a validated research prototype into a deployable, production-grade decision-support system for post-quantum IoT network security.

Bibliography

[1] IoT Analytics, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," IoT Analytics, Sep. 2024. https://iot-analytics.com/number-connected-iot-devices/

[2] Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," IEEE Commun. Surveys Tuts., vol. 17, no. 4, pp. 2347–2376, 2015, doi: 10.1109/COMST.2015.2444095. https://doi.org/10.1109/COMST.2015.2444095

[3] V. Hassija, V. Chamola, V. Saxena, D. Jain, P. Goyal, and B. Sikdar, "A survey on IoT security: Application areas, security threats, and solution architectures," IEEE Access, vol. 7, pp. 82721–82743, 2019, doi: 10.1109/ACCESS.2019.2924045. https://doi.org/10.1109/ACCESS.2019.2924045

[4] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in Proc. 26th USENIX Security Symp., Vancouver, BC, 2017, pp. 1093–1110. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

[5] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM J. Comput., vol. 26, no. 5, pp. 1484–1509, 1997, doi: 10.1137/S0097539795293172. https://doi.org/10.1137/S0097539795293172

[6] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?," IEEE Secur. Privacy, vol. 16, no. 5, pp. 38–41, 2018, doi: 10.1109/MSP.2018.3761723. https://doi.org/10.1109/MSP.2018.3761723

[7] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in Proc. IEEE Int. Conf. Computers, Systems and Signal Processing, Bangalore, India, 1984, pp. 175–179, doi: 10.1016/j.tcs.2014.05.025. https://doi.org/10.1016/j.tcs.2014.05.025

[8] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, "The security of practical quantum key distribution," Rev. Mod. Phys., vol. 81, no. 3, pp. 1301–1350, 2009, doi: 10.1103/RevModPhys.81.1301. https://doi.org/10.1103/RevModPhys.81.1301

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in Proc. 31st USENIX Security Symp., 2022, pp. 3971–3988. https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953 https://doi.org/10.1613/jair.953

[11] P. Sengupta, M. Raikwar, S. Maharjan, F. Eliassen, and Y. Zhang, "Adaptive quantum-safe cryptography for 6G vehicular networks via context-aware optimization," arXiv, 2026, doi: 10.48550/arXiv.2602.01342. https://arxiv.org/abs/2602.01342

[12] Al-Kuwari, N. Mohamed, S. Al-Kuwari, A. Farouk, and B. K. Behera, "Resisting quantum key distribution attacks using quantum machine learning," IET Quantum Commun., p. e70028, 2026, doi: 10.1049/qtc2.70028. https://ietresearch.onlinelibrary.wiley.com/doi/pdfdirect/10.1049/qtc2.70028

[13] N. Mohamed and S. Al-Kuwari, "Adversarial learning game for intrusion detection in quantum key distribution," arXiv, 2026, doi: 10.48550/arXiv.2603.03502. https://arxiv.org/pdf/2603.03502

[14] D. Chaudhary, S. Rajasegarar, and S. R. Pokhrel, "Towards adapting federated & quantum machine learning for network intrusion detection: A survey," arXiv, 2025, doi: 10.48550/arXiv.2509.21389. https://arxiv.org/pdf/2509.21389?

[15] V. Aandaram, P. Deepalakshmi, and V. P. Rajendran, "Quantum-assisted hybrid security framework for secure authentication with intrusion detection for 5G networks," J. Theor. Appl. Inf. Technol., vol. 104, no. 1, pp. 455–475, 2026. : https://www.jatit.org/volumes/Vol104No1/32Vol104No1.pdf

[16] H. Zangana, A. B. Sallow, and F. M. Mustafa, "Quantum-assisted architectures for 6G and IoT: A framework for secure and efficient wireless networks," Inform: J. Ilmiah Bid. Teknol. Inf. Komun., vol. 10, no. 2, pp. 152–158, 2025, doi: 10.25139/inform.v10i2.10436. https://ejournal.unitomo.ac.id/index.php/inform/article/view/10436/4877

[17] M. K. Mounagurusamy, A. A. K. Reddy, C. M. Velu, G. V. Nirmala, D. Arivazhagan, M. M. Adnan, K. Rahmaan, and T. Prabhakaran, "Adaptive and quantum-resilient intrusion detection for wireless sensor networks and IoT environments," Eng., Technol. Appl. Sci. Res., vol. 15, no. 4, pp. 24723–24728, 2025, doi: 10.48084/etasr.10464. https://mail.etasr.com/index.php/ETASR/article/view/10464/5216

[18] N. R. R. Reddy, S. Suryadevara, K. G. R. Reddy, R. Umamaheswari, R. Guttula, and R. Kotoju, "Quantum secured blockchain framework for enhancing post quantum data security," Sci. Rep., vol. 15, p. 31048, 2025, doi: 10.1038/s41598-025-16315-8. [Online].https://www.nature.com/articles/s41598-025-16315-8.pdf

[19] R. Gunasridharan, A. Altalbe, B. M. Gurusamy, G. Pallavi, and P. K. Rangarajan, "A hybrid framework integrating QML, AI, and quantum-safe cryptography for cybersecurity," J. Comput. Cogn. Eng., vol. 5, no. 1, pp. 16–29, 2026, doi: 10.47852/bonviewJCCE52025121. https://ojs.bonviewpress.com/index.php/JCCE/article/view/5121/1568

[20] H. Biswas and K. E. Zoiros, "Power network SCADA quantum communications: A comparison of BB84, B92, E91, and SARG04 quantum key distribution protocols," arXiv, 2026, doi: 10.48550/arXiv.2603.01060. https://arxiv.org/pdf/2603.01060

[21] Z. Kaleem, M. U. Khan, A. Suleman, W. Khalid, K.-K. Wong, and C. Yuen, "Quantum Skyshield: Quantum key distribution and post-quantum authentication for low-altitude wireless networks in adverse skies," IEEE Wireless Commun., vol. 33, no. 2, pp. 235–243, 2026, doi: 10.1109/MWC.2025.3602373. https://arxiv.org/pdf/2507.14822

[22] F. K. Mupila and A. Bhardwaj, “Quantum-resilient cloud forensics: A hybrid QKD-AI framework for post-quantum evidence integrity,” in Proc. Int. Conf. Advancement Cyber Security Digital Forensics, 2026, doi: 10.1007/978-981-95-3701-3_37. https://www.researchgate.net/profile/Francis_Mupila/publication/400103432_Quantum-Resilient_Cloud_Forensics_A_Hybrid_QKD-AI_Framework_for_Post-quantum_Evidence_Integrity/links/69b885c15ba285703c8929d4/Quantum-Resilient-Cloud-Forensics-A-Hybrid-QKD-AI-Framework-for-Post-quantum-Evidence-Integrity.pdf

[23] A. Al-Khazraji, A. M. Ali, T. T. Abdulridha, M. M. A. Rijab, and M. H. Al-Janabi, “Secure quantum key distribution over VLC networks for next-generation smart cities,” Int. J. Netw. Distrib. Comput., vol. 13, p. 33, 2025, doi: 10.1007/s44227-025-00077-w. https://link.springer.com/content/pdf/10.1007/s44227-025-00077-w.pdf

[24] K. Revathi and K. Suganthi, “Advanced quantum key distribution protocol for mitigating quantum-based vulnerabilities in blockchain applications,” EPJ Quantum Technol., vol. 12, p. 1, 2025, doi: 10.1140/epjqt/s40507-025-00442-2. https://link.springer.com/content/pdf/10.1140/epjqt/s40507-025-00442-2_reference.pdf

[25] O. Shirko and S. Askar, “A novel security survival model for quantum key distribution networks enabled by software-defined networking,” IEEE Access, vol. 11, pp. 21641–21650, 2023, doi: 10.1109/ACCESS.2023.3251649. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10057401

[26] M. G. Majumdar, “Quantum-secured 6G networks: Integrating quantum machine learning and beyond-QKD technologies,” ResearchGate, 2025, doi: 10.13140/RG.2.2.21975.79529. https://www.researchgate.net/profile/Mrittunjoy_Guha_Majumdar2/publication/393514383_Quantum-Secured_6G_Networks_Integrating_Quantum_Machine_Learning_and_Beyond-QKD_Technologies/links/68b084b17984e374acec2837/Quantum-Secured-6G-Networks-Integrating-Quantum-Machine-Learning-and-Beyond-QKD-Technologies.pdf

[27] R. Madhubala and P. K. Bolisetty, “An energy-efficient AI-assisted quantum-secure communication framework for scalable and reliable IoT networks,” J. Wireless Netw. Commun. Syst., vol. 2, no. 2, 2026, doi: 10.32595/jwncs/v2i2.2026.28. https://jwncs.melangepublications.com/index.php/jwncs/article/view/32/34

[28] Rehman and O. Alharbi, "QESIF: A lightweight quantum-enhanced IoT security framework for smart cities," Smart Cities, vol. 8, no. 4, p. 116, 2025, doi: 10.3390/smartcities8040116. https://www.mdpi.com/2624-6511/8/4/116

[29] Rubio García, A. Cano Aguilera, C. Stan, J. J. Vegas Olmos, S. Rommel, and I. Tafur Monroy, "Enhanced network security protocols for the quantum era: Combining classical and post-quantum cryptography, and quantum key distribution," IEEE J. Sel. Areas Commun., vol. 43, no. 8, pp. 2765–2781, 2025, doi: 10.1109/JSAC.2025.3568011. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11002706

[30] M. Malik and A. Alsanoosy, "Hybrid intrusion detection framework combining peer-to-peer networking and quantum machine learning for smart traffic systems," 2024. https://journal.uob.edu.bh/items/9ac190eb-e06b-4706-8d31-2c02bac6ec7e

[31] V. B. Chalampalem, S. Nagaraju, R. K. Kumar, and T. Santhivandana, "Quantum machine learning enhanced SDR architecture for adaptive and secure wireless communication using OQAM," Research Square, 2025, doi: 10.21203/rs.3.rs-7239745/v1. https://assets-eu.researchsquare.com/files/rs-7239745/v1/3a1924b4-bbf0-4336-a130-dd4be85402cb.pdf

[32] K. Purohit and A. K. Vyas, "Quantum key distribution through quantum machine learning: A research review," Front. Quantum Sci. Technol., vol. 4, p. 1575498, 2025, doi: 10.3389/frqst.2025.1575498.https://www.frontiersin.org/journals/quantum-science-and-technology/articles/10.3389/frqst.2025.1575498/full

[33] Shukla et al., "Quantum-enhanced cybersecurity framework for Internet of Things environments," 2026 https://link.springer.com/content/pdf/10.1007/s42452-025-08107-6.pdf

[34] R. K. Inakoti, M. J. Stephen, and P. V. G. D. P. Reddy, "Enhancing quantum cryptography with machine and deep learning: A hybrid approach for secure and scalable post-quantum security," J. Theor. Appl. Inf. Technol., vol. 103, no. 11, 2025. https://www.jamesstephen.in/papers/Ravi%20inakoti.pdf

[35] B. Mallick, P. Parida, B. Prasad, C. Nayak, M. K. Panda, N. Ali, and N. M. Kumar, “Quantum-secure communication for future cyber-physical and IoT systems: A systematic review of classical to learning approaches,” Computers, vol. 15, no. 6, p. 389, 2026, doi: 10.3390/computers15060389. https://www.mdpi.com/2073-431X/15/6/389

[36] Komal and S. Li, “Intrusion detection in the Internet of Things: A comprehensive review of techniques, architectures, datasets, and emerging trends,” Sensors, vol. 26, no. 11, p. 3405, 2026, doi: 10.3390/s26113405. https://www.mdpi.com/1424-8220/26/11/3405

[37] O. E. Akinbowale, A. T. Adesina, M. F. Zerihun, and P. Mashigo, “Machine learning based approach to intrusion detection in internet of things environments,” Front. Artif. Intell., vol. 9, p. 1760137, 2026, doi: 10.3389/frai.2026.1760137. https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1760137/full

[38] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. https://doi.org/10.1023/A:1010933404324

[39] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785. https://doi.org/10.1145/2939672.2939785

[40] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Stat., vol. 29, no. 5, pp. 1189–1232, 2001. https://www.jstor.org/stable/2699986

[41] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018. https://doi.org/10.1007/BF00994018

[42] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 3146–3154. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015. https://arxiv.org/abs/1412.6980

[44] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 31, 2018, pp. 6638–6648. https://arxiv.org/abs/1706.09516

[45] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," Mach. Learn., vol. 63, no. 1, pp. 3–42, 2006, doi: 10.1007/s10994-006-6226-1. https://doi.org/10.1007/s10994-006-6226-1

[46] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," J. Comput. Syst. Sci., vol. 55, no. 1, pp. 119–139, 1997, doi: 10.1006/jcss.1997.1504. [Online]. https://doi.org/10.1006/jcss.1997.1504

[47] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proc. 28th Annu. ACM Symp. Theory of Computing (STOC), 1996, pp. 212–219, doi: 10.1145/237814.237866. https://doi.org/10.1145/237814.237866

APPENDIX

Mapping of Course and Program Outcomes

ICE 496-C

CO	Objective	Key Knowledge (K)	Engineering Practice (EP)	Description
CO1	Extend a single-dataset ML-security prototype into a validated, cross-dataset framework for the capstone project.	K1, K2, K3	EP1, EP2, EP3	We integrated new and previously acquired knowledge of ML-based intrusion detection and quantum key distribution to identify and address the real-life engineering problem of validating security-system generalization across independent network-traffic sources.
CO2	Design and evaluate a checkpoint-safe, stability-verified training and comparison methodology across multiple datasets.	K3, K4, K5	EP2, EP4, EP5	We designed a non-overlapping stratified sampling and stability-check pipeline, trained twelve classifiers on three independent datasets, and evaluated results against F1, AUC, and QKD-overhead metrics to validate the methodology's robustness.

CO	Objective	Key Knowledge (K)	Engineering Practice (EP)	Description
CO3	Develop an adaptive, quantum-assisted security response system with explainable, attack-type-aware recommendations.	K5, K6, K7	EP4, EP5, EP6, EP7	We implemented the Adaptive Risk Engine, the BB84 QKD simulation, SHAP explainability, and the Attack-Type-Aware Response Recommendation Engine, and assessed their societal, health, safety, and ethical implications across all three validated datasets.

The mapping is based on the same three COs that are employed in the CSE 400 A/B/C capstone series of the department; in this case, it applies to an extension of the particular project in the form that the CSE 400-B phase was devoted to the validation of ML-QKD design in one dataset, whereas the CSE 400-C phase mapped out above was to prove that the same design works in new datasets, which is quite a different engineering achievement.